SWE5006 PRACTICE PROJECT · TEAM 8

# BLOCBUY

ONE FOR ALL • ALL FOR ONE

## A community group-buying platform with pay-first teams

*Two prices: solo & team* ● *Pay first to team up* ● *Full refund if the team size is not met*

1 Introduction 2 Analysis, design, patterns 3 Testing 4 CI/CD demo 5 System demo

Lim Jiechao Liu Jin Panpakorn Siripanich Wang Ziang Yu Qianshuo

Graduate Certificate in Designing Modern Software Systems, 26 October 2026

# Group buying fails when buyers cannot commit together

> **Small merchants can discount only against committed volume, but no single buyer can reach it: demand exists, but no buyer commits first.**

## Today

- Merchants discount only when volume is predictable
- One buyer can never reach a bulk price
- Advertising is costly and not tied to real purchases

## BlocBuy: pay-first teams

- Two prices: solo and team
- Pay upfront to start a team, then share the invite link
- Friends join by paying upfront too
- Fills in time: team price. Expires: everyone refunded automatically

*Stripe test mode only*

# Scope: nine use cases, six quality attributes with targets

## IN SCOPE · PROPOSAL §6.1

**UC1** Browse and view listings
**UC2** Register / authenticate
**UC3** Initiate a team (pay first)
**UC4** Join a team
**UC5** Team lifecycle resolution
**UC6** Refund on team expiry
**UC7** Share attribution
**UC8** View my orders and teams
**UC9** Admin listing management

*+ Receive notifications (Sprint 4, SWE5001; off on dev)*

## OUT OF SCOPE · PROPOSAL §6.2

| | |
|---|---|
| Merchant self-service | Analytics |
| Recommendations | Automatic team matching |
| Email, SMS, push | Native apps |
| Live payments | |

## QUALITY ATTRIBUTES AND TARGETS · PROPOSAL §6.3

| Attribute | Target | Shown in |
|---|---|---|
| **Correctness under concurrency** | Team capacity never exceeded<br>No payment double-charged or double-refunded | **§5 demo**<br>**§3** |
| **Performance** | Interactive operations ≤ 2 s at 100 concurrent users<br>Expiry resolved ≤ 1 min after the window | **§5 k6 run**<br>**§3** |
| **Security** | Google sign-in<br>Role-based admin<br>Signed Stripe webhooks<br>No credentials in code | **§5 demo**<br>**§4** |
| **Maintainability, extensibility** | New pricing, payment provider or team consequence without editing domain logic | §2 patterns<br>§4<br>§5 |
| **Code quality** | SonarCloud gate on every PR<br>Unit tests on the complex classes | §3<br>§4 |
| **Auditability** | Every payment, refund and transition traceable<br>Every artefact traceable to a commit | §4 |

# Promised, delivered, changed, dropped: status on 24 October

DELIVERED **UC1–UC9**
All nine closed as completed

OPEN **Crash recovery on dev not shown**
Paid start, join and refund shown on dev 23–24 Oct

DROPPED **Gated prod, ECS circuit breaker**
For SWE5006; built for SWE5001

PARTIAL **E2E, burndown, saved plans**
Stubs; daily data, no chart; manual

**OUTSTANDING**

SSH port 22 open on the dev host
CloudFront → ALB over HTTP until 16 Nov
Dev volume unencrypted

| Changed: promised | Built |
|---|---|
| **UC9 admin API only, no GUI** | Admin console, built in Sprint 3 |
| **Adapter (UC6), Observer (UC7)** | Gateway ports; Observer on UC5 → UC6 |
| **State for the team lifecycle** | Transition table; State on listings |
| **Google sign-in with JWT** | Sa-Token session, HttpOnly cookie |
| **Spring Data JPA** | MyBatis behind repository ports |
| **Dev on ECS Fargate and RDS** | One EC2 host, Compose, database on the host |
| **Secrets Manager, no static keys** | SSM and a root-only env file; 2 IAM users, 1 key |
| **Blocking OWASP and IaC scans** | Trivy, not required; 3 per-job roles |

Six of these eight changes have no ADR yet.

# Milestones achieved and effort per member

## MILESTONES ACHIEVED

| Date | Milestone |
|---|---|
| **27 Aug** | Proposal submitted |
| **5 Sep** | Backend pipeline live on dev |
| **14 Sep** | Sprint 1 tag |
| **20 Sep** | Stripe Sandbox acceptance (local) |
| **22 Sep** | Web app live on dev |
| **27 Sep** | Sprint 2 tag |
| **30 Sep** | Admin console merged |
| **11 Oct** | Sprint 3 tag |
| **20 Oct** | SWE5001 production release on ECS |
| **23 Oct** | Paid start and refund on dev |
| **24 Oct** | Sprint 4 tag |

## EFFORT PER MEMBER

| Member | S1 | S2 | S3 | S4 | Planned h | Actual days |
|---|---|---|---|---|---|---|
| **Liu Jin** | — | 30 | 30 | 51 | **111** | **12.5** |
| **Yu Qianshuo** | — | 30 | 30 | 48 | **108** | **9** |
| **Wang Ziang** | — | 30 | 30 | 46 | **106** | **8.5** |
| **Panpakorn** | — | 30 | 30 | 41.5 | **101.5** | **11.5** |
| **Lim Jiechao** | — | 30 | 30 | 38 | **98** | **8.5** |

Sprint 1 hours were not recorded

Team done: Sprint 2 134 of 150 h; Sprint 3 117 of 150 h

Proposal budget: 10 person-days each

**Actual, ≈ 50 person-days:** the split the team agreed for the joint SWE5001 and SWE5006 project, estimated from the issues each person delivered; no one logged hours.

# Ownership: at least one use case and one design problem each

EVERY USE CASE HAS A REPORT OWNER

**UC1** Pan **UC2** Pan **UC3** Eric **UC4** Eric **UC5** Liu Jin **UC6** Yorik **UC7** Lim **UC8** Lim **UC9** Liu Jin

| Member | Use case | Design problem → GoF pattern |
|---|---|---|
| **Liu Jin** | UC9 admin listings, UC5 team lifecycle | Listing lifecycle → **State** (full form) |
| **Wang Ziang (Eric)** | UC3 start, UC4 join, pay upfront | Team creation → **Factory Method** |
| **Yu Qianshuo (Yorik)** | UC6 refund on expiry | Team outcome consequences → **Observer** |
| **Panpakorn (Pan)** | UC1 browse, UC2 sign in | Outbox transports → **Template Method** |
| **Lim Jiechao** | UC8 my orders and teams, UC7 sharing | Pricing → **Strategy** |

Each member presents their own design problem in section 2; Eric presents the team statechart, which the report places in Liu Jin's UC5.

Report: use cases §3.4.1–§3.4.5, design problems §3.5.1–§3.5.5, in section 2 order

# Four sprints: the Definition of Done reached, and what each retro changed

**Definition of Done ladder** coded → tested → integrated → releasable → released

| Sprint | Goal | Definition of Done reached by the goal increment |
|---|---|---|
| **1 · 1–12 Sep** | Walking skeleton: sign-in, browse, admin on dev | **integrated**: backend on dev 5 Sep; no frontend, so not releasable |
| **2 · 15–26 Sep** | Pay, join a team, see the order, replay-safe | **integrated**: merged, tested, Checkout off on dev; 134 of 150 h |
| **3 · 28 Sep–9 Oct** | Invite → pay → refund on deployed dev | **integrated**; goal not met under our "on dev" rule; 117 of 150 h |
| **4 · 12–23 Oct** | Complete system on dev; three services on ECS (SWE5001) | **released**: SWE5001 production on ECS, 17 and 20 Oct, Stripe test mode, up per session; SWE5006 dev **releasable**: paid start, a second buyer's join and the refund shown on dev, 23–24 Oct |

Same basis every sprint: integrated = merged to dev with every required check green; releasable = shown working on deployed dev, our "on dev" rule; released = promoted to production and verified there

## Retro improvement → change in the next sprint

| | | |
|---|---|---|
| **1 → 2** | No burndown, no hours | → Daily board snapshots, dated day logs |
| **2 → 3** | A 10,000-line PR; cliff burndown | → PRs under 1,500 lines, mid-sprint check, "on dev" rule in the DoD |
| **3 → 4** | Board estimates left empty | → Owners keep board estimates current, checked mid-sprint |

# Agile artefacts: a story card, a burndown, a practice

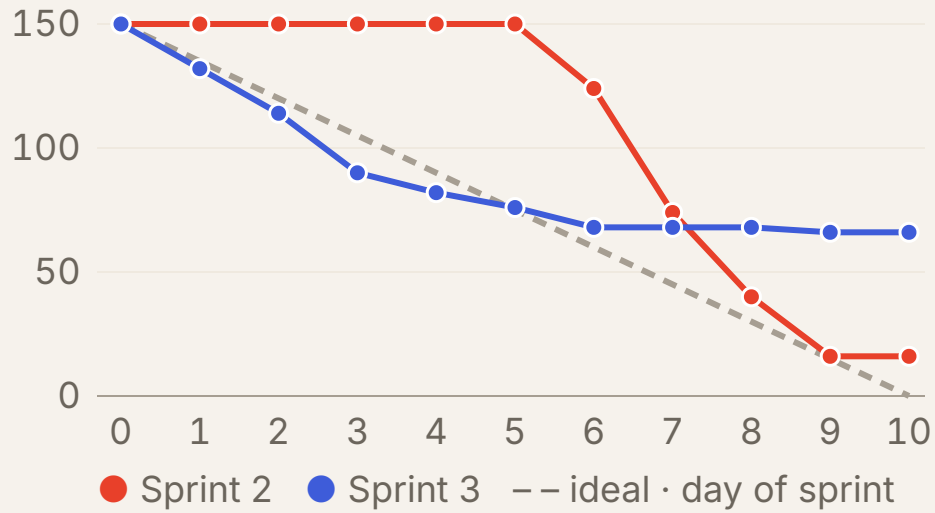
STORY CARD — Liu Jin

**As an admin, I can set a listing's team size, lifetime and team price, so that buyers can start blocs on real listings.**

- [x] Admin-only API: 403 for non-admins, 401 anonymous
- [x] Team size ≥ 2, lifetime in range, team price below individual; 400 with field errors
- [x] Stale version → 412; every write audited
- [x] A configured listing can start a team
- [x] Contract changes additive, merged before the controller
- [x] Unit, PostgreSQL and authorization tests; ArchUnit green

Design problem field: State, Optimistic Offline Lock

BURNDOWN · STORY HOURS LEFT



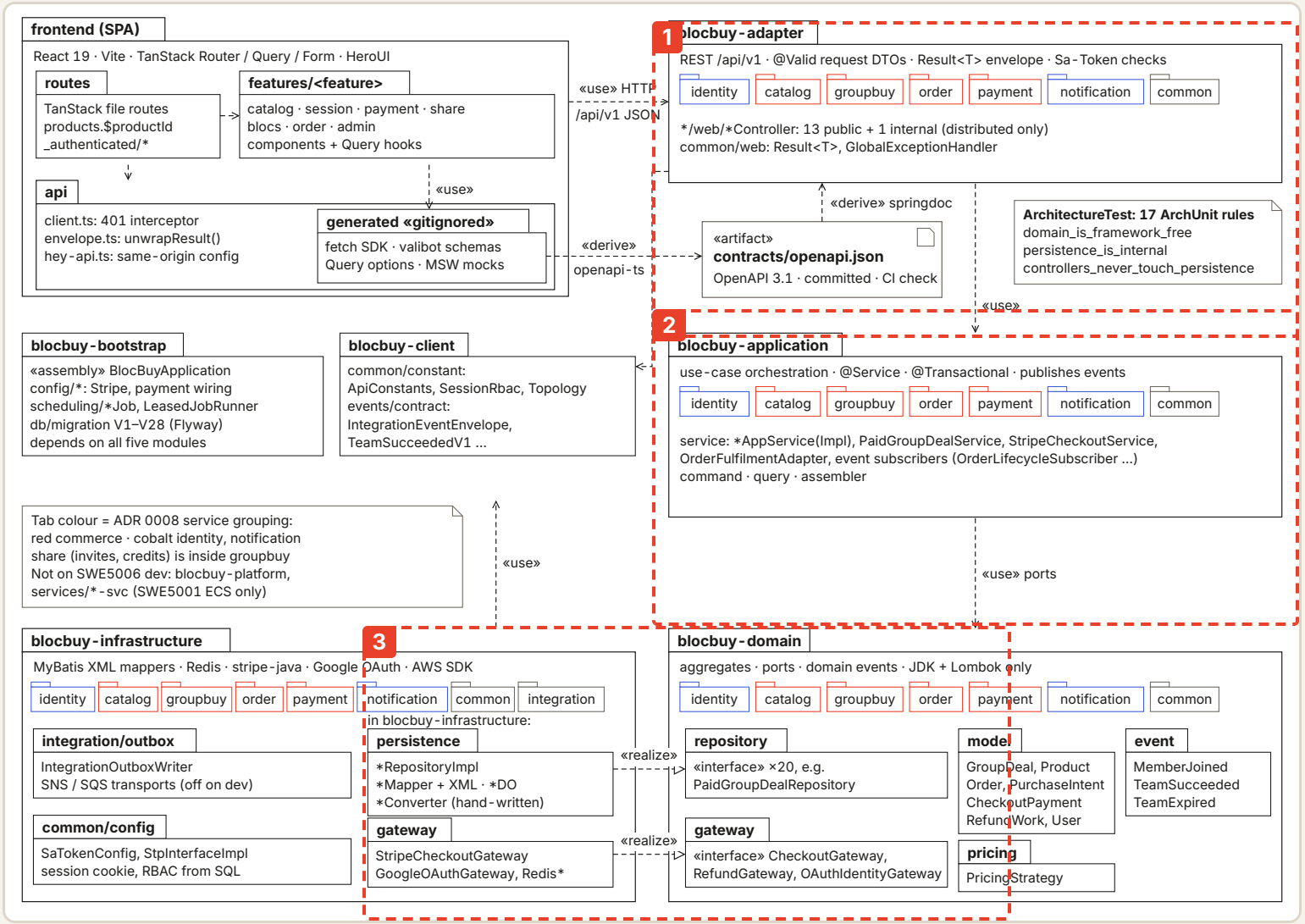

Sprint 2: flat until the first story PR on 22 Sep, then a cliff

Sprint 3, PRs kept small: burns from day 1, then stalls on Checkout in dev

Reconstructed; no hours were logged

**Test first, then refactor:** two failing tests reproduced a price-quote bug before the fix; then team admission was refactored into shared checks.

# Logical architecture: COLA layers across six bounded contexts




## DECISIONS THAT SHAPE IT

Modular monolith: COLA layers, six contexts
skeleton plan D1, D4

Framework-free domain; 17 ArchUnit rules
skeleton plan D1 · no ADR

Consequences run before the commit
ADR 0002

Cross-service events through an outbox
ADR 0007

One codebase, two topologies
ADR 0008

Router owns URL state; generated client
frontend ADR 0001, 0003

## PLATFORM

Java 21, Spring Boot 4, MyBatis, Sa-Token
PostgreSQL 18, Redis 8, Stripe Checkout
React 19, TanStack, TypeScript
AWS EC2, S3, CloudFront

# Transition strategies from analysis to design

| Strategy · concern | Object types · use cases | Become in design |
|---|---|---|
| **Language, framework**<br>maintainability | «control», «entity» · all | Spring `@Service` by constructor injection; plain Java domain |
| **Presentation**<br>usability | GUI «boundary» · UC1–UC4, UC7–UC9 | TanStack route + component + Query hook |
| **Distribution**<br>interoperability | «control» · all | `@RestController` returning `Result`; generated client as stub |
| **Persistence**<br>integrity | «entity» · all | Aggregate + repository port, implemented with MyBatis |
| **External interface**<br>provider isolation | system «boundary» · UC2–UC6 | Gateway port + Stripe / Google adapter; signed webhook |
| **Security**<br>security | every «control» · all | Login interceptor; `checkRole(ADMIN)`; CSRF header; rate limits |
| **Scheduling**<br>expiry ≤ 1 min | Scheduler actor · UC5, UC6 | `@Scheduled` job under a database lease, every minute |
| **Concurrency**<br>no oversell | «entity» with capacity · UC4 | Row locks in a fixed order + guarded UPDATE + unique indexes |
| **Domain events**<br>decoupling | «control» · UC5–UC7 | Event record + transactional listener; outbox across services |

# Typical changes: class and sequence diagrams

## CLASS DIAGRAMS: EACH ANALYSIS TYPE BECOMES DESIGN CLASSES

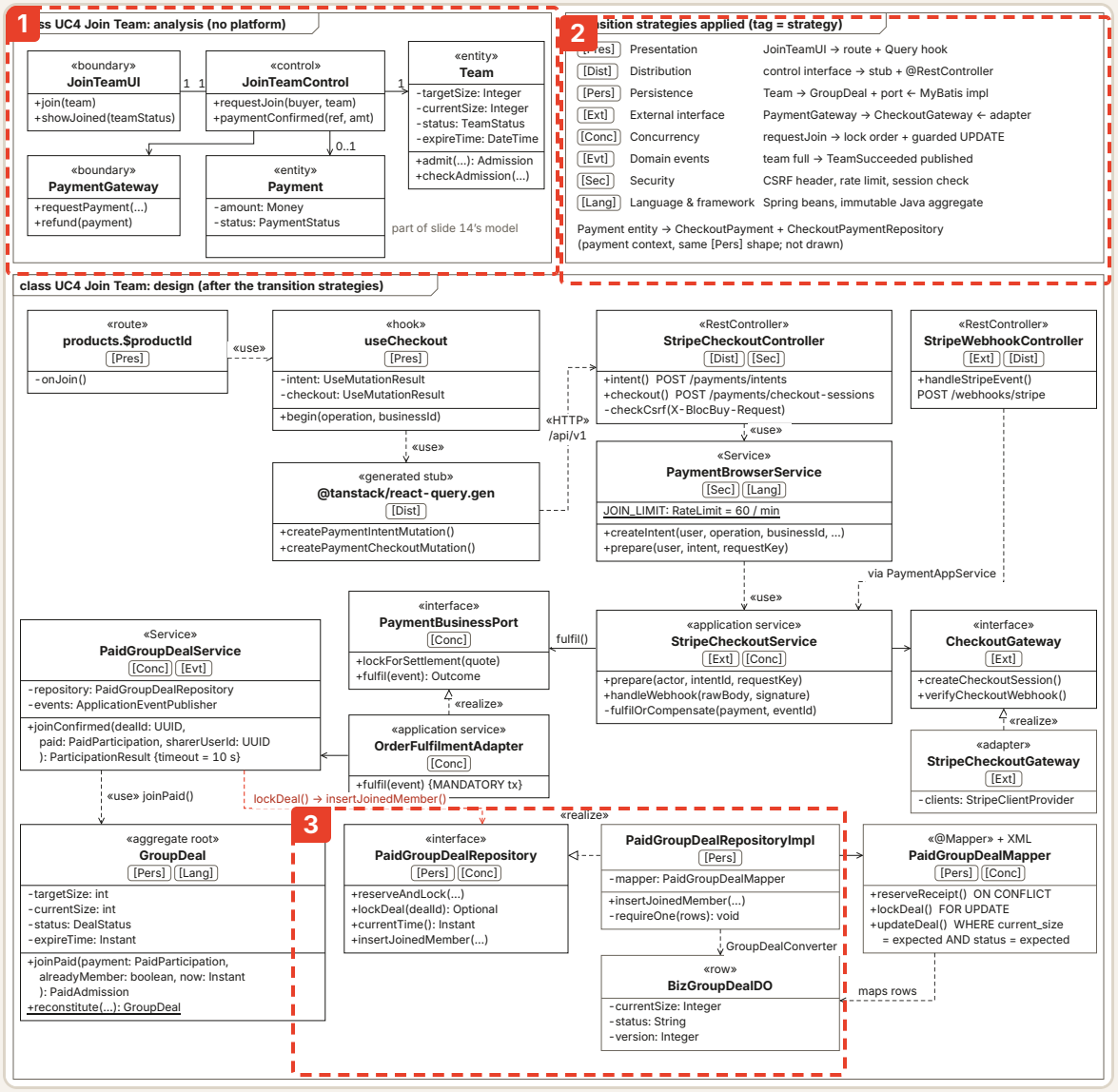
| | | |
|---|---|---|
| **GUI «boundary»** | → | Route, component and Query hook |
| **«control»** | → | REST controller + application service, called through the generated client |
| **«entity»** | → | Aggregate behind a repository port, implemented with MyBatis |
| **External «boundary»** | → | Gateway port + adapter |

## SEQUENCE DIAGRAMS: A PLAIN CALL BECOMES A GUARDED TRANSACTION

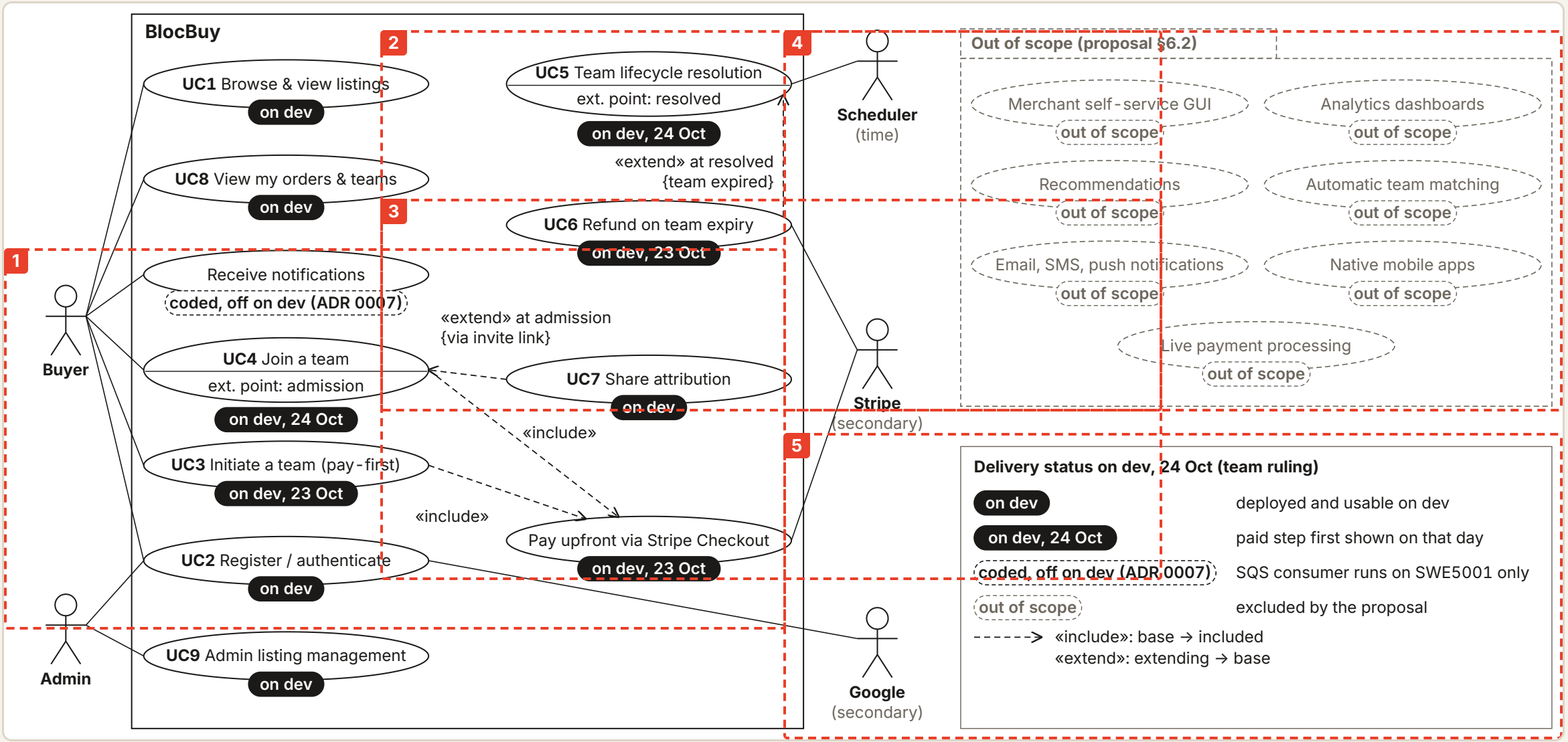
| | |
|---|---|
| **Analysis** | `control → entity.update()` |
| **Design** | `service → repository.lockDeal()` (SELECT … FOR UPDATE)<br>`→ aggregate.joinPaid() → guarded UPDATE`<br>all in one transaction |

**Why:** the domain names its needs as interfaces and infrastructure supplies them, so persistence and providers can change without touching the rules.

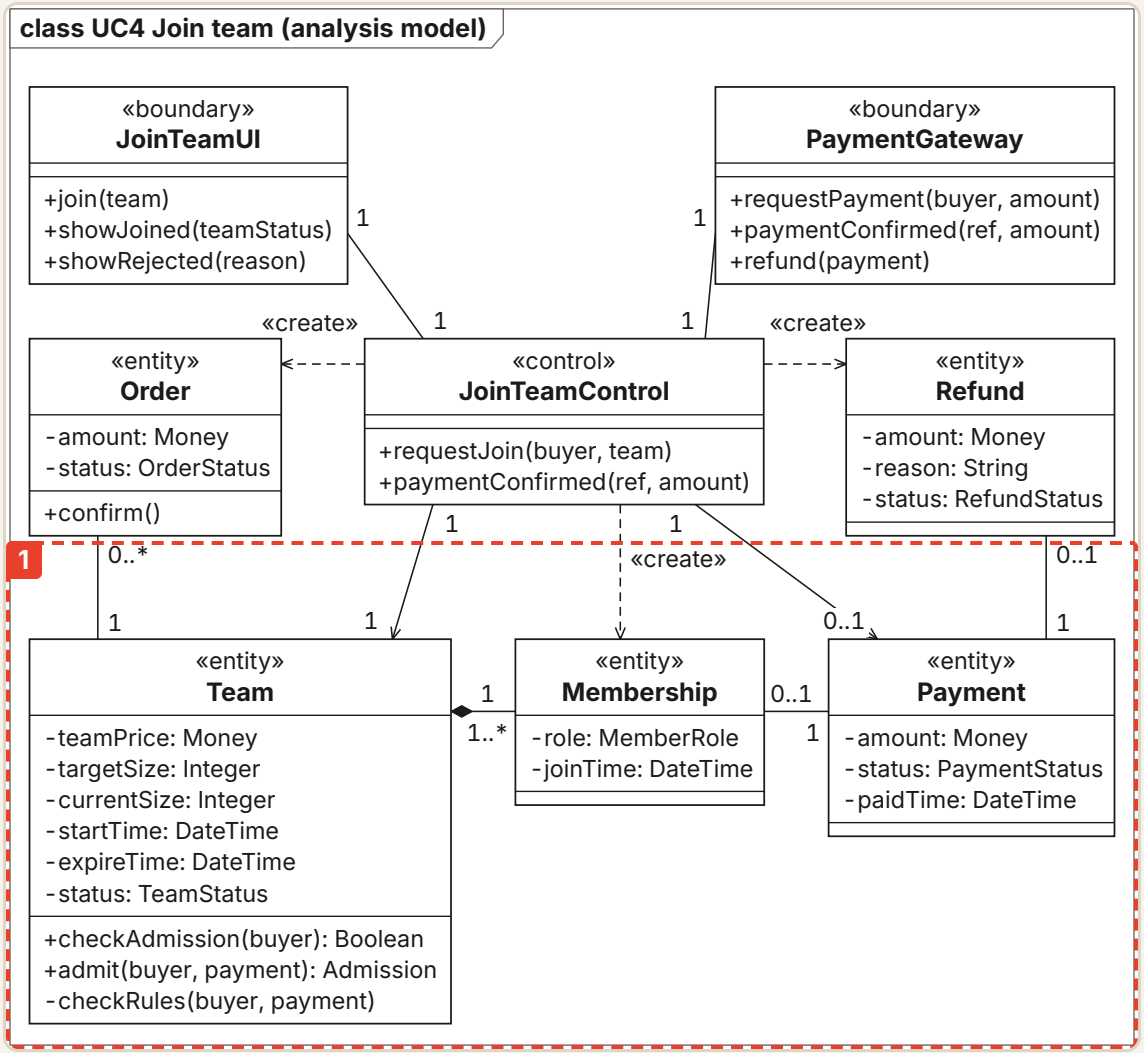
# One strategy pair: analysis objects become design classes

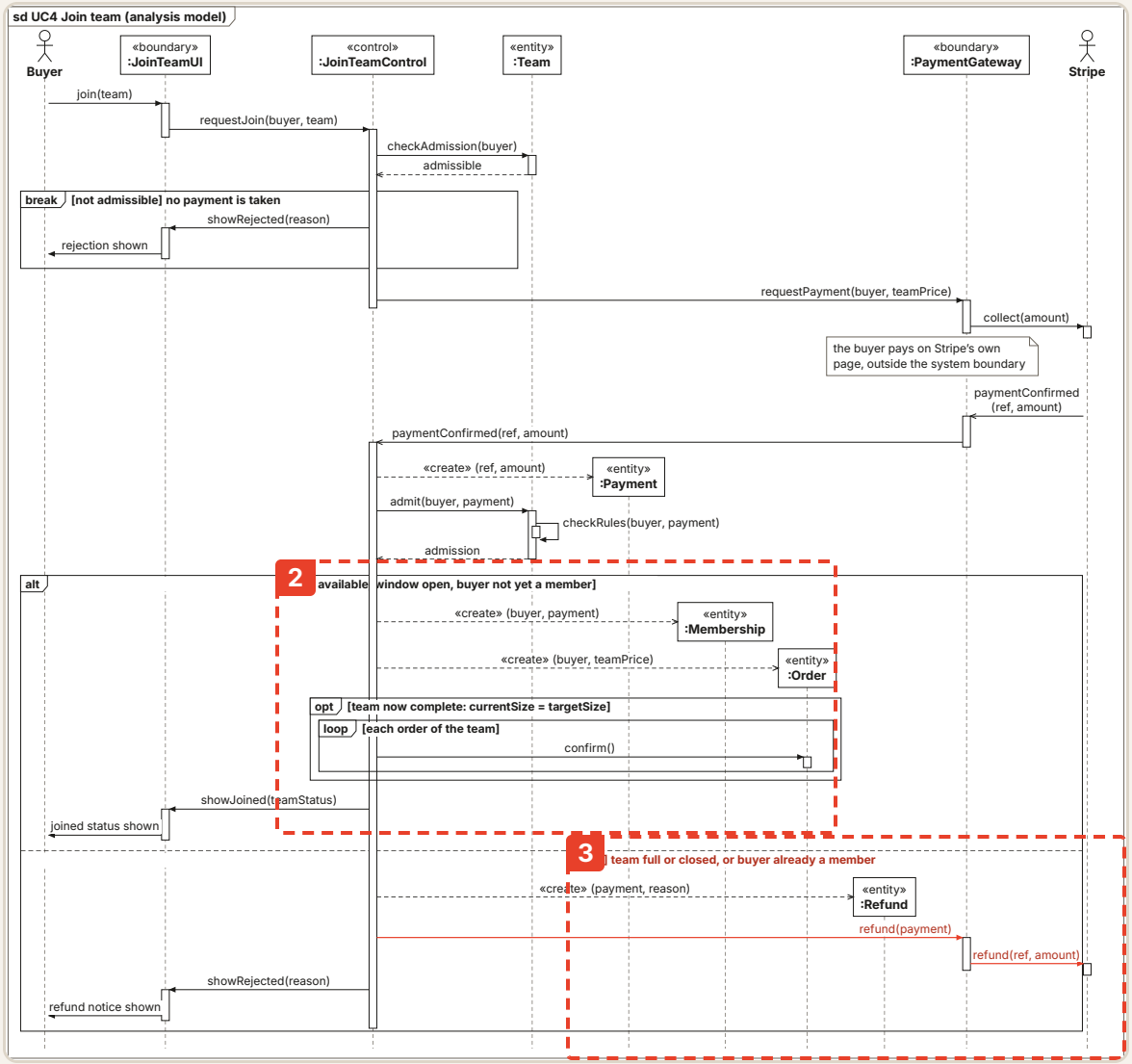


# Use case diagram: in scope, out of scope and status

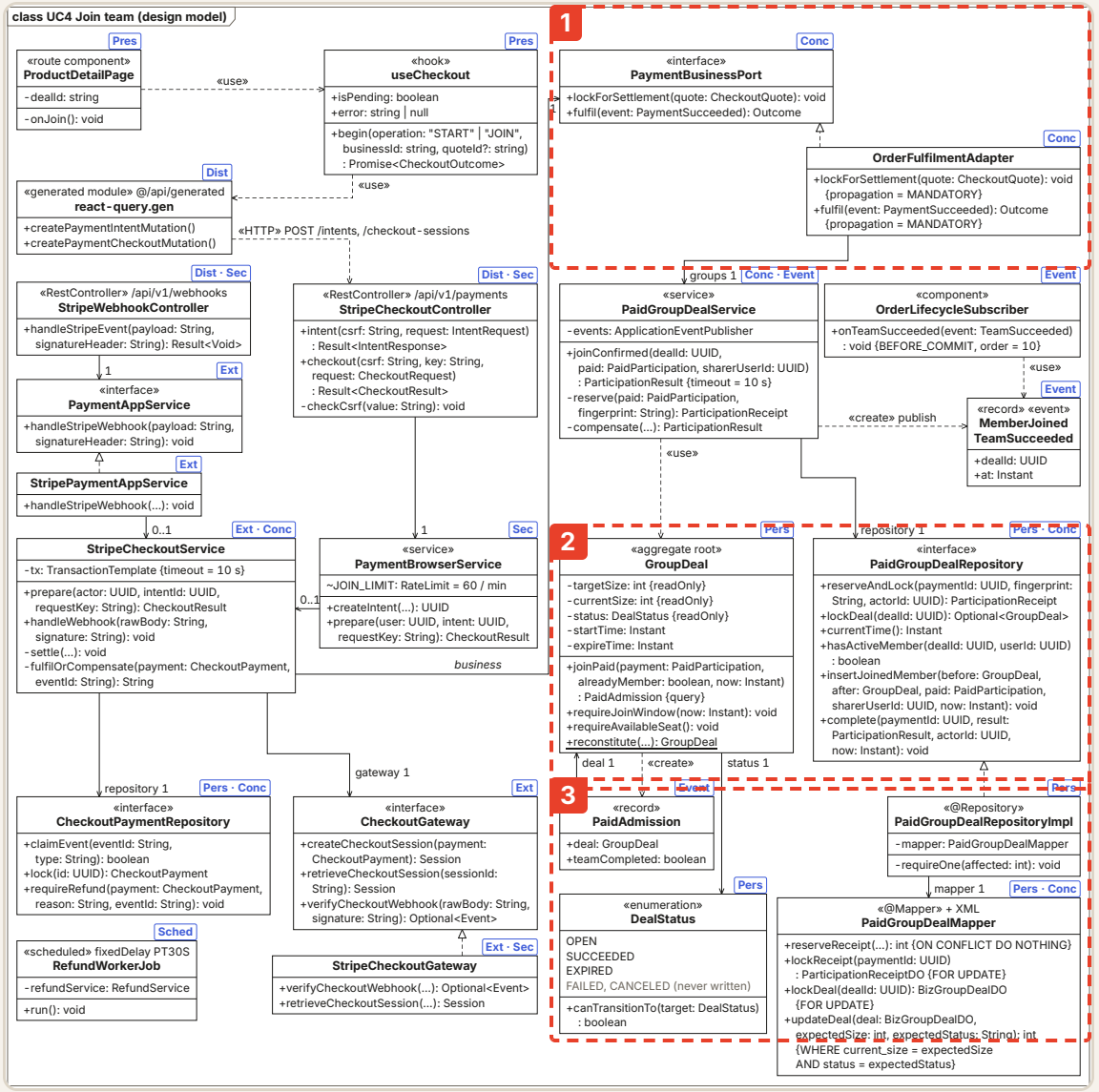


# UC4 join: the analysis model



# UC4 join: the design class diagram

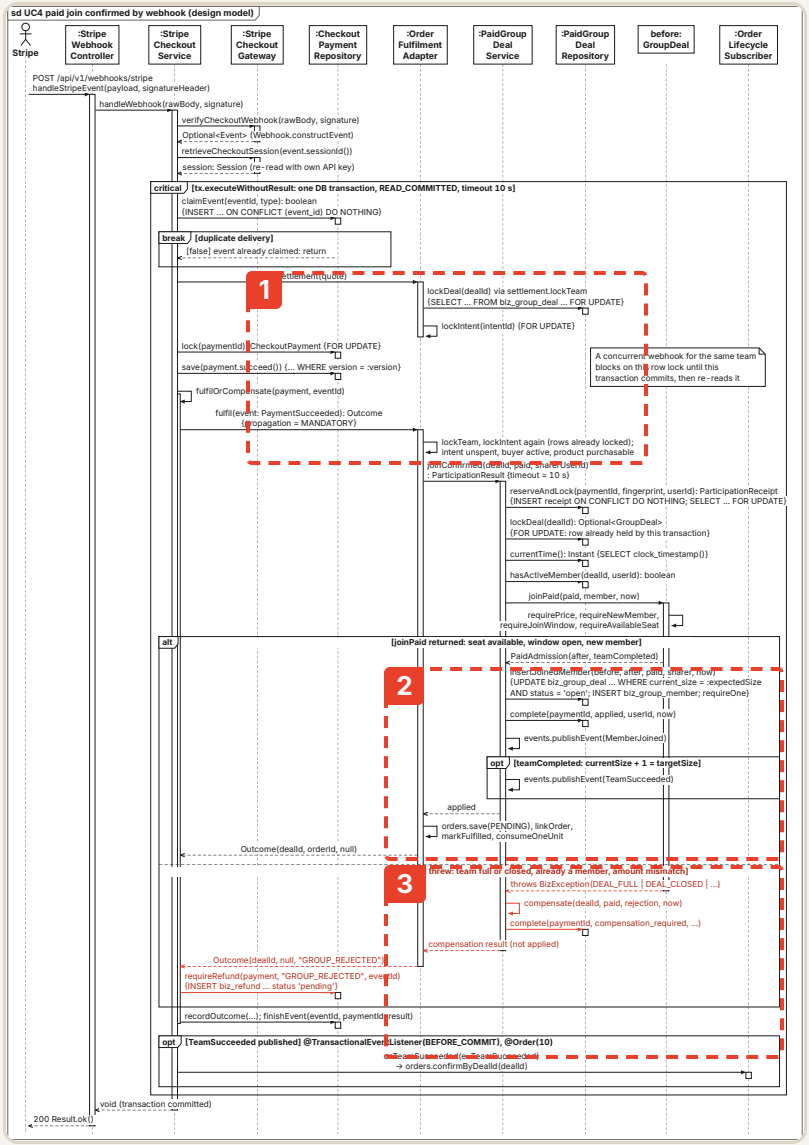



## STRATEGY TAGS

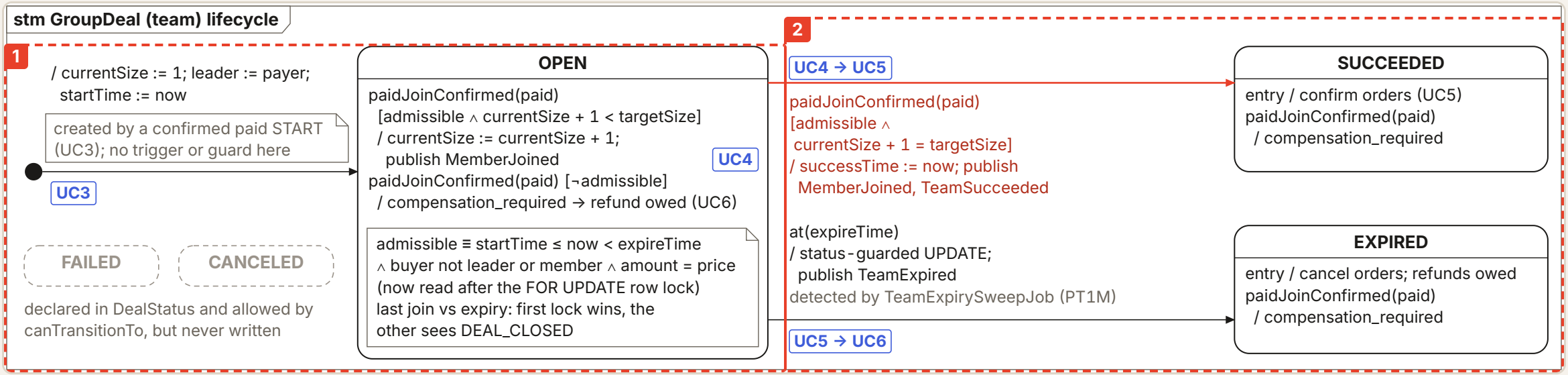
**Pres** presentation
**Dist** distribution
**Pers** persistence
**Ext** external interface
**Sec** security
**Conc** concurrency
**Sched** scheduling
**Evt** domain events
**Lang** language, framework

Typed signatures, visibility and multiplicities

# UC4 join: the design sequence, paid join by webhook



# Team-formation statechart




## WHY A STATECHART

Team state gates four use cases: UC4 join, UC5 resolution, UC6 refund and UC8 my teams.

## IN CODE

`DealStatus` and the guards of `GroupDeal`

# UC3 start a team: Factory Method

### WHERE IT SHOWS

The «create» of a team: the service once named the concrete creator itself

### CAUSE FOR REDESIGN

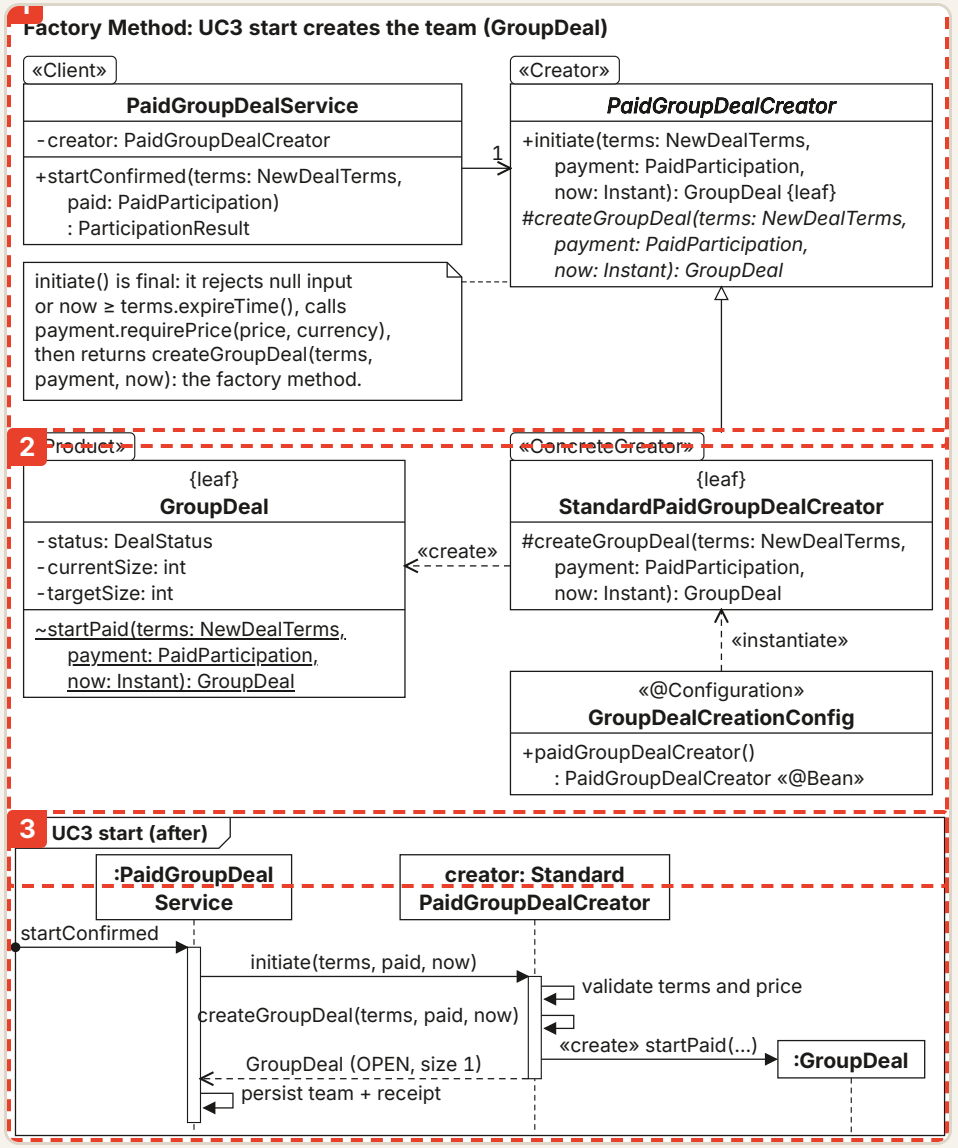
Creating an object by specifying its class explicitly

### WHAT VARIES

The class of the team that is created

### PATTERN

**Factory Method**



# UC5 to UC6 team outcome: Observer

## WHERE IT SHOWS

In the expiry sequence: it called the order and refund code inline

## CAUSE FOR REDESIGN

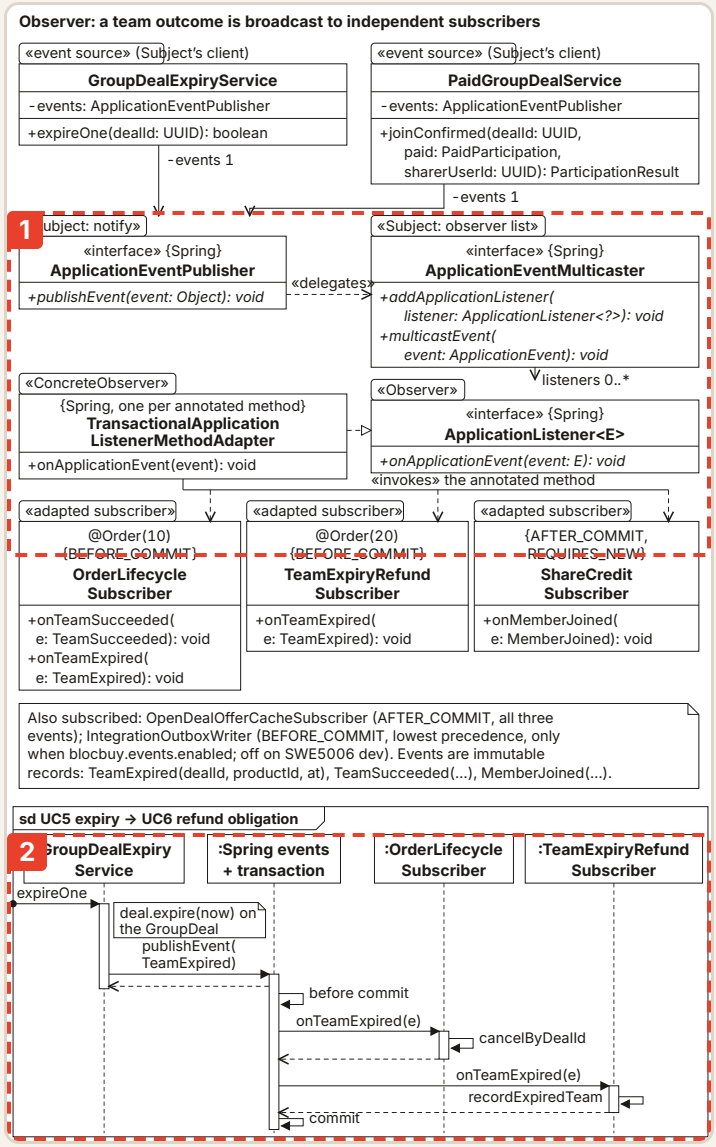
Tight coupling: more and more objects depend on a team's outcome

## WHAT VARIES

The objects that depend on a team's outcome

## PATTERN

**Observer**



# Notifications: Template Method for outbox transports

## WHERE IT SHOWS

In the class diagram: the SQS and SNS transports would each copy the batch loop

## CAUSE FOR REDESIGN

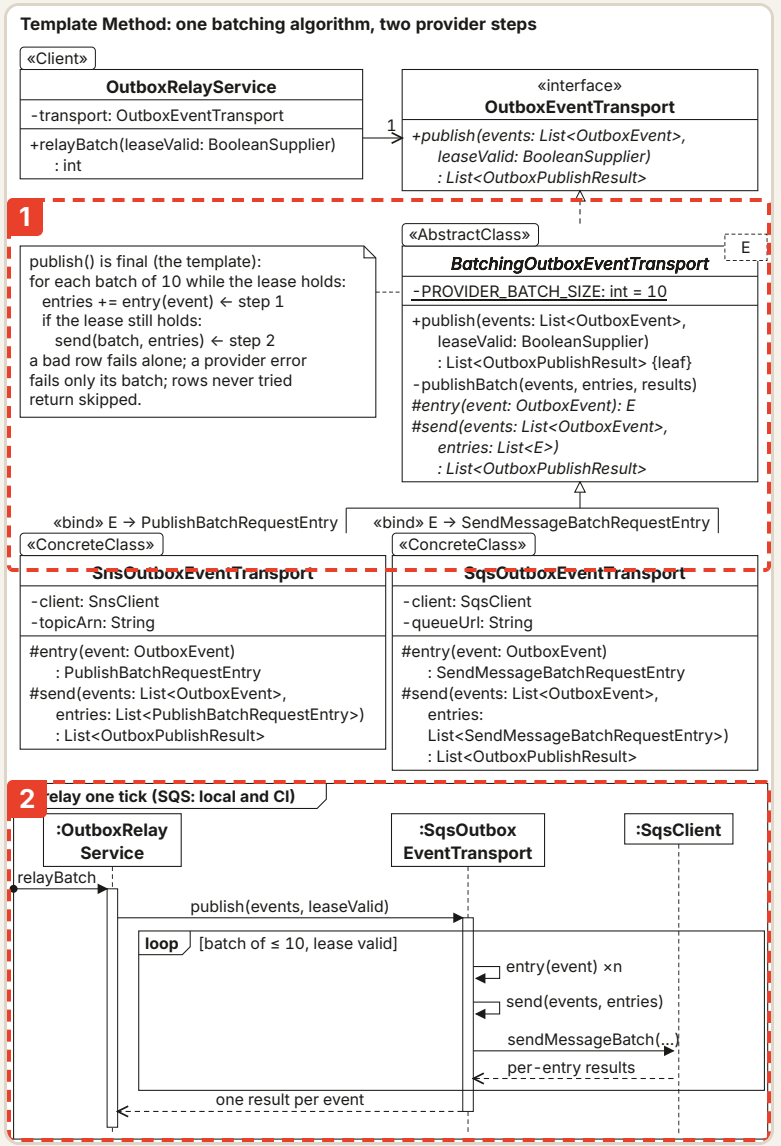
Algorithmic dependencies: the same steps, two providers

## WHAT VARIES

Steps of an algorithm

## PATTERN

**Template Method**



# UC1 pricing: Strategy

## WHERE IT SHOWS

In the class diagram: browse and the team quote each switch on the pricing mode

## CAUSE FOR REDESIGN

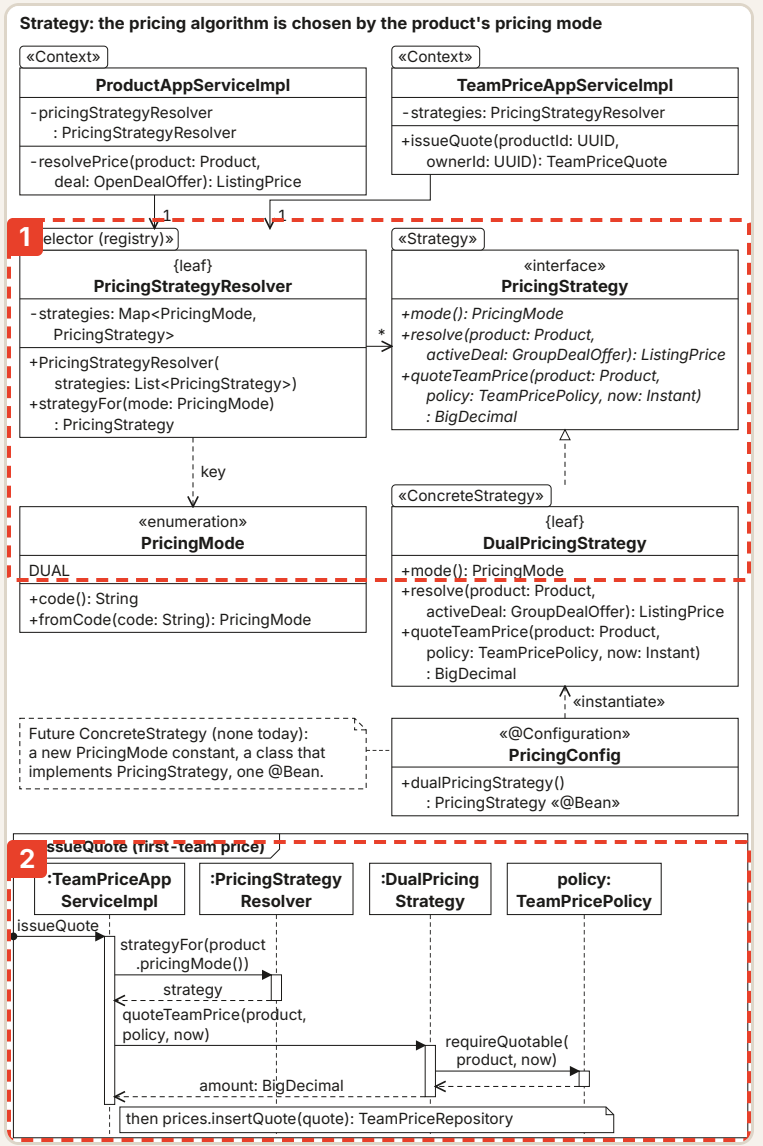
Algorithmic dependencies: the arithmetic sits inline in both

## WHAT VARIES

An algorithm: how a price is computed

## PATTERN

**Strategy**



# Design problem: listing rules that depend on its state

PROBLEM (UC9)

Publish, delist, revise and delete are legal in different states of a listing

WHY REDESIGN

Hard to maintain: one rule in four places

Hard to extend: a new state edits every switch

Hard to reuse: the rule cannot be tested apart from Product

WHAT VARIES

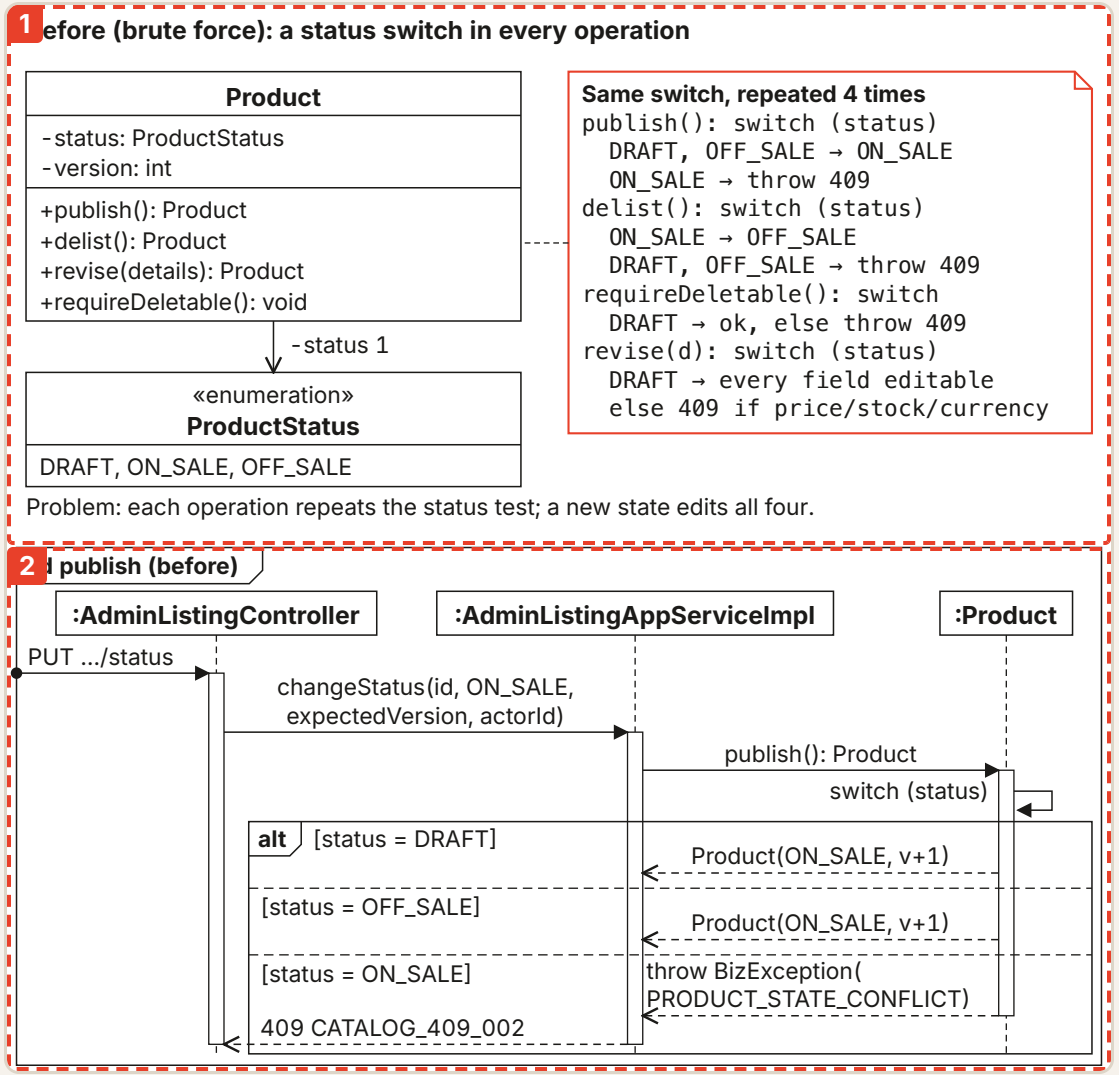
The states of an object

CANDIDATES

**State**

Strategy: nobody chooses; the state decides

Transition table, as on the team



# After: the State pattern, and why the team uses a table

## PATTERN

**State**: the listing delegates to its state

## IMPLEMENTATION DECISIONS

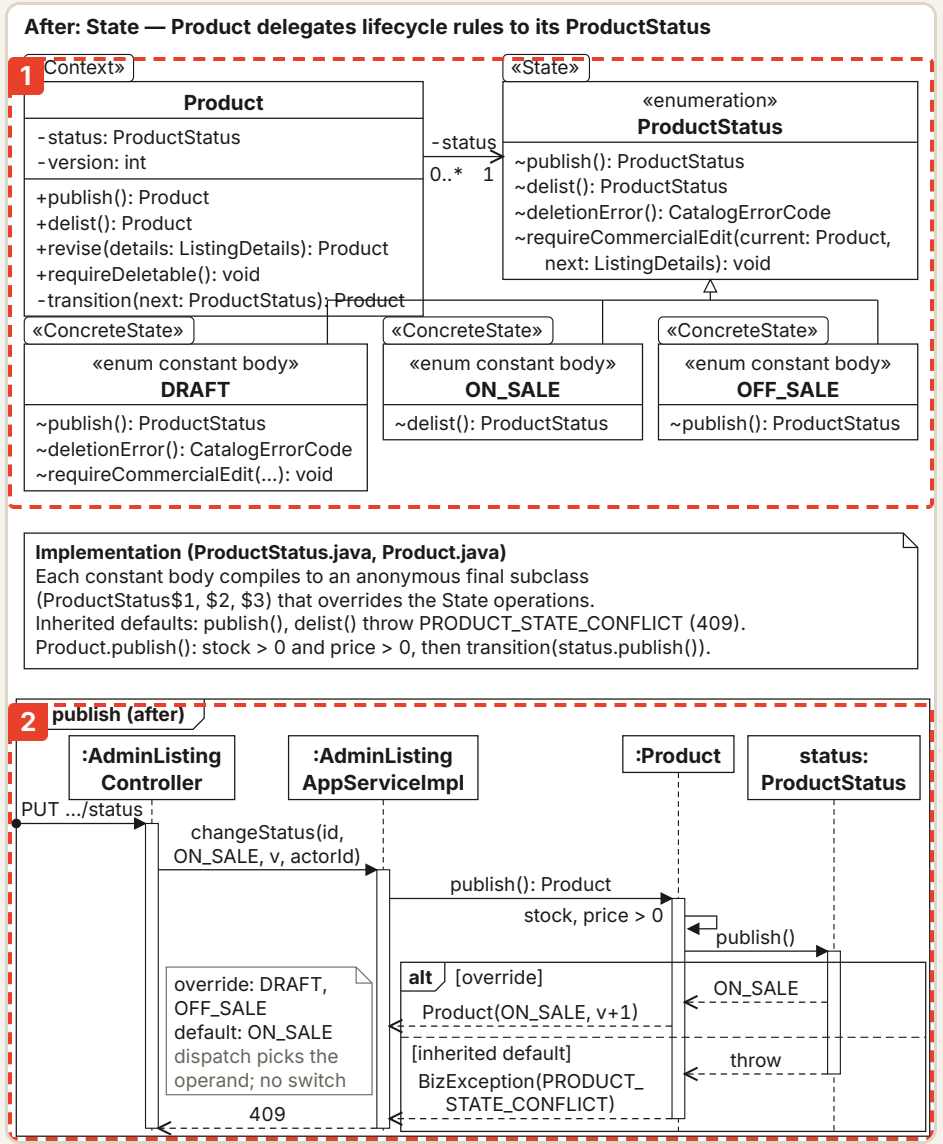
The state returns its successor; the listing applies it

Moves a state does not allow refuse with 409

Each move makes a new revision; a stale write gets 412

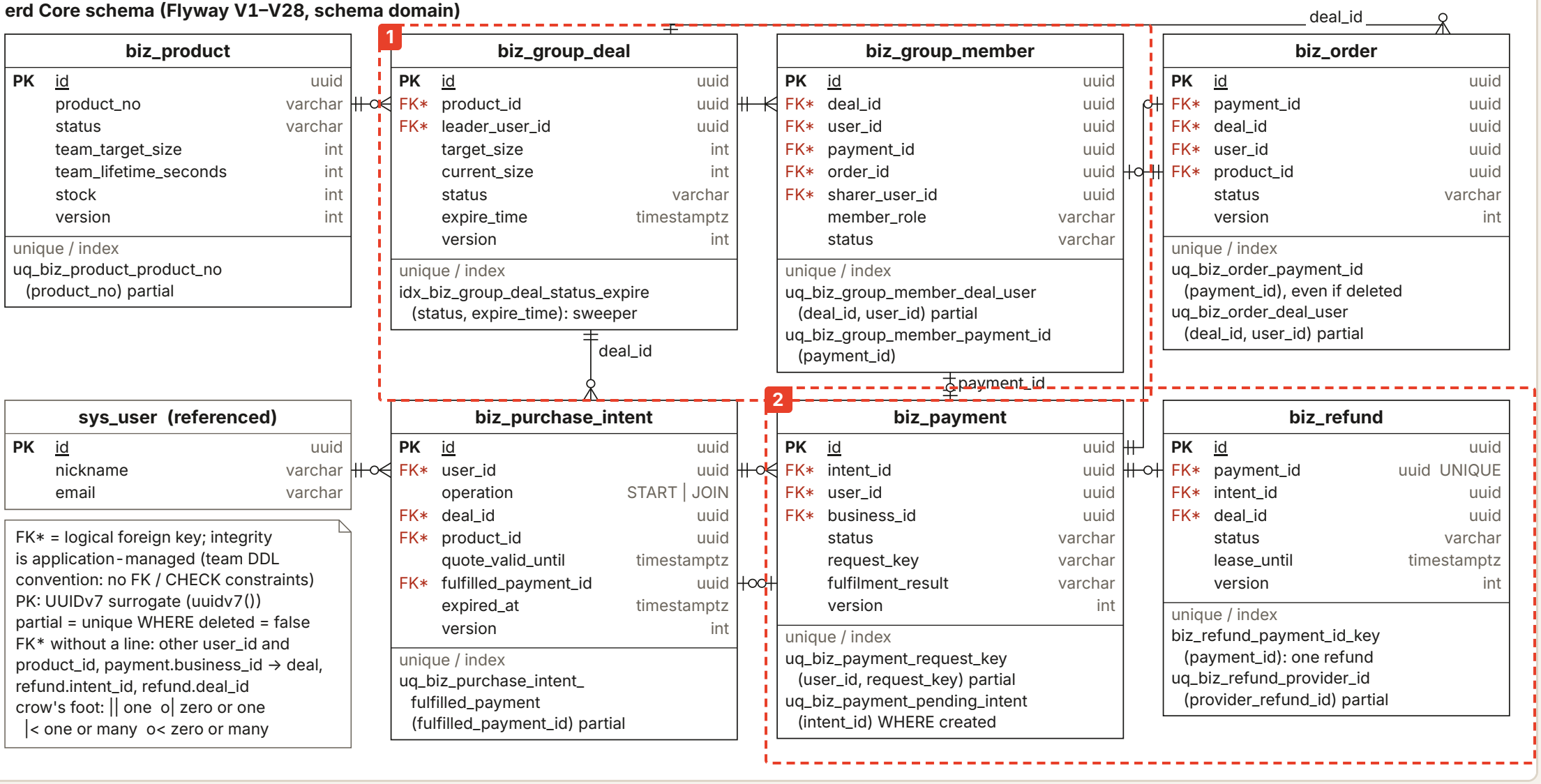
## WHY THE TEAM USES A TABLE INSTEAD

Only OPEN has behaviour; SUCCEEDED and EXPIRED are final, so a transition table suffices



# Database schema: core entities and mapping decisions

**erd Core schema (Flyway V1–V28, schema domain)**



# Testing on the Agile Testing Quadrants

**Q2** business-facing · supporting the team — automated

- **Story acceptance** system
- **End-to-end** system

**Q3** business-facing · critiquing the product — manual

- **Exploratory, acceptance** system

Not done: outside users

**Q1** technology-facing · supporting the team — automated

- **Unit, component** unit
- **Integration** integration
- **Contract, ArchUnit** integration

**Q4** technology-facing · critiquing the product — tools

- **Performance, load** system
- **Security scanning** types

# Scripted performance test: tool, scenario, target

## THE SCRIPTED TEST

| | |
|---|---|
| **Tool** | k6 1.6.1, image pinned by digest |
| **Target** | **SWE5001 ECS**, dev image of 6 Oct:<br>CloudFront → ALB → Fargate<br>Never the shared SWE5006 dev host, which also runs its database |
| **Load** | Open arrival rate 10 → 200 requests/s<br>Ten 60 s steps; 100 VUs preallocated, 400 max |
| **Mix** | 7 listing pages : 3 listing details, anonymous<br>No think time; 10 s timeout |
| **Pass, per step** | p95 < 500 ms<br>Errors < 1%<br>Zero dropped requests |
| **Result** | Run 20261016-v2-monolith-repeat, 16 Oct; shown on slide 36 |

## PROPOSAL TARGET (§6.3)

**Interactive operations ≤ 2 s with up to 100 concurrent users**

Little's law $N = X \cdot (R + Z)$: 100 users, 0.5 s think, ~30 ms response → **X ≈ 190 requests/s**

Inside the 200 driven, with a p95 gate four times stricter than 2 s

The 0.5 s think time is our assumption

## EXPIRY ≤ 1 MIN AFTER THE WINDOW

Measured **locally** under 100-user load (30 Sep): 30 teams, worst lag 58.2 s

On ECS: not run, so this evidence is local only

# Source code: one monorepo, Branch by Feature, reviewed changes

## ONE MONOREPO

| | |
|---|---|
| `backend/` | Maven COLA modules; SWE5001 services/ |
| `frontend/` | React SPA |
| `contracts/` | openapi.json, the API contract |
| `deployment/` | Compose, SSM bundle, deploy.sh |
| `infra/` | Terraform: dev frontend, SWE5001 |
| `perf/` | k6 load tests |
| `docs/` | ADRs, sprint logs |
| `.github/` | workflows, CODEOWNERS, Dependabot |

## BRANCHING: BRANCH BY FEATURE

One short branch per story

A pull request, then a squash merge onto the trunk, **dev**

Linear history since 12 Sep

Up to date with dev before merging, instead of merging the trunk daily

## WHO MAY CHANGE IT

GitHub: a **code owner** approves; six checks green

Owners: backend Liu Jin, frontend Lim, CI/CD Pan

Payments, refunds Yorik; group buying Eric

AWS: IAM Identity Center, MFA enforced, one-hour sessions

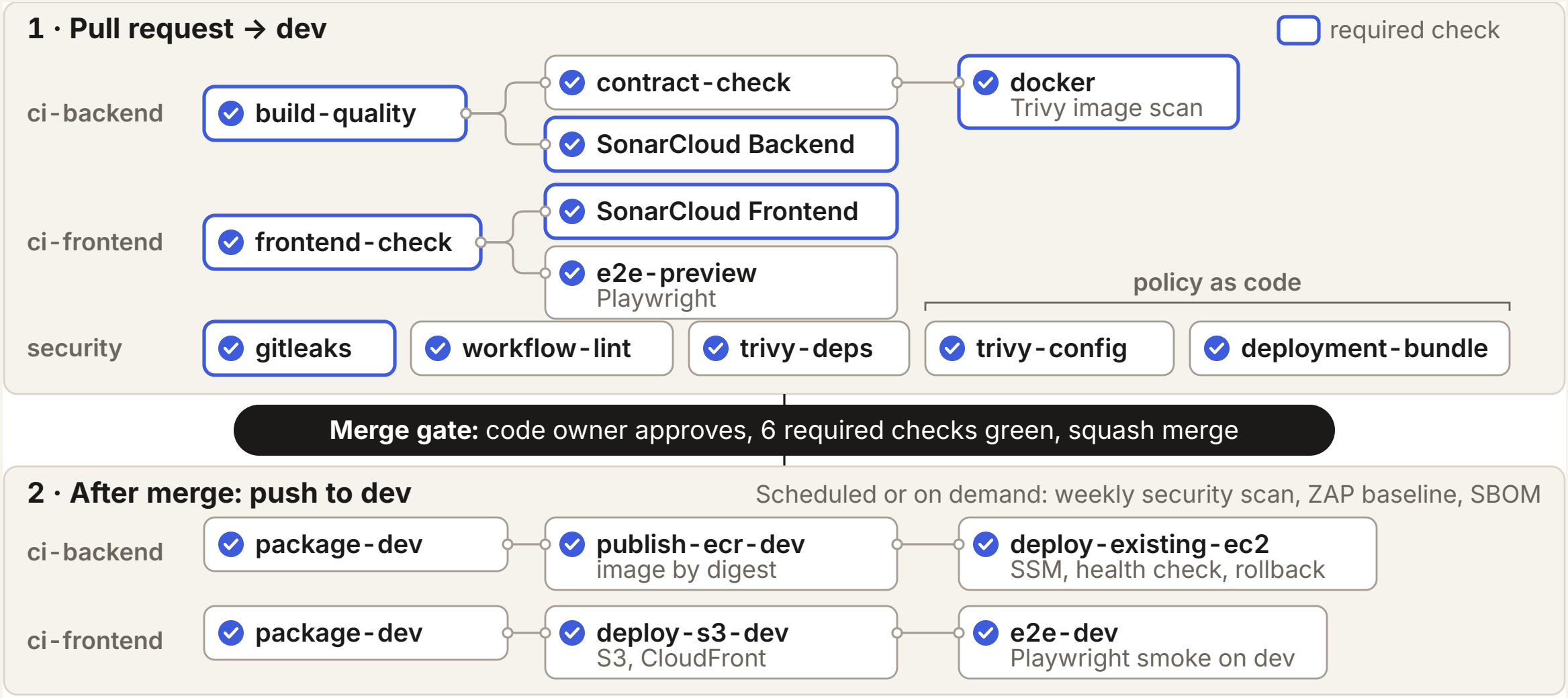
CI: one OIDC role per job, no AWS keys in GitHub

Exception: two IAM users for SSM port forwarding, one with an access key

# Artefacts: each one traceable to a commit

| Artefact | Identity | Stored in |
|---|---|---|
| **Backend image** | `git-<sha>-run-<n>`, deployed by its `@sha256` digest | ECR, immutable tags, newest 30 kept |
| **API contract** | `openapi.json` committed; drift fails CI | Repository |
| **TypeScript client** | Generated from the contract, gitignored | Build only |
| **Frontend bundle** | Content-hashed files, immutable cache | S3 behind CloudFront |
| **Database schema** | Flyway migrations V1–V28 | Repository |
| **Infrastructure** | Terraform plan saved, applied by an operator; CI never applies | S3 state, DynamoDB lock |
| **SBOM + licences** | CycloneDX source-dependency SBOMs and licence inventories | Actions artefacts, kept 90 days |

# Continuous integration: pull-request jobs and post-merge jobs



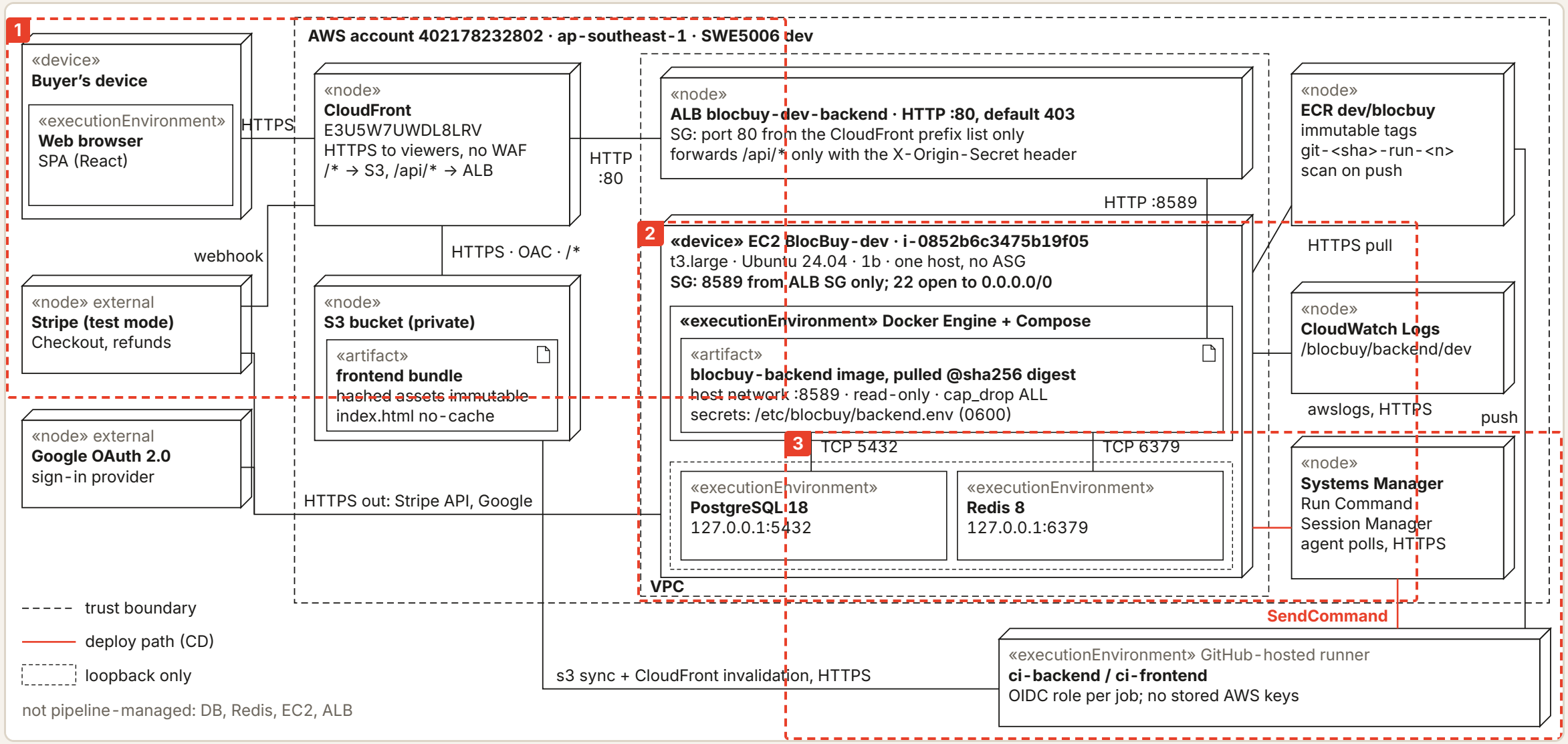

Real job names; every step, trigger and path filter: backups B8 and F16

# Security controls by pipeline stage, and what each one catches

| Stage | Control | Catches | Gap |
|---|---|---|---|
| **Pre-commit** | Lefthook: format, lint, type-check (frontend) | Style and type errors | — |
| **Commit · PR** | **gitleaks** required: the PR's commits, full history weekly<br>Code-owner review; CodeRabbit review, advisory<br>actionlint | Secrets, unsafe workflows, review findings | — |
| **Build** | SAST: **SonarCloud** required, SpotBugs<br>SCA: Trivy deps, Dependabot daily | Code flaws, vulnerable libraries | — |
| **Test** | **Trivy image** required (HIGH, CRITICAL)<br>Trivy config, deployment-bundle rules | Image CVEs, IaC misconfiguration | — |
| **Deploy** | OIDC role per job<br>Image by digest<br>Read-only container, all capabilities dropped<br>OWASP ZAP: passive, anonymous dev pages, advisory | Stolen keys, swapped images, missing security headers | Dev delivery rebuilds the image without repeating the Trivy image gate; ZAP: no logins, payments or attacks |

**Types tested (slide 25, Q4):** static analysis, dependencies, image and IaC configuration, secrets, passive DAST
Every third-party action is pinned to a commit SHA

# Delivery: one environment, the code owner approves, the deploy verifies itself



# Technical findings: three issues, their root causes and fixes

| Finding | Root cause | Fix |
|---|---|---|
| **Early dev deploys failed (5–17 Sep)** | The hand-built environment drifted from its design<br>Old OIDC subject format, a placeholder instance ID, port 8589 open wide | Fixed on dev by hand<br>The prod stack renders each value in Terraform, so a plan shows it<br>No dev deploy job failed from 18 Sep to 22 Oct 13:54 UTC |
| **build-quality red for two days (19–20 Oct)** | Dependabot upgraded the SpotBugs engine without its Maven plugin | Both now upgrade together<br>A POM policy test guards it |
| **Weekly full-history gitleaks failing (since 21 Sep)** | 20 matches were file digests in a contract baseline, not secrets | Each digest re-hashed, then fingerprinted exactly<br>4 badge-token findings still open |

**Still not shown:** crash recovery under replay on dev. Paid start, a second buyer's join and the expiry refund were shown on dev on 23–24 Oct, after a webhook configuration repair.

# Live: one change from pull request to dev

| Step | What you see | Time |
|---|---|---|
| **P1** | The pull request: one frontend change, six required checks green | 60 s |
| **P2** | Lim approves as the frontend code owner | 30 s |
| **P3** | Squash merge: one commit on dev | 20 s |
| **P4** | Delivery runs; the backend PR and deploy, triggered beforehand | 90 s |
| **P5** | Verified: e2e-dev green, the change live on dev | 90 s |
| **P6** | Merge to live in minutes; hand-over | 40 s |

### WHY THE BACKEND IS PRE-RUN

Backend PR checks take about 16 min, longer than the segment, so that run was triggered before the talk and we walk through it finished.

### BACKUP VIDEO

Recorded on dev on 24 Oct: finished runs, checks to smoke test

Shared in the team chat; ready on the presenter’s laptop

YQ Yorik

# System demo: two buyers, one bloc, and the qualities it shows

| Step | What you see | Use cases | Quality attribute (slide 3) | Runs on |
|---|---|---|---|---|
| **D1** | Leader signs in, starts a bloc, pays in Stripe Sandbox, copies the invite | UC2 UC1 UC3 UC7 | Security: only a signed webhook confirms payment | dev, if GO |
| **D2** | Joiner opens the invite, joins and pays: 2 of 2, the team succeeds | UC4 UC5 UC7 | Correctness: guards re-checked after payment | dev, if GO |
| **D3** | My blocs and orders, filtered by the URL | UC8 | Functionality | dev |
| **D4** | The bloc started at 16:05 expired and refunded itself | UC5 UC6 | Unattended expiry and refund; ≤ 1 min measured locally (slide 36) | dev, if GO |
| **D5** | No session, a buyer on admin, a write without the marker | — | Security: 401, 403, 400, HttpOnly | dev |
| **D6** | Twenty paid buyers race for one seat | UC4 | Correctness under concurrency | local test |
| **36** | Scripted k6 load, 16 Oct run | UC1 | Performance: ≤ 2 s at 100 users | SWE5001 ECS |
| **38** | Back to the §4 pipeline run | — | Maintainability | dev |

“dev, if GO”: live on dev after the 24 Oct go/no-go; else its 24 Oct recording if one exists, else local evidence labelled local, never dev

YQ Yorik

# Live: two buyers, one bloc

| Step | What you see | Time |
|---|---|---|
| **D1** | Start a bloc and share the invite | 120 s |
| **D2** | A friend joins and pays: the team succeeds | 120 s |
| **D3** | My blocs and my orders | 75 s |
| **D4** | The 16:05 bloc: expired, order cancelled, refunded | 90 s |
| **D5** | Security: 401, 403, 400, HttpOnly cookie | 60 s |
| **D6** | Last-seat race: the test, in the IDE | 45 s |

### TWO BUYERS, ONE SCREEN

Two Chrome profiles, Leader and Joiner, both signed in with Google

Stripe Sandbox: a test card, no money moves

### STARTED BEFORE THE TALK

A team of two on `US34-04` at 16:05: its 40-minute window, the shortest allowed, ends ≈ 16:45.

### BACKUP VIDEO

Recorded on dev on 24 Oct: start, join, refund

Shared in the team chat; ready on the presenter’s laptop

YQ Yorik

# Performance against the target: the scripted k6 run

K6 RUN · SWE5001 ECS ENVIRONMENT · NOT THE DEV HOST

| Requests/s | p95 (ms) | Errors | Dropped | Step |
|---|---|---|---|---|
| **10** | 53 | 0 % | 0 | pass |
| **20** | 39 | 0 % | 0 | pass |
| **40** | 27 | 0 % | 0 | pass |
| **60** | 28 | 0 % | 0 | pass |
| **80** | 23 | 0 % | 0 | pass |
| **100** | 23 | 0 % | 0 | pass |
| **125** | 42 | 0 % | 0 | pass |
| **150** | 23 | 0 % | 0 | pass |
| **175** | 23 | 0 % | 0 | pass |
| **200** ● | 23 | 0 % | 0 | pass |

● at or above ≈ 190 requests/s: the proposal's 100 users, 0.5 s think time assumed (slide 26)

Run 20261016-v2-monolith-repeat, 16 Oct: 57,600 requests, 0 failed, 0 dropped

TARGET (PROPOSAL §6.3)

**≤ 2 s with up to 100 concurrent users**

Our gate per step:
p95 < 500 ms
Errors < 1 %
No dropped requests

**Verdict: pass on every step**
Worst p95 53 ms, 0 % errors, 0 dropped

EXPIRY ≤ 1 MIN

Worst lag 58.2 s for 30 teams under 100-user load (**local**, 30 Sep)

NOT CLAIMED

A capacity limit: 200/s is the last step
Authenticated paths under load
Anything about the dev host
Scaling (one task)

YQ Yorik

# Scalability: a stated trade-off, not a claim

DEV, THE SYSTEM YOU SAW

| | |
|---|---|
| **Compute** | One EC2 t3.large, no Auto Scaling group |
| **Data** | PostgreSQL 18 and Redis 8 on the same host |
| **Effect** | One point of failure; a deploy restarts the API |

WHY, DELIBERATELY

- One complete system, always on, at low cost
- The proposal lists no scalability target
- One codebase, two topologies

**Scale path, not claimed here:** the same code on ECS in our SWE5001 topology, commerce 2–6 tasks on requests per target, notification 1–4 on queue backlog.

YQ Yorik

# Maintainability: the pipeline you just saw, and the seams in the code

SHOWN IN §4 (SLIDE 33)

One change, live on dev

Six required checks, a code-owner review, an automatic deploy and a smoke test

17 ArchUnit rules hold the module boundaries on every build

PRIMARY DESIGN DRIVER: EXTENSIBILITY

| Change | Added as | Pattern |
|---|---|---|
| **New pricing mode** | One more `PricingStrategy` and its `PricingMode` constant | Strategy slide 21 |
| **New consequence of a team's outcome** | One more subscriber to `TeamSucceeded` / `TeamExpired` | Observer slide 19 |
| **New payment provider** | One more adapter behind `CheckoutGateway`, `RefundGateway` | Gateway port (external-interface strategy) slide 12 |

Existing rules stay unedited: each change adds classes; a pricing mode also adds one enum constant.

# Thank you

Questions?

github.com/PPsyrius/iss-groupbuy-team8  d1b3np88z657cp.cloudfront.net

# Backup slides: index


B1 **Agile artefacts appendix**
Where are the backlog, stories, sprint plans, reviews and retros?

B2 **Sprint 2 and 3 burndowns**
How did earlier sprints burn down?

B3 **ADR index**
Which decisions are recorded, and where?

B4 **Database schema, all tables**
Which tables does each context own?

B5 **Expiry and refund sequence**
How does an expired team become refunds?

B6 **UC6 refund recovery: candidates and decisions**
How do you never refund twice?

B7 **Test inventory**
How many tests of each kind, and where do they run?

B8 **Workflows, required checks and timings**
What runs when, how long, and how often is it green?

B9 **Known gaps and limitations**
What is not done yet in the pipeline and on dev?

B10 **SWE5001 production topology**
What would scaling out look like?

B11 **Stripe Sandbox refund, local evidence (20 Sep)**
Is the refund real, and where has it been proven?

B12 **Scale-out and load results (SWE5001 ECS + local baseline)**
How does it scale, and how fast is it?

B13 **UC9 listing statechart**
Which listing moves are legal in each state?

B14 **UC4 design diagrams, complete**
Can we see every class and message of the join?

B15 **Every member's use case and design problem**
Where are the other members' models?

B16 **Entity-relationship diagram, all tables**
What does the whole schema look like?

DX **Diagram details, 1:1 (D slide-step, e.g. D15-2)**
Every crop the talk steps through, with its caption, readable in the PDF

FX **Diagrams at full size**
Every diagram in the deck at 1:1, scrollable


*Click a row (or type #B6 or #D15-2 in the URL). Slide 39 stays up during questions.*

JC Lim

# Agile artefacts appendix

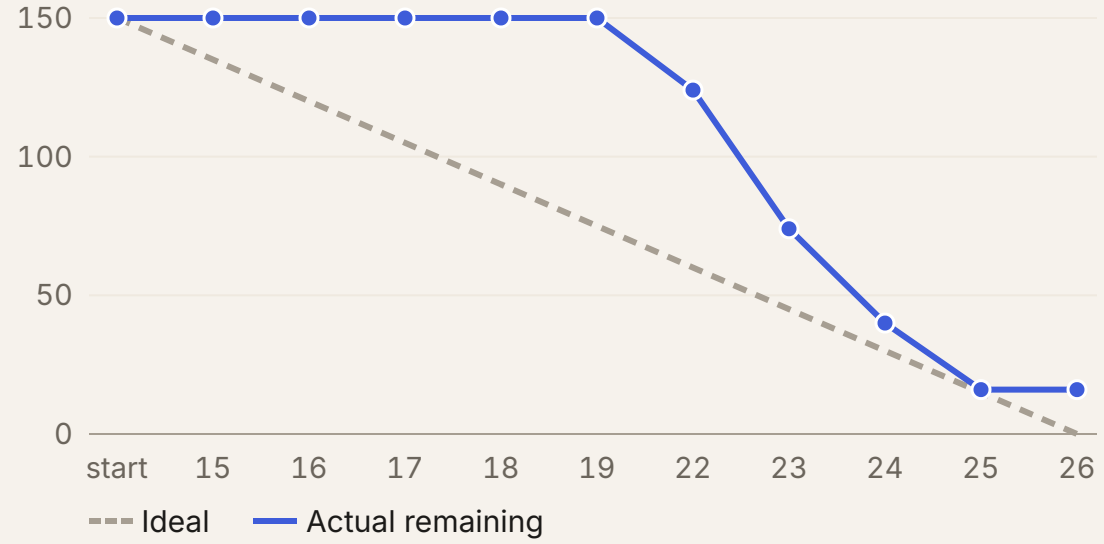
| Artefact | Where it lives | State on 21 Oct |
|---|---|---|
| **Product backlog** | GitHub Issues and Project board 2 (GroupBuy Kanban Board); issue templates in .github/ISSUE_TEMPLATE | 170 items: 137 Done, 15 Ready, 15 Backlog, 3 In progress |
| **User stories with acceptance criteria** | UC issues #24–#32; story issues (e.g. #221, #223, #180); AC per story in docs/agile/sprint-03/plan.md §5 | template: complexity, accountable members, design problem, AC |
| **Sprint plans** | S1 #49; S2 Sprint 2 plan (docx) and #93; S3 docs/agile/sprint-03/plan.md and #232; S4 docs/agile/sprint-04/plan.md and #465 | all four sprints |
| **Burndown charts** | S2 chart in docs/sprint2-progress-report.typ; daily data docs/agile/metrics/*.csv (board-snapshot.yml, since 17 Sep) | S2, S3 reconstructed (B2); S4 from snapshots |
| **Sprint reviews** | S1, S2 in the progress reports; S3 docs/agile/sprint-03/review.md | S4 after the talk, with its remarks (#619) |
| **Retrospectives** | S1 report; S2 report §Sprint Retrospective; S3 docs/agile/sprint-03/retro.md | S3 reconstructed 24 Oct · S4 after the talk (#619) |
| **Day logs, DoD, practices** | docs/agile/sprint-0N/log/ (S2–S4); DoD per sprint: docs/agile/definition-of-done.md; EPAT practices: docs/agile/practices.md | dated logs per working day |

*Source: board and issues read with gh on 21 Oct; docs/agile/README.md. Report Appendix A holds the full set (#463 plan, #681).*

# Sprint 2 and 3 burndowns

## Sprint 2 · remaining hours, 15–26 Sep

150 h planned · 16 h left · reconstructed from merge dates

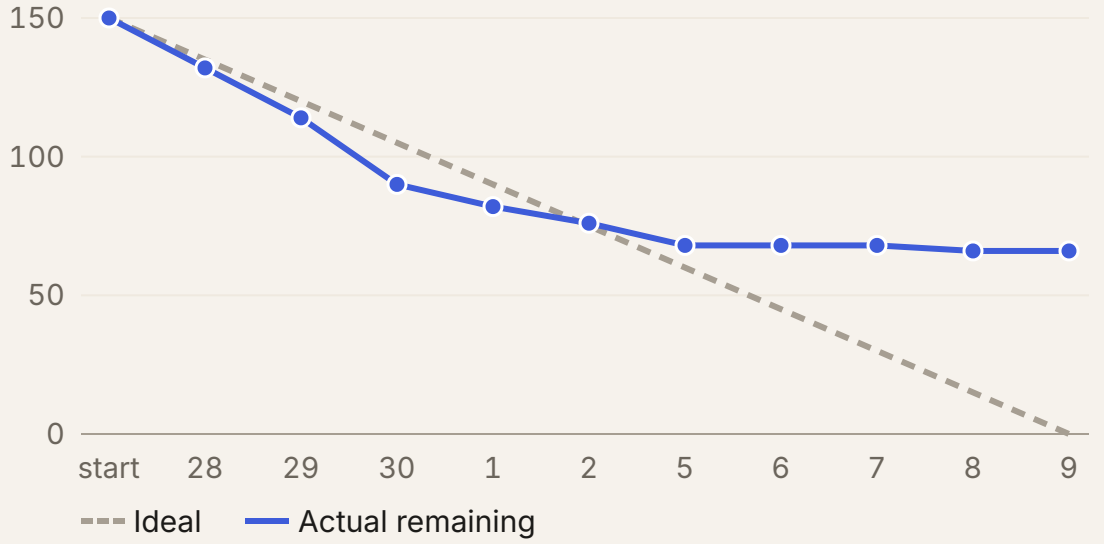



Flat for a week, then a cliff: no committed story merged before 22 Sep; week-1 merges were carry-over and enabling work. Retro answer: PRs under 1,500 lines.

*Source: docs/sprint2-progress-report.typ, "Daily burndown data"*

## Sprint 3 · remaining hours, 28 Sep–9 Oct

150 h planned · 66 h left, whole stories · 33 h with partial credit (review)




On the ideal line for the first week, then stuck from 5 Oct: #223 (Checkout on dev) stayed open, #224 had local evidence only, #229 waited on #223 and #222, #226 on the NFR-37 decision. Daily card counts agree: 19, 16, 13, 12, then 7.

*Source: sprint-03/review.md hours, burned on issue close dates (SGT); docs/agile/metrics/*.csv*

WZ Eric

# ADR index

## Backend · backend/docs/adr

| # | Decision |
|---|---|
| 0001 | Purchase intent lives in order; group terms on the product |
| 0002 | Team resolution is published; consequences run before commit |
| 0003 | Referral credit is a post-commit subscriber |
| 0004 | Retire the unused legacy PaymentIntent path |
| 0005 | Gate Stripe Checkout with a deployment property |
| 0006 | Payment owns the admin attention read model |
| 0007 | Outbox, one SNS topic, one SQS queue per consumer (Proposed) |
| 0008 | One codebase, two topologies: dev monolith, three services on ECS |

## Frontend · frontend/docs/adr

| # | Decision |
|---|---|
| 0001 | React SPA, pure CSR; Router owns URL state, Query server state |
| 0002 | Same-origin /api with an HttpOnly cookie session |
| 0003 | Generated API client, not committed |
| 0004 | One build artifact promoted dev to prod |
| 0005 | SPA fallback by a CloudFront Function |
| 0006 | oxlint and oxfmt toolchain |
| 0007 | Hand-managed dev CloudFront, then Terraform import |
| 0008 | Documentation and evidence conventions |
| 0009 | Result envelope: one unwrapResult as select |
| 0010 | React Compiler via the Babel preset |
| 0011 | Testing strategy: Vitest, MSW, Playwright smoke |
| 0012 | Agent configuration in AGENTS.md |
| 0013 | pnpm and Node pinned, exact versions |
| 0014 | Stripe Hosted Checkout with a server-owned intent |
| 0015 | Admin console shell, roles guard, versioned writes |

LJ Liu Jin

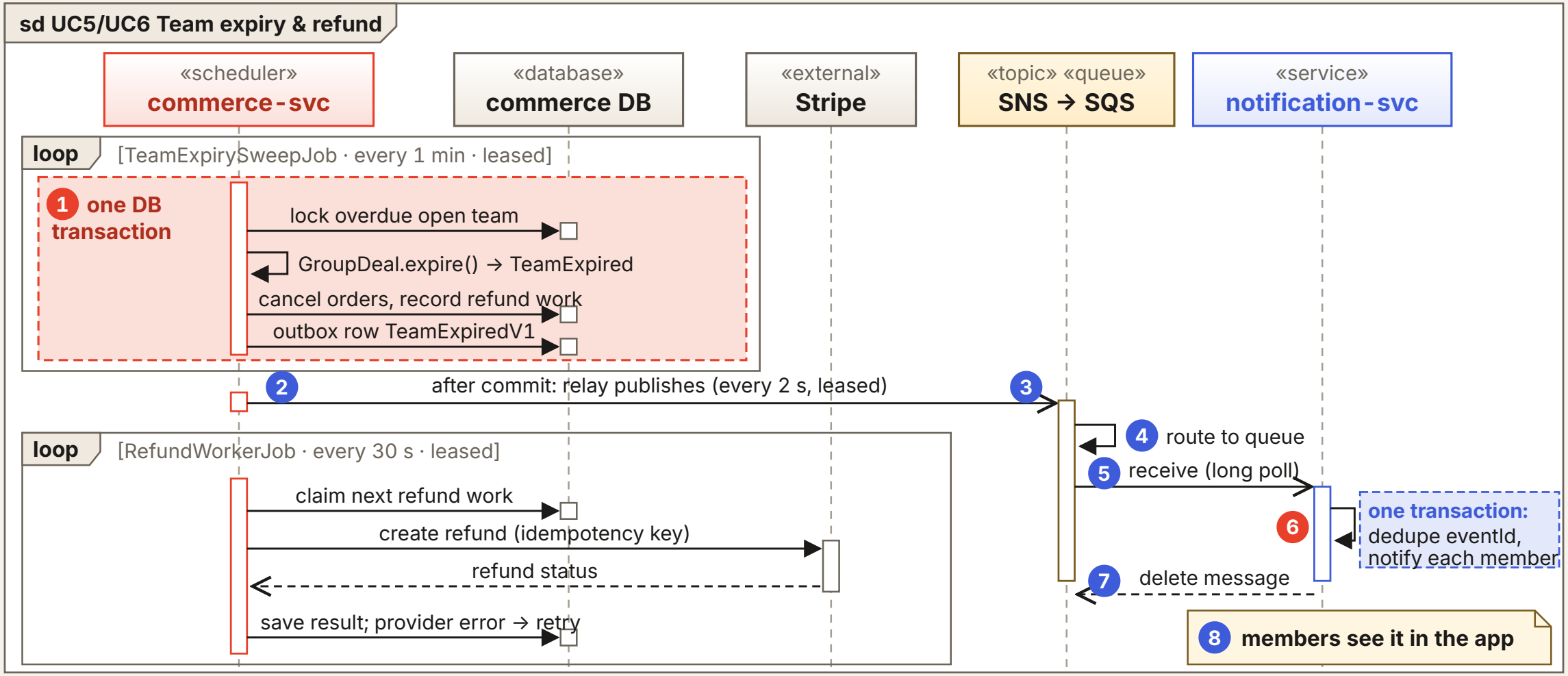
# Database schema, all tables

| Context | Tables | Migrations | Keys that make “once” hold |
|---|---|---|---|
| **identity** | `sys_user, sys_user_oauth, sys_role, sys_permission, rel_user_role, rel_role_permission` | V3, V24 | one binding per Google subject |
| **catalog** | `biz_product, biz_team_price_policy, biz_team_price_quote, log_product_team_terms` | V4, V6, V14, V20, V21 | biz_product.version (Optimistic Offline Lock) |
| **groupbuy** | `biz_group_deal, biz_group_member, biz_group_participation_receipt, biz_share_invite, biz_share_credit` | V5, V7, V18 | one member per user and team; one receipt per payment |
| **order** | `biz_order, biz_purchase_intent` | V12, V15, V19, V22 | intent fulfilled by one payment |
| **payment** | `biz_payment, log_payment_webhook_event, biz_refund, log_payment_review, biz_payment_review_case, log_refund_attempt` | V2, V9–V11, V16 | webhook event_id; one refund per payment |
| **notification** | `biz_notification, sys_consumed_message` | V27, V28 | PK (event_id, consumer); (event_id, user_id) |
| **jobs, outbox** | `sys_outbox, sys_scheduler_lock, sys_scheduled_item_retry, sys_checkout_expiry_cursor` | V13, V17, V25, V26 | one lease per job; outbox id = eventId |

28 Flyway files, 29 tables in schema `domain`; V1 sets the rules: snake_case, timestamptz, numeric money, invariants in the database. On ECS (ADR 0008): one database per service on one RDS instance; identity-svc and notification-svc own their rows above, commerce-svc the rest.

YQ Yorik

# Expiry and refund sequence




As built for ECS. The SNS → SQS hop and notification-svc (steps 2–8) are the ECS topology; on dev the monolith runs no broker.

YQ Yorik

# UC6 refund recovery: candidates and implementation decisions

| Candidate | Decision | Why |
|---|---|---|
| Mark refunded, then call Stripe | **Rejected** | A crash would report money returned that never left |
| Hold a DB transaction across the call | **Rejected** | A rollback cannot undo a refund; locks held over network I/O |
| Idempotency Key per obligation | **Selected** | Every retry reuses one request identity |
| Retry after a delay | **Selected** | Fixed 30 s in code, not exponential backoff |
| Human-in-the-loop review | **Selected** | Mismatched or ambiguous work leaves automatic claims |
| Distributed transaction with Stripe | **Rejected** | No shared protocol across PostgreSQL and the provider |

## Implementation decisions

1. Obligation commits with the expiry, before any I/O
2. Key from the obligation ID: `"blocbuy-refund-" + id`
3. Short claim and finish transactions; lease and version fence workers
4. Verify payment facts before, provider snapshot after
5. After 23 h: search Stripe by metadata, never create
6. Reconcile only retrieves an existing refund; no admin endpoint exists

*Source: docs/swe5006/report-drafts/yorick-yu-contribution.md. Evidence is local with a provider substitute.*

LJ Liu Jin

# Test inventory

| Kind · level | Tools | Count by module | Example classes | Runs |
|---|---|---|---|---|
| **Unit** · unit | JUnit Jupiter, Mockito | 185 *Test classes (162 unit; 23 Spring/Testcontainers), 1,329 annotations: domain 42, application 41, infrastructure 47, bootstrap 46, adapter 4, platform 2, commerce-svc 2, client 1 (some, like StripeCheckoutFlowTest, use real PostgreSQL) | `RefundServiceTest`, `PaidGroupDealTest`, `DealStatusTest`, `ProductStatusTest` | every PR (build-quality, required) |
| **Integration** · integration | Testcontainers: PostgreSQL 18, Redis 8.10, ElasticMQ | 49 *IT classes (45 start containers), 242 annotations: bootstrap 24, infrastructure 15, services 9, platform 1 | `OrderRepositoryIT`, `TeamExpirySchedulingIT`, `StripeRolloutAcceptanceIT` (acceptance) | every PR (build-quality, required) |
| **Architecture** · unit | ArchUnit | 17 rules in 1 class | `ArchitectureTest` | every PR (build-quality, required) |
| **API contract** · integration | springdoc export, diff of contracts/, openapi-diff | 1 contract file | `contracts/openapi.json` | backend PRs (contract-check, not required) |
| **Component** · unit | Vitest, Testing Library, MSW | 77 files: features 58, routes 10, components 4, test 3, api 2 | `account.test.tsx`, `admin-listing-edit.test.tsx` | every PR (frontend-check, required) |
| **End-to-end** · system | Playwright, Chromium | 8 specs, 24 test declarations | `buyer-journey.spec.ts`, `join-refusal.spec.ts`, `smoke.spec.ts` | PR (e2e-preview); after deploy (e2e-dev) |
| **Performance** · system | k6 1.6.1 (SWE5001 ECS); Node workload (local) | k6 browse V2/V3; 3 local scenarios | `perf/k6/browse.js`, `backend/perf/workload.mjs` | by hand (k6 tooling tested in CI) |

Counted 22 Oct at a1c6ac6e: 186 `*Test.java` (185 + ArchitectureTest), 49 `*IT.java` (45 start containers). Latest full runs: backend 2,048 passed, 1 opt-in skip (21 Oct log, local); frontend 799 (19 Oct log).

PS Pan

# Workflows, required checks and timings

| Workflow · trigger | Jobs (required in blue) | Median | Green |
|---|---|---|---|
| **ci-backend** · PR | **required**: build-quality (13.1 min), docker (2.8 min), SonarCloud Backend; contract-check (12 s) | 16.1 min to docker end (18 runs, 16–19 Oct) | 18 of 20 finished runs; 16 cancelled by a newer push |
| **ci-backend** · push to dev | package-dev 46 s → publish-ecr-dev 73 s → deploy-existing-ec2 40 s; SonarCloud Backend refreshes the dashboard | 3.1 min to deployed (88 runs) | deploy 88 of 97 in this sample; its 9 failures 14–17 Sep (SSM: instance not in a valid state) |
| **ci-frontend** · PR | **required**: frontend-check (3.4 min), SonarCloud Frontend; e2e-preview (69 s), intent-stale | 4.8 min to e2e-preview end (19 runs) | 19 of 20 finished runs |
| **ci-frontend** · push to dev | package-dev 35 s → deploy-s3-dev 21 s → e2e-dev 82 s; SonarCloud Frontend | 2.7 min to e2e-dev green (94 runs) | deploy 97 of 97; e2e-dev 94 of 97 (3–4 Oct) |
| **security** · PR, dev, weekly | **required**: gitleaks (10 s); trivy-deps (72 s), trivy-config, workflow-lint, deployment-bundle when their paths change | 12 s on most PRs | PR 97 of 98 finished; dev push 100 of 100 |
| **dast-baseline** · weekly, manual, scan PRs | zap-baseline: passive OWASP ZAP of anonymous dev pages; findings advisory, coverage or scanner errors fail it | — | PR runs 22 Oct green; dev dispatch 24 Oct green (added after this read) |
| **sbom** · tag, weekly, manual, run-sbom label | sbom: CycloneDX source SBOMs and licence inventories; licences advisory, a missing inventory fails it | — | PR runs 22 Oct green on earlier PR heads (added after this read) |
| **rollback-backend** · manual | validate, then SSM rollback to the recorded previous digest or a given one | — | never run |
| **SWE5001** · prod branch | swe5001-release-gate, swe5001-deploy, infra-swe5001 (promotion PR; see the SWE5001 deck) | — | — |

Last 100 runs per workflow and event, read with `gh run list` and the jobs API on 19 Oct. PR durations count only runs where the gated jobs ran (docs-only PRs skip them in about 10 s). Push medians run from the push run start, not the merge click.

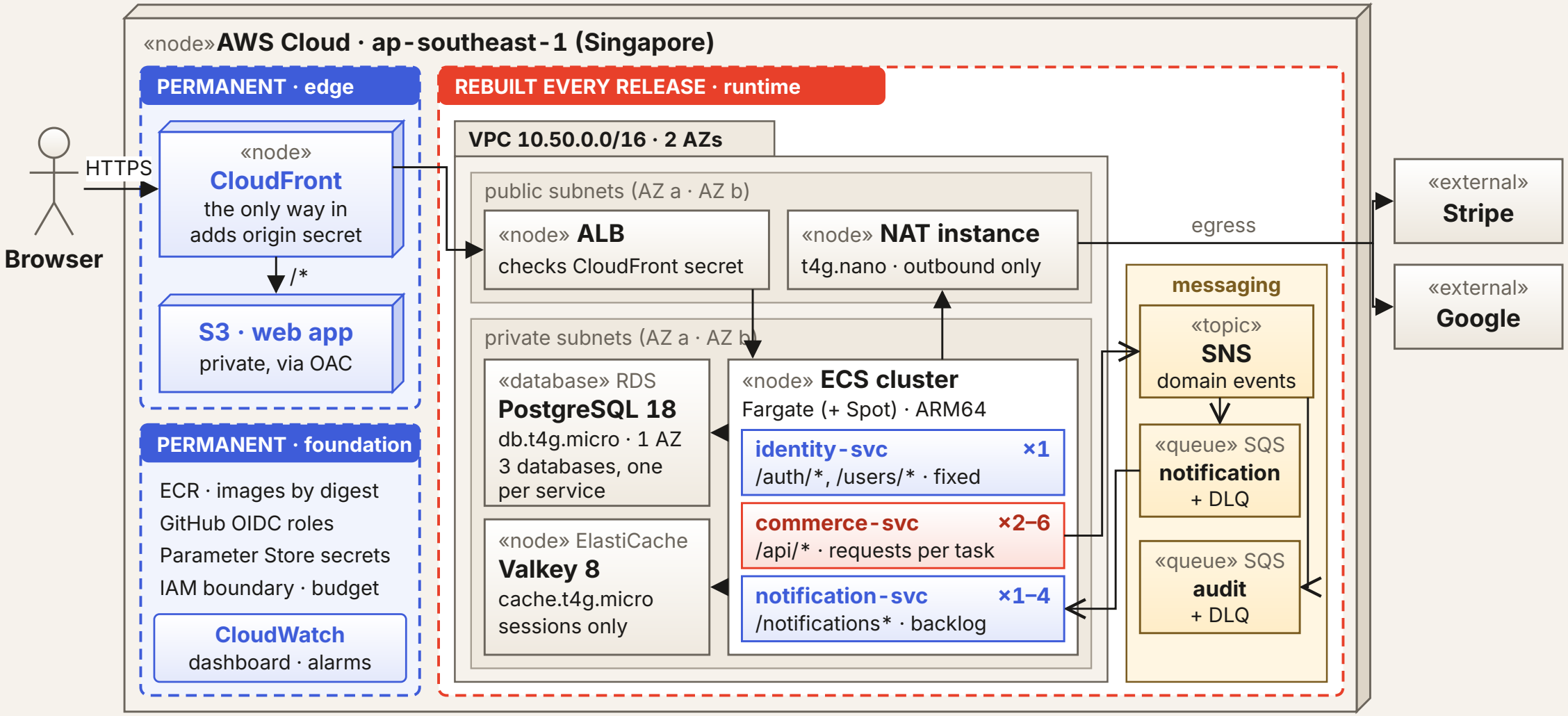
PS Pan

# Known gaps and limitations

| Gap | Effect | Status |
|---|---|---|
| CloudFront → ALB over HTTP | /api payloads and the origin header cross the AWS network unencrypted | NFR-37 TLS still unmet; accepted risk on dev until 16 Nov (#227, NFR register); fix needs an origin hostname and ACM |
| SSH open to the internet | port 22 from 0.0.0.0/0 on the dev host, which has a public IP; SSM sessions make it unnecessary | AWS read-only check, 21 Oct; open item 9 |
| Dev EBS volume not encrypted | database and host data at rest unencrypted | AWS read-only check, 19 Oct; not raised as an issue yet |
| Rollback never exercised | recovery path untested on the real host | rollback-backend.yml: 0 runs; no automatic rollback has fired |
| No dev alarms | failures are noticed by e2e-dev or by people | all six CloudWatch alarms belong to SWE5001 |
| Dev not zero-downtime | a deploy restarts the only API container (502 on 3–4 Oct) | accepted for dev |
| Secrets in a host file | manual creation and rotation, root-only 0600 | accepted for dev; prod uses Parameter Store |
| Delivery image rebuilt | dev delivery rebuilds the backend JAR and image without repeating the Trivy image gate, so the deployed image is not the PR-scanned one; SonarCloud, deploy health checks and the frontend smoke still run | artifact-assurance limit; ECR scans on push (not a gate); ADR 0004 text to reconcile (open item 18) |
| Weekly full-history gitleaks red | 20 digest false positives fingerprinted; 4 badge-token findings still fail the scan | PR #580, ci-failure-audit-20261021.md |
| Passive DAST; SBOM not per image | OWASP ZAP of dev is anonymous and passive: no logins, payments or active attacks; advisory, 2 Medium, 4 Low CSP and header findings (22 Oct), fixes in #610; the CycloneDX SBOMs list the backend Maven and frontend pnpm source dependencies, not the exact deployed JAR, image or bundle; licence findings advisory | #607, #610, #609 |

PS Pan

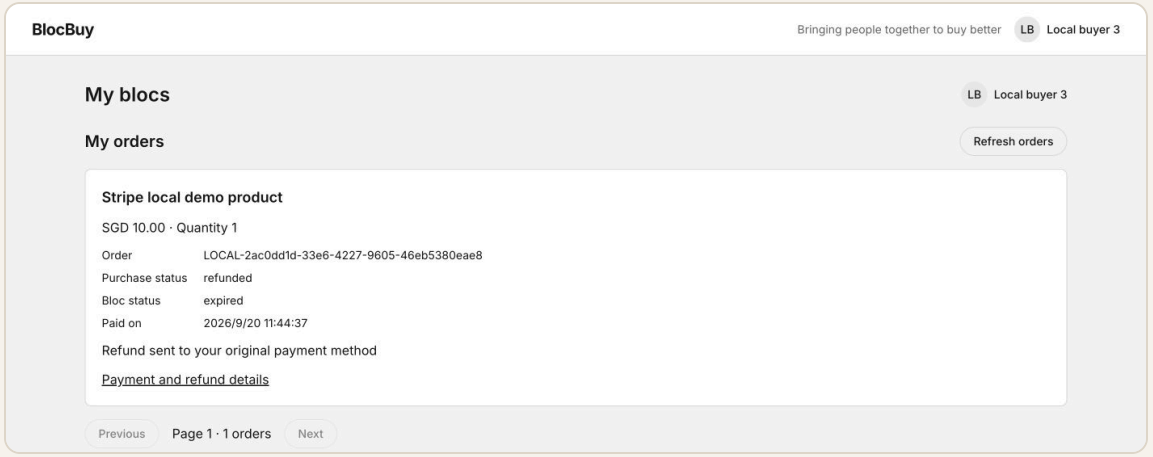
# SWE5001 production topology



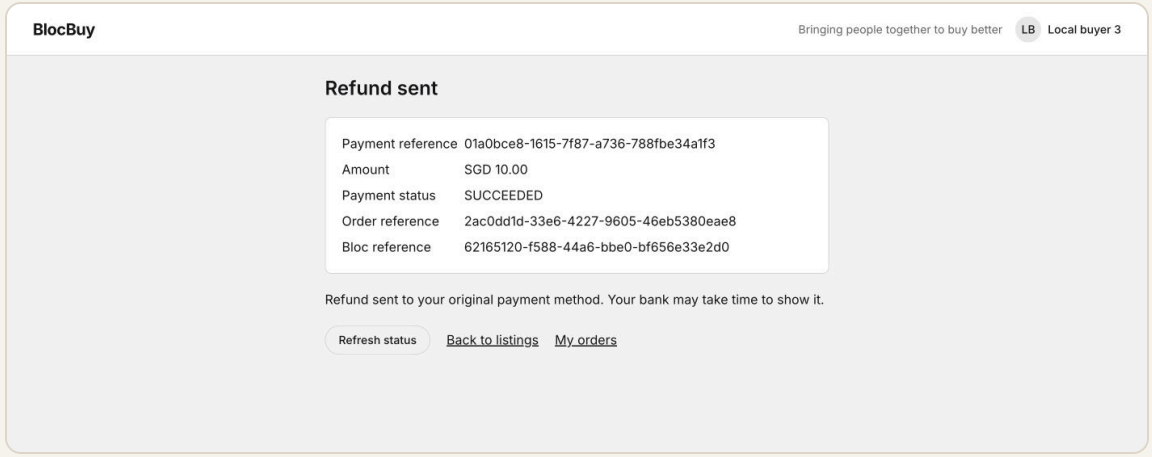

SWE5001 prod: the same code on ECS, rebuilt from scratch on every release and destroyed after it. SWE5006 dev is slide 31.

YQ Yorik

# Stripe Sandbox refund, local evidence (20 Sep)




My orders: the expired bloc's order is refunded (UI of 20 Sep, since redesigned)




Receipt: "Refund sent", payment 01a0bce8..., SGD 10.00

| Scenario | Result on 20 Sep (Team8 Stripe Sandbox, livemode=false) |
|---|---|
| START and JOIN | two SGD 10 payments → one member and one order each; the bloc reached succeeded, 2 of 2 |
| Webhook replay | replayed valid events changed nothing; a modified payload with the old signature got 400 |
| Expiry | an under-filled bloc expired → one SGD 10 refund obligation; its unpaid Checkout session expired |
| Refund | one full Sandbox refund `re_3UHbeV5iEVLN2uQJ1V71krwQ`, succeeded; a recheck created no duplicate |

**Local, not dev:** real Stripe Sandbox calls from a local backend and PostgreSQL; no Google sign-in, no deployed environment. On dev the refund waits for #223 and #517. Source: `backend/docs/payment/stripe-sandbox-acceptance-20260920.md`

YQ Yorik

# Scale-out and load results (SWE5001 ECS + local baseline)

SWE5001 PROD · ECS FARGATE

## Scale-out

- Target tracking on ALB requests per target: 12,000 a minute, scale-out cooldown 60 s
- commerce 2–6 tasks in two AZs, notification 1–4 (read back 16 Oct)
- V3, 19 Oct: commerce **1 → 3 tasks** in about four minutes, no manual scaling; at 3 tasks ≈ 455 req/s, server p95 13–50 ms, 2 errors in 570,445
- Not V3 acceptance: the workstation generator dropped iterations, so the analyzer rejects the run
- V4 queue drain: not run on AWS; still open in #458

SWE5001 V2 · ECS, 16 OCT

## Load, one task

- k6, ten one-minute plateaus from 10 to 200 requests a second, through CloudFront
- 57,600 requests, **0 errors**, run twice
- p95 between 23 and 57 ms on every plateau (target under 500 ms)
- No saturation reached: 200 a second is a tested lower bound, not a limit

DEV CODE · LOCAL RUN, 30 SEP

## Workload baseline

| 100 users each | p95 | Errors |
|---|---|---|
| Browse (list + detail) | 24 ms | 0 |
| JOIN intent, unpaid | 57 ms | 0 |
| My Orders | 18 ms | 0 |

30 paid teams expired within 58 s of their deadlines under that load. One laptop with real HTTP, PostgreSQL and Redis; **not a deployed result**.

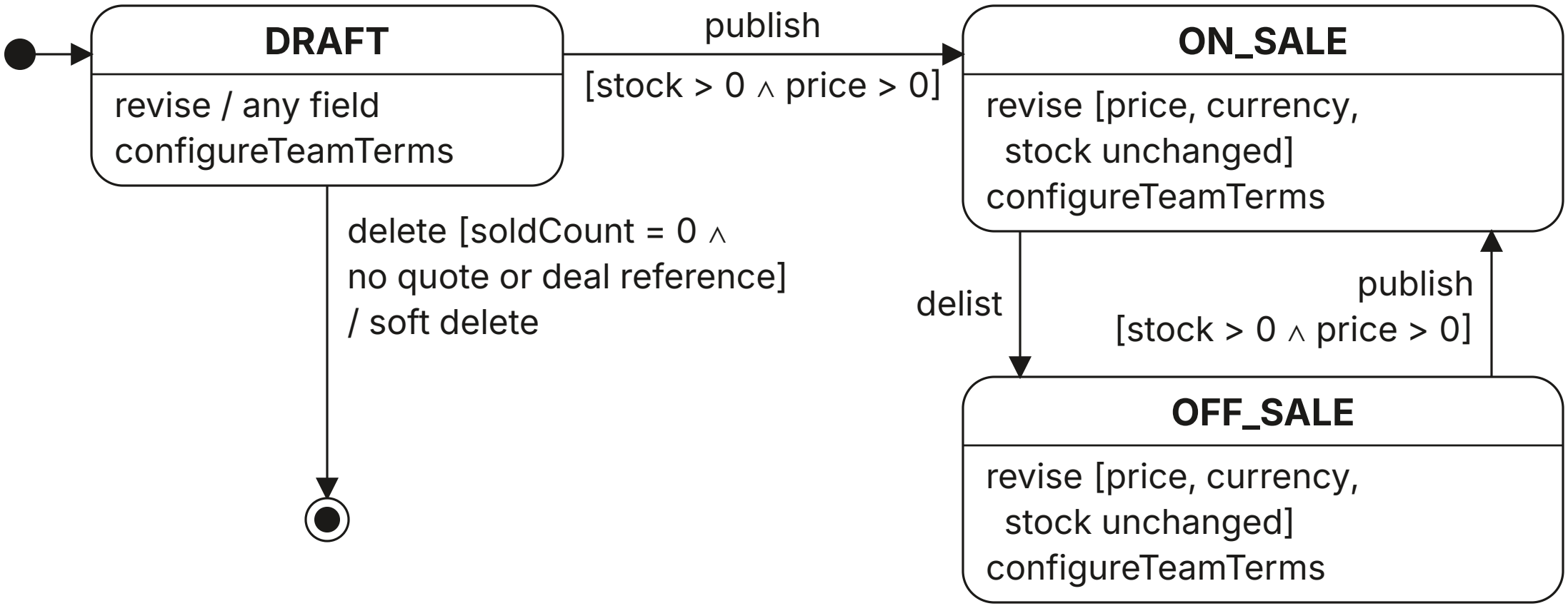
**Dev, where the demo runs:** one EC2 host by design (backend ADR 0008), so no auto-scaling there. Same code, two topologies: scale-out exists only in the SWE5001 topology and is not claimed for SWE5006 (slide 37).

*Sources: docs/swe5001/evidence/20261016-http-baseline-dashboard/README.md (V2, scaling policy); docs/swe5001/evidence/20261019-v3-autoscaling/README.md (V3); docs/perf/sprint3-baseline.md (local workload).*

LJ Liu Jin

# UC9 listing statechart

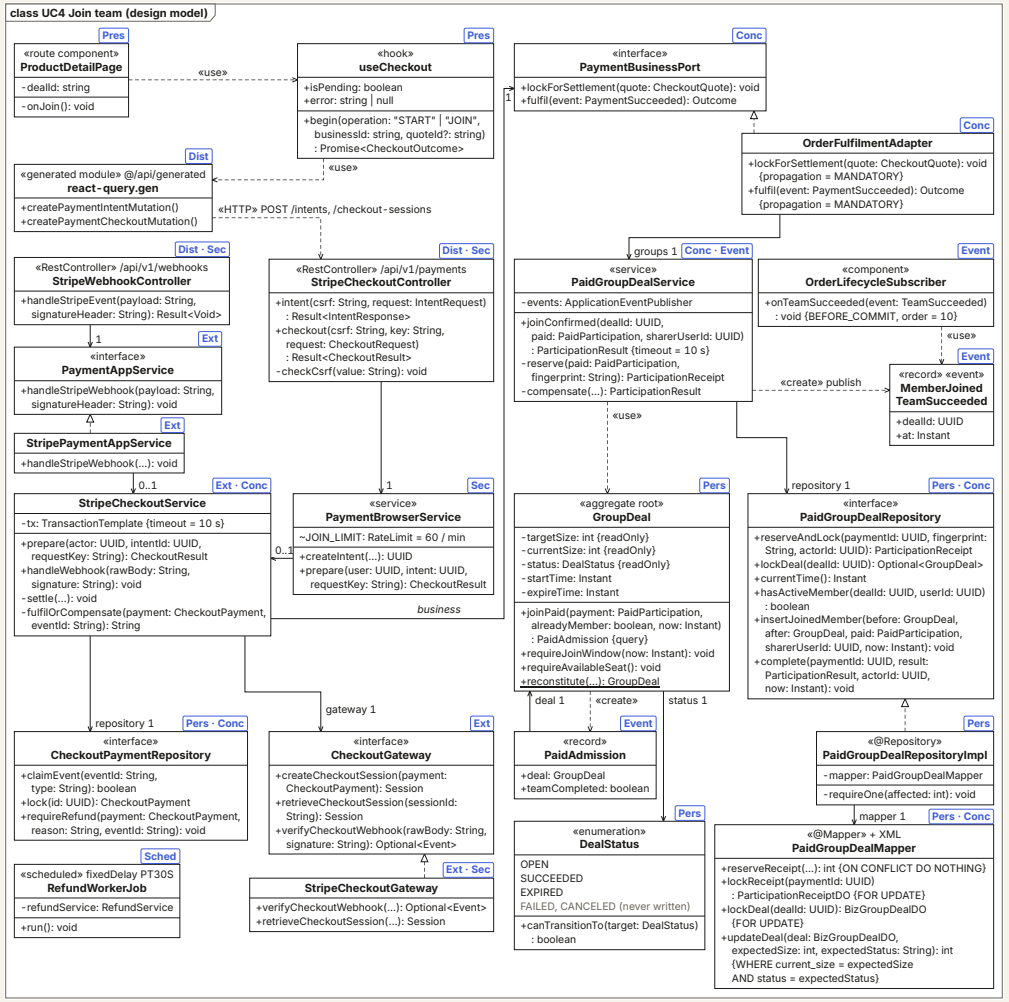
## stm Listing lifecycle (Product.status)



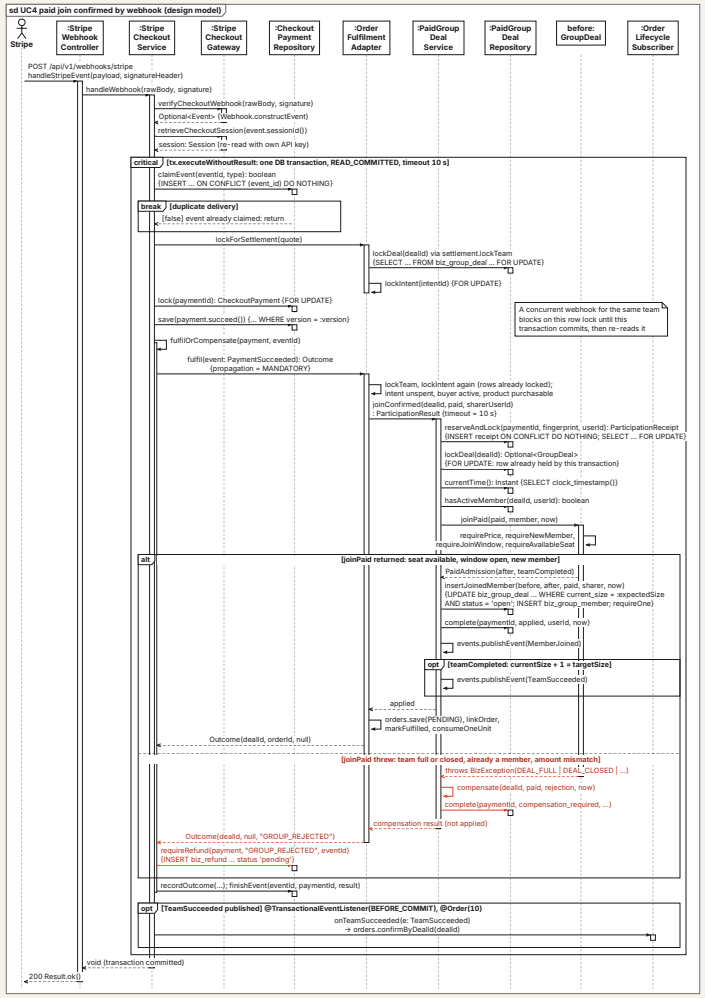

Nothing returns to DRAFT; any other event → 409, state unchanged.

WZ Eric

# UC4 design class and sequence diagrams, complete




Design class diagram (2320 × 2314)




Design sequence (2320 × 3306)

Both are vector: open `src/diagrams/uc4-design-*.svg` in a browser tab and zoom. Every message carries its source line in the diagram spec. Report §3.4.1 (Eric).

WZ Eric

# Every member’s use case and design problem

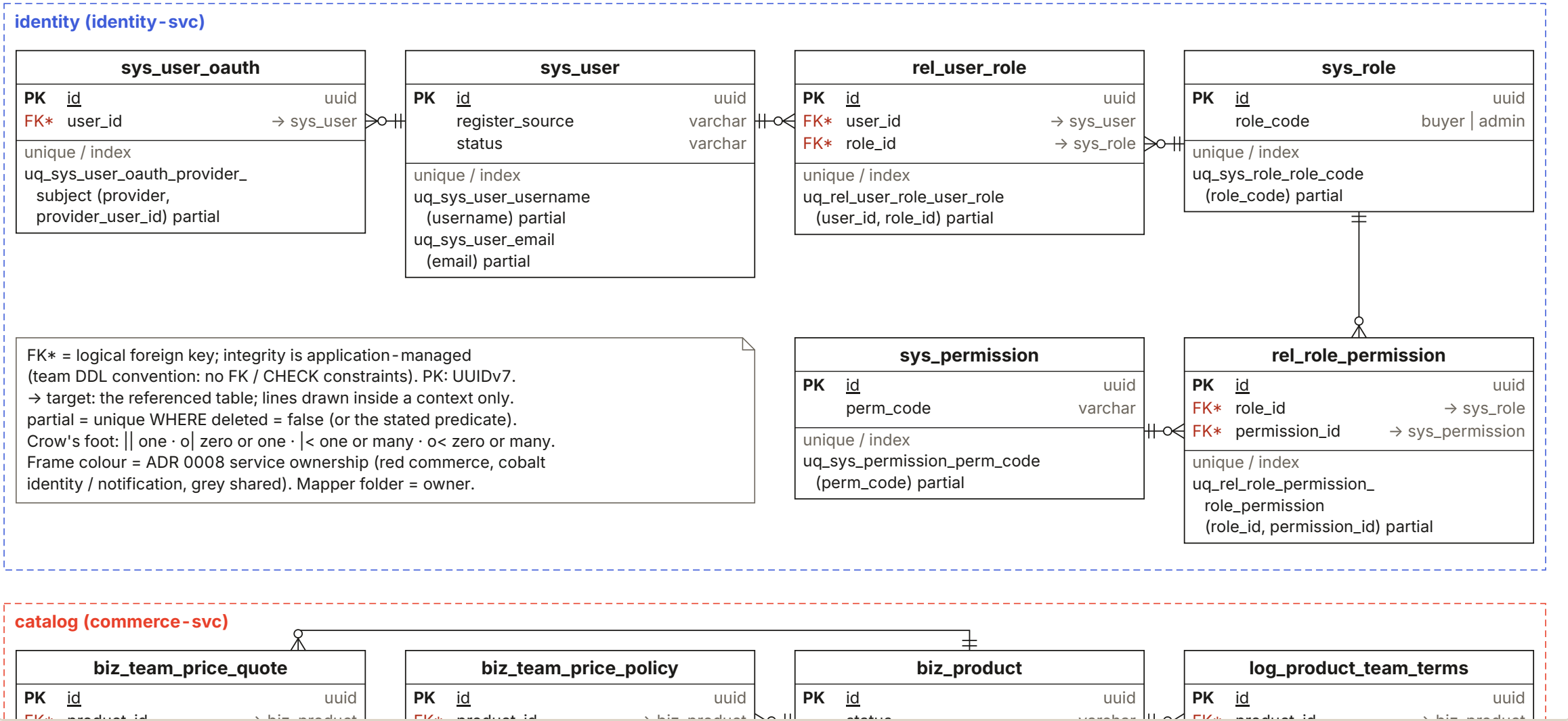
| Member | Use case (report section) | Design problem (report section) | On a slide |
|---|---|---|---|
| Eric | §3.4.1 UC3, UC4 start and join a team, pay upfront | §3.5.1 Factory Method: team creation | UC4 models 14–16, statechart 17; pattern 18 |
| Yorik | §3.4.2 UC6 refund on team expiry | §3.5.2 Observer: team outcome; refund idempotency as a mechanism (B6) | pattern 19; sequence B5 |
| Pan | §3.4.3 UC1 browse listings, UC2 register and sign in | §3.5.3 Template Method: outbox transports | pattern 20 |
| Lim | §3.4.4 UC8 my orders and my teams, UC7 share attribution | §3.5.4 Strategy: UC1 pricing | pattern 21 |
| Liu Jin | §3.4.5 UC9 listing administration (first version by Eric, PR #72), UC5 team lifecycle with the team statechart | §3.5.5 State: listing lifecycle (full form) | patterns 22–23; statechart B13 |

Each member’s analysis class, analysis sequence, design class and design sequence diagrams are in report §3.4.1–§3.4.5 (template RT 3.4); the deck shows UC4 only (rubric G8). Owners: #463, decided 24 Oct; notifications are SWE5001 scope.

LJ Liu Jin

# Entity-relationship diagram, all 29 tables (scroll; 1:1 pages B16-1 to B16-21 follow)

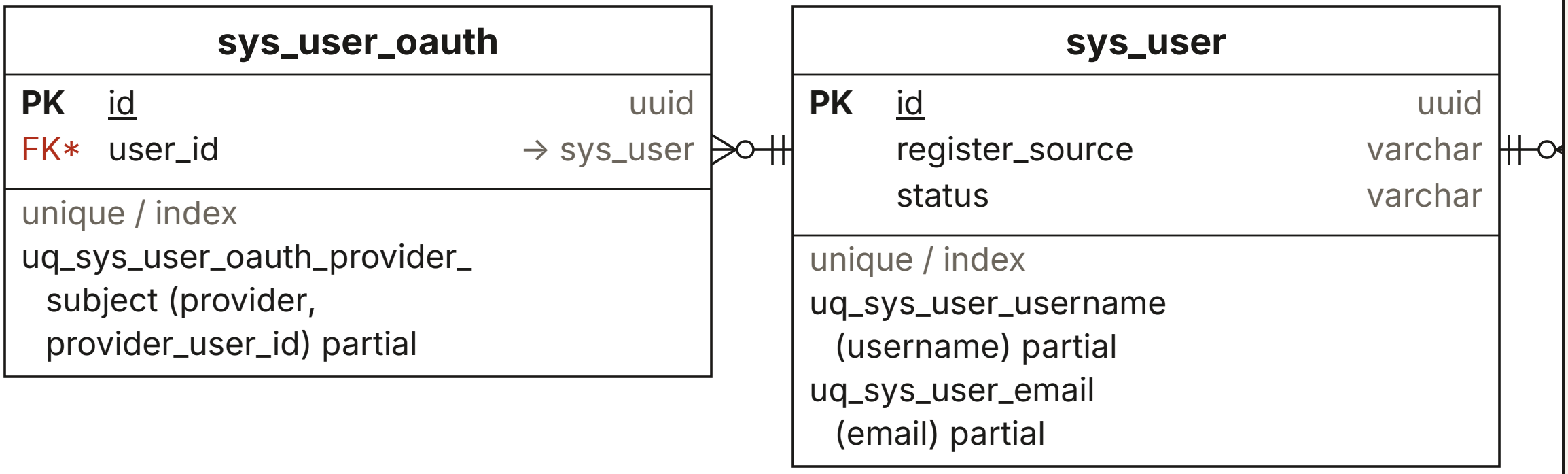
**erd Full schema of the SWE5006 monolith (Flyway V1–V28, schema domain), by owning context**



LJ Liu Jin

# ERD 1:1 · identity (left, part 1/2) · 1/21

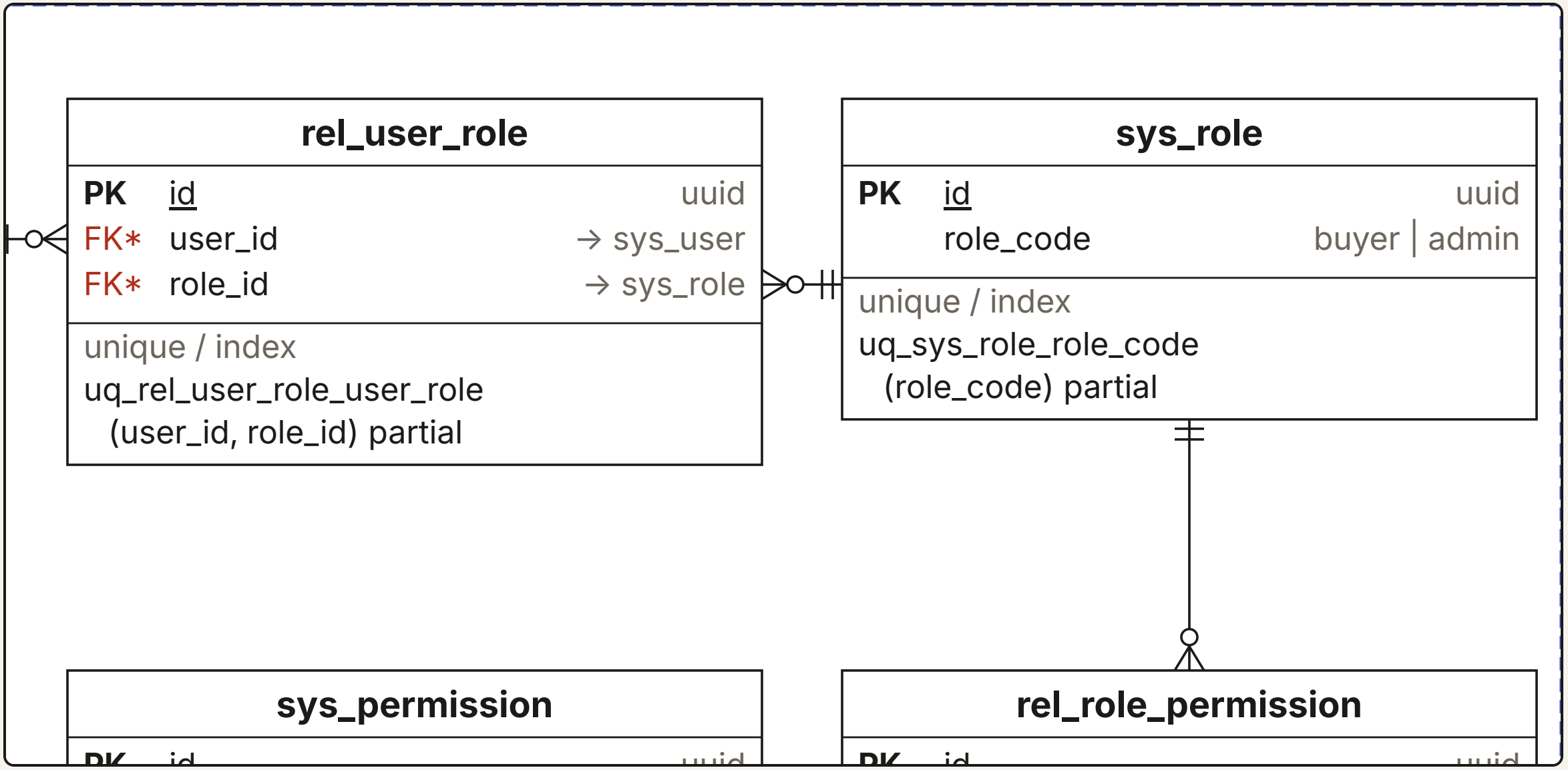
## identity (identity-svc)




FK* = logical foreign key; integrity is application-managed (team DDL convention: no FK / CHECK constraints). PK: UUIDv7.

LJ Liu Jin

# ERD 1:1 · identity (right, part 1/2) · 2/21



LJ Liu Jin

# ERD 1:1 · identity (left, part 2/2) · 3/21

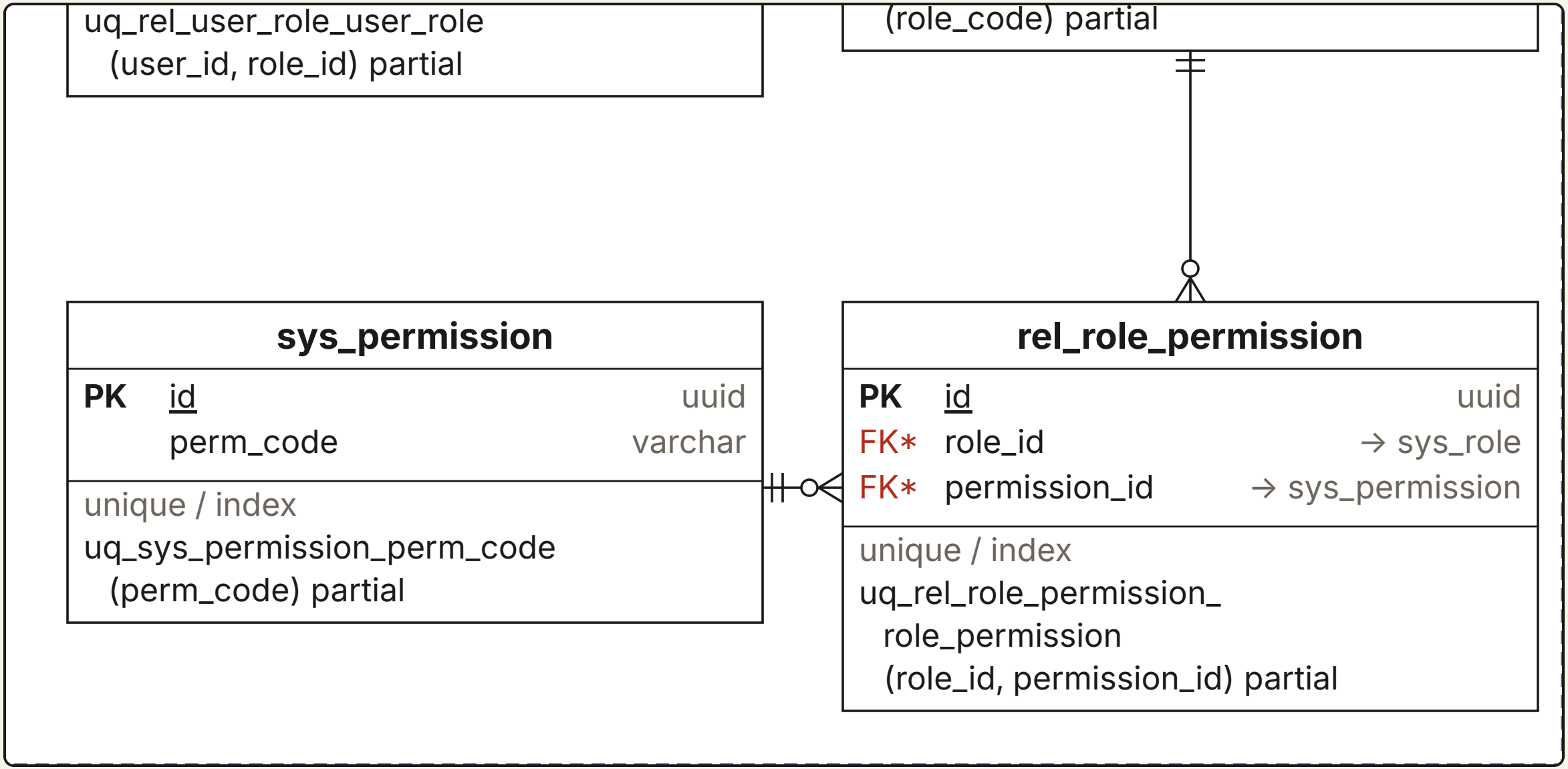
subject (provider,
provider_user_id) partial

uq_sys_user_username
(username) partial
uq_sys_user_email
(email) partial

FK* = logical foreign key; integrity is application-managed
(team DDL convention: no FK / CHECK constraints). PK: UUIDv7.
→ target: the referenced table; lines drawn inside a context only.
partial = unique WHERE deleted = false (or the stated predicate).
Crow's foot: || one · o| zero or one · |< one or many · o< zero or many.
Frame colour = ADR 0008 service ownership (red commerce, cobalt
identity / notification, grey shared). Mapper folder = owner.

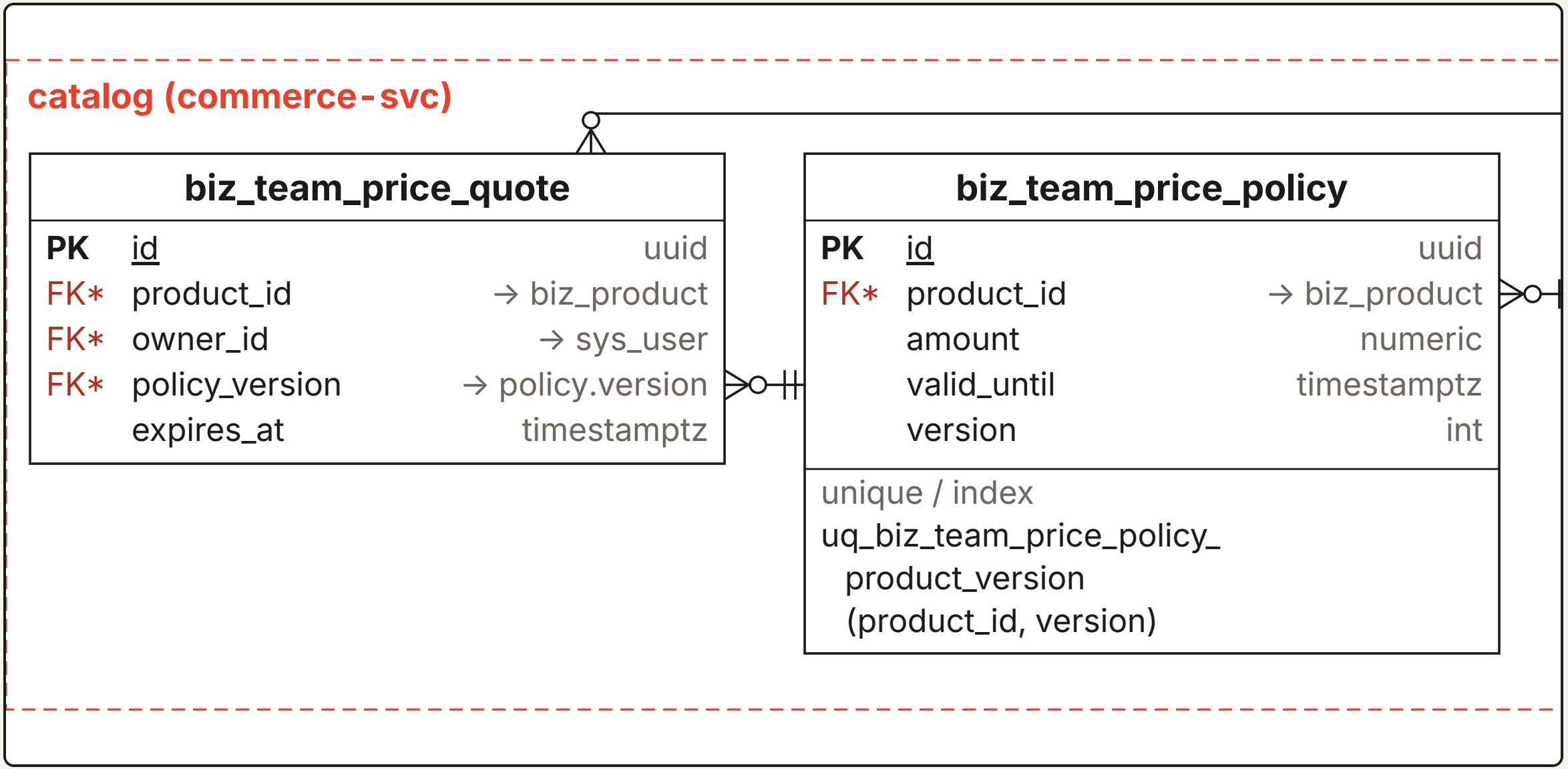
LJ Liu Jin

# ERD 1:1 · identity (right, part 2/2) · 4/21



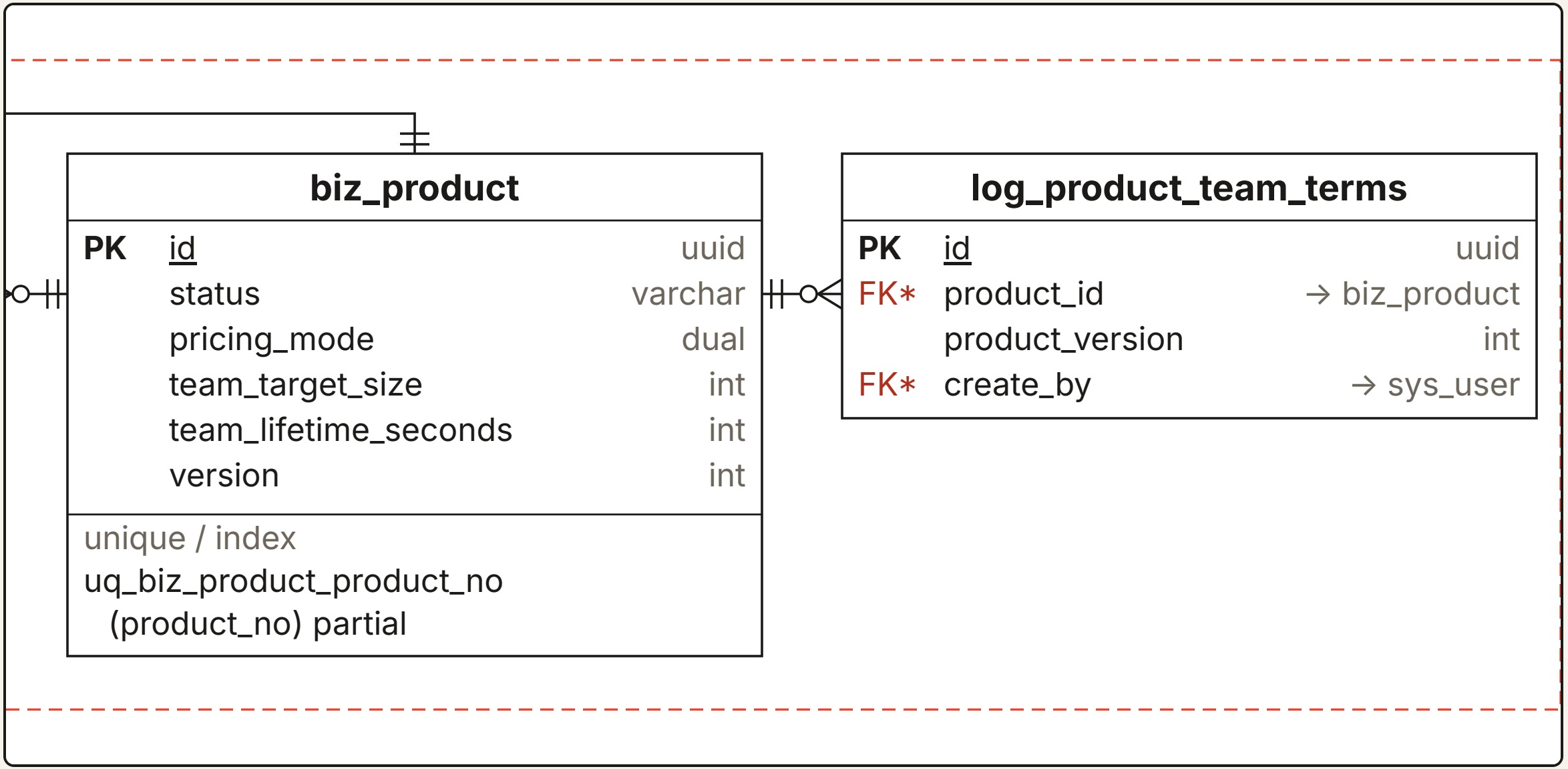
LJ Liu Jin

# ERD 1:1 · catalog (left) · 5/21



LJ Liu Jin

# ERD 1:1 · catalog (right) · 6/21



LJ Liu Jin

# ERD 1:1 · groupbuy + share (left, part 1/2) · 7/21

## groupbuy + share (commerce-svc)

### biz_group_member

| | | |
|---|---|---|
| PK | id | uuid |
| FK* | deal_id | → biz_group_deal |
| FK* | user_id | → sys_user |
| FK* | payment_id | → biz_payment |
| FK* | order_id | → biz_order |
| FK* | sharer_user_id | → sys_user |

unique / index

uq_biz_group_member_deal_user
(deal_id, user_id) partial

uq_biz_group_member_payment_id
(payment_id)

### biz_group_deal

| | | |
|---|---|---|
| PK | id | uuid |
| FK* | product_id | → biz_product |
| FK* | leader_user_id | → sys_user |
| | target_size, current_size | int |
| | status | varchar |
| | expire_time | timestamptz |
| | version | int |

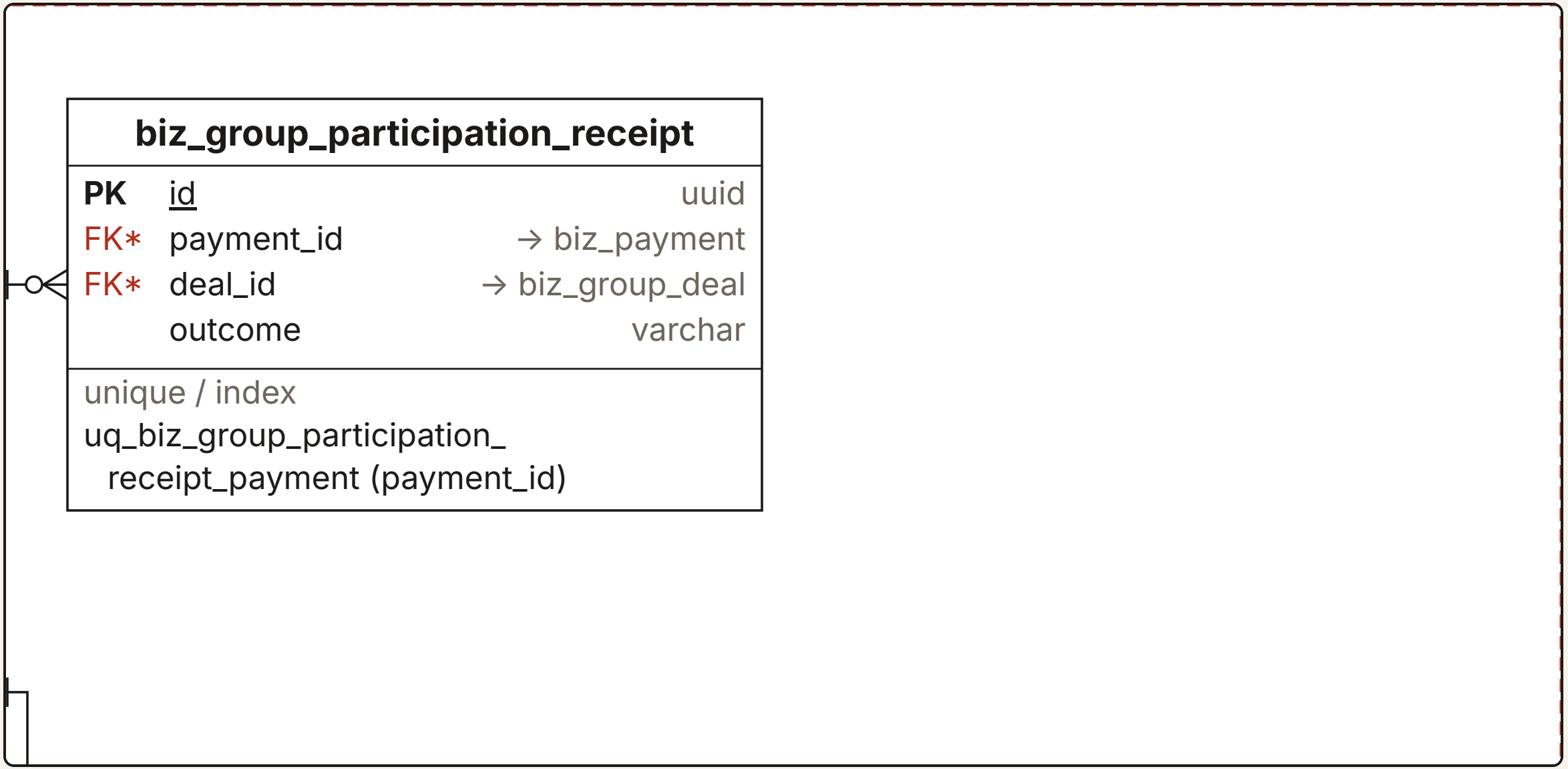
unique / index

uq_biz_group_deal_deal_no
(deal_no) partial

idx_biz_group_deal_status_expire
(status, expire_time) partial

LJ Liu Jin

# ERD 1:1 · groupbuy + share (right, part 1/2) · 8/21



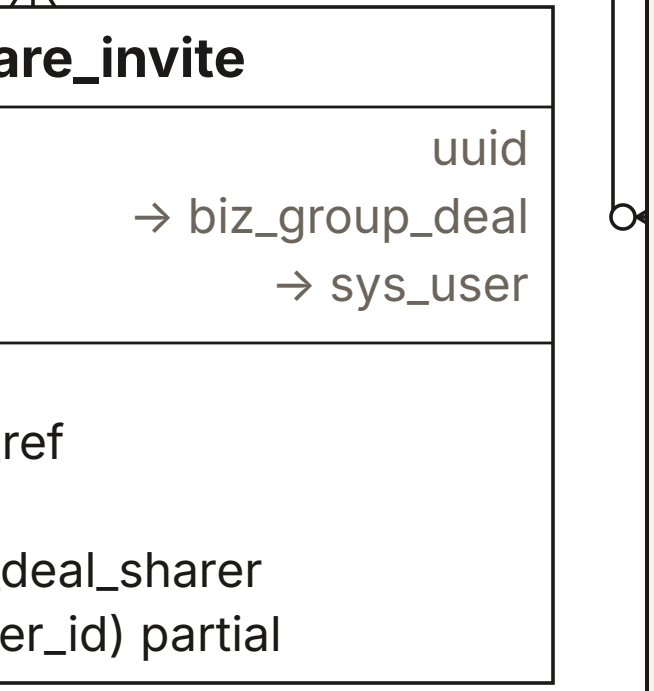
LJ Liu Jin

# ERD 1:1 · groupbuy + share (left, part 2/2) · 9/21

uq_biz_group_member_payment_id
(payment_id)

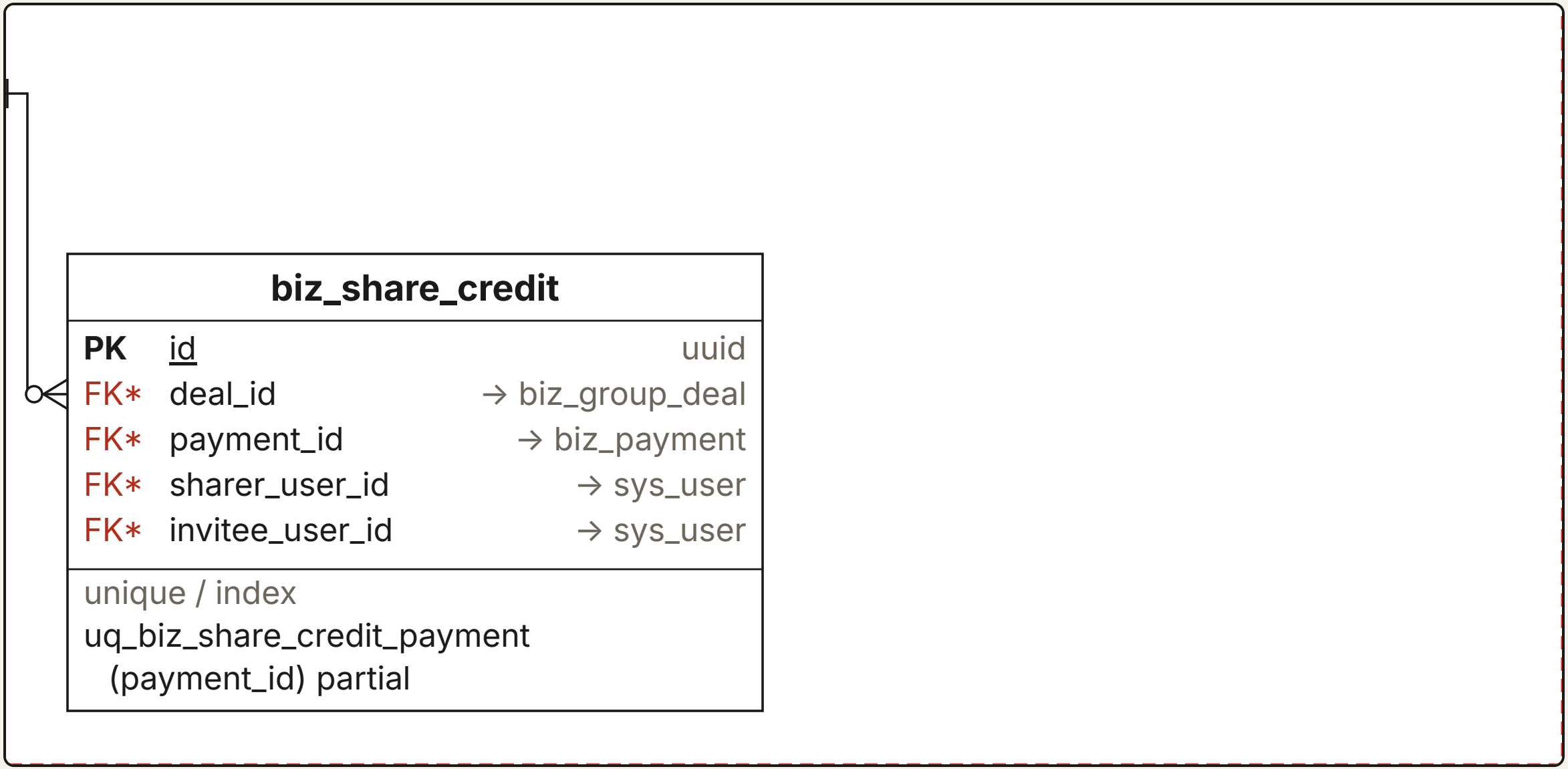
(deal_no) partial
idx_biz_group_deal_status_expire
(status, expire_time) partial





LJ Liu Jin

# ERD 1:1 · groupbuy + share (right, part 2/2) · 10/21



LJ Liu Jin

# ERD 1:1 · payment (left, part 1/3) · 11/21

## payment (commerce-svc)

### log_payment_webhook_event

| | | |
|---|---|---|
| PK | id | uuid |
| FK* | payment_id | → biz_payment |
| | event_id | varchar |

unique / index

uq_log_payment_webhook_event_event_id (event_id)

### biz_payment

| | | |
|---|---|---|
| PK | id | uuid |
| FK* | order_id | → biz_order |
| FK* | intent_id | → biz_purchase_intent |
| FK* | user_id | → sys_user |
| FK* | business_id | → biz_group_deal |
| FK* | fulfilled_deal_id | → biz_group_deal |
| | status | varchar |
| | version | int |

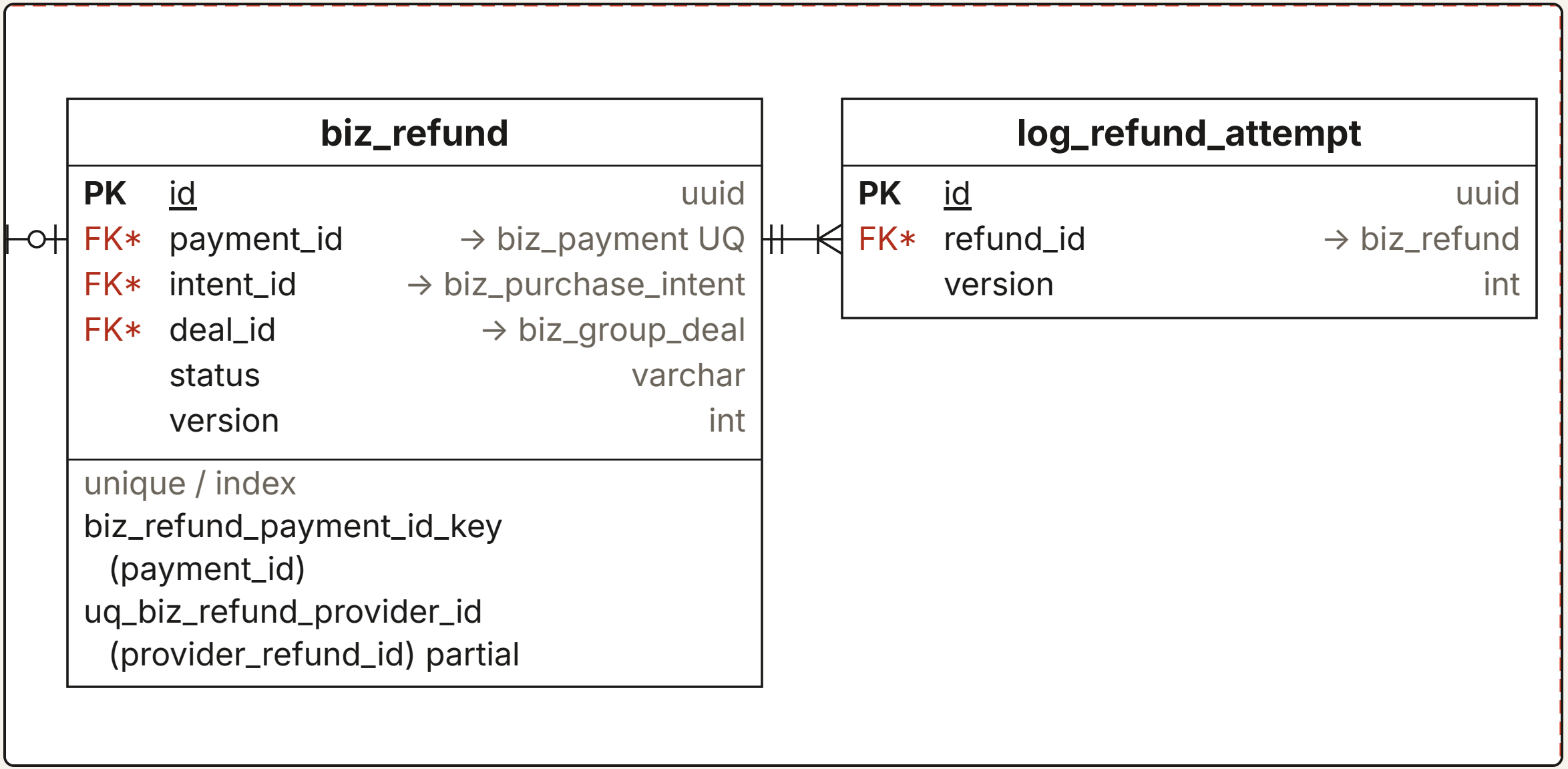
unique / index

uq_biz_payment_payment_no (payment_no) partial

uq_biz_payment_provider_intent (provider, provider_payment_

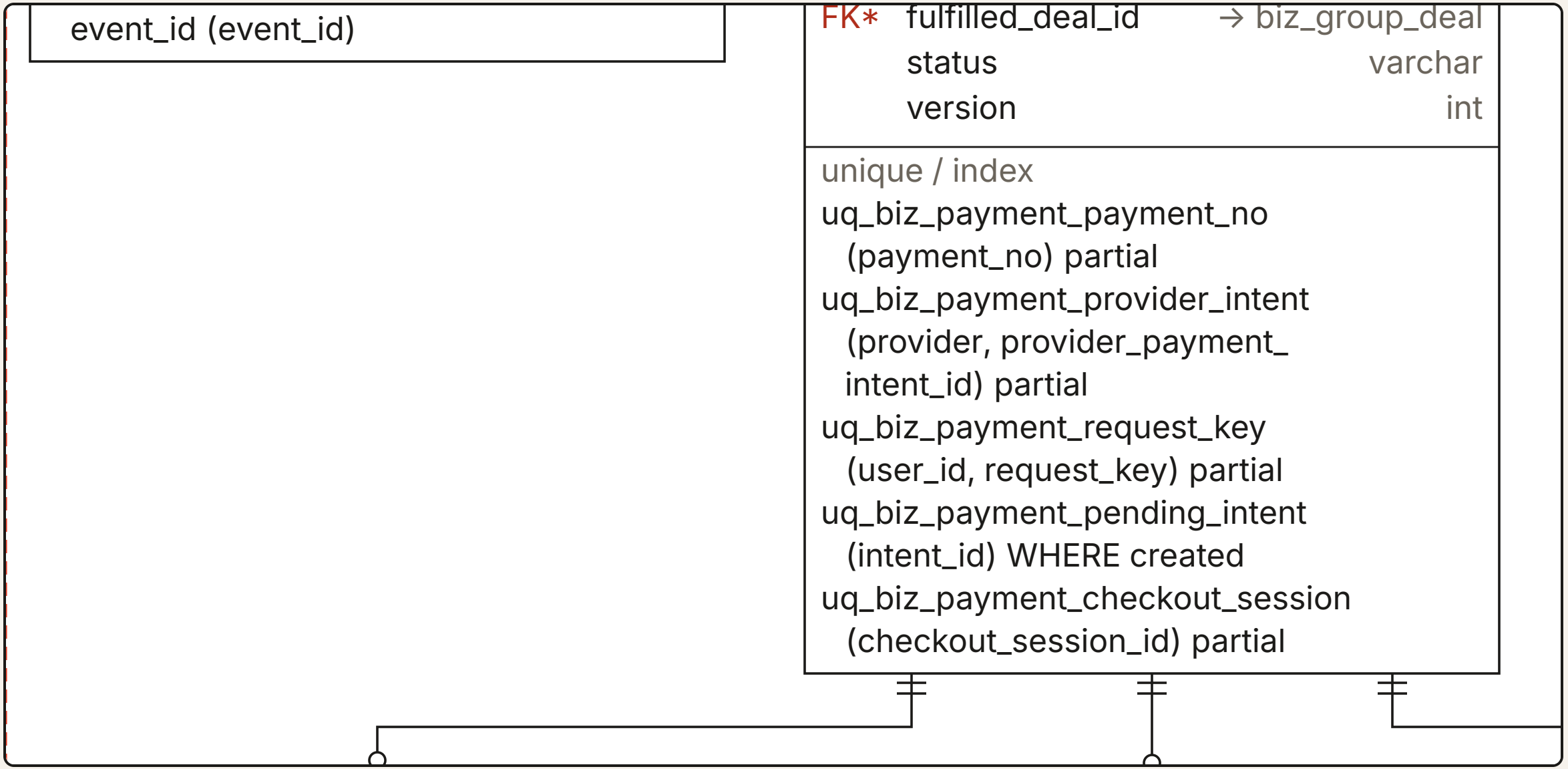
LJ Liu Jin

# ERD 1:1 · payment (right, part 1/3) · 12/21



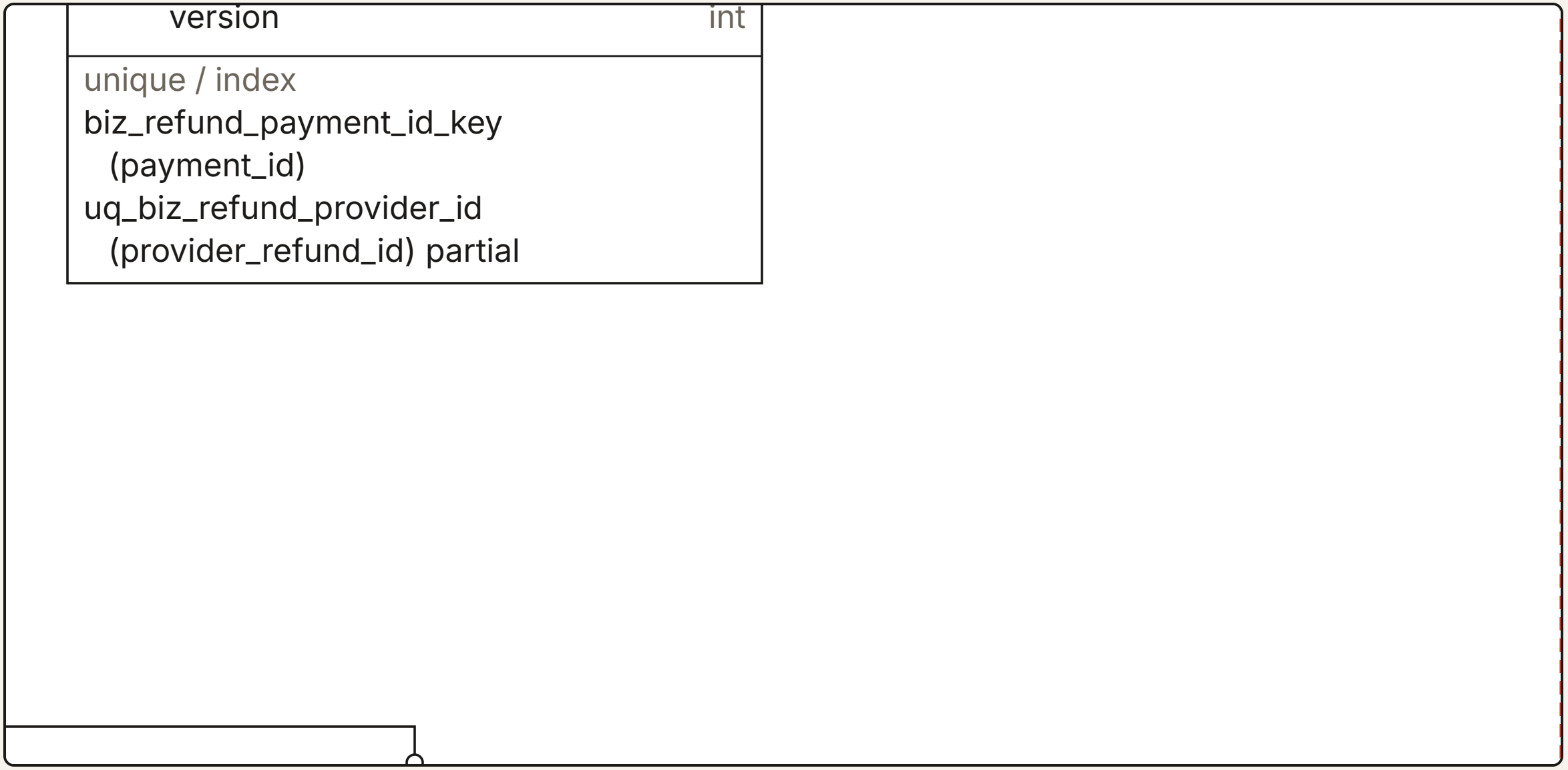
LJ Liu Jin

# ERD 1:1 · payment (left, part 2/3) · 13/21



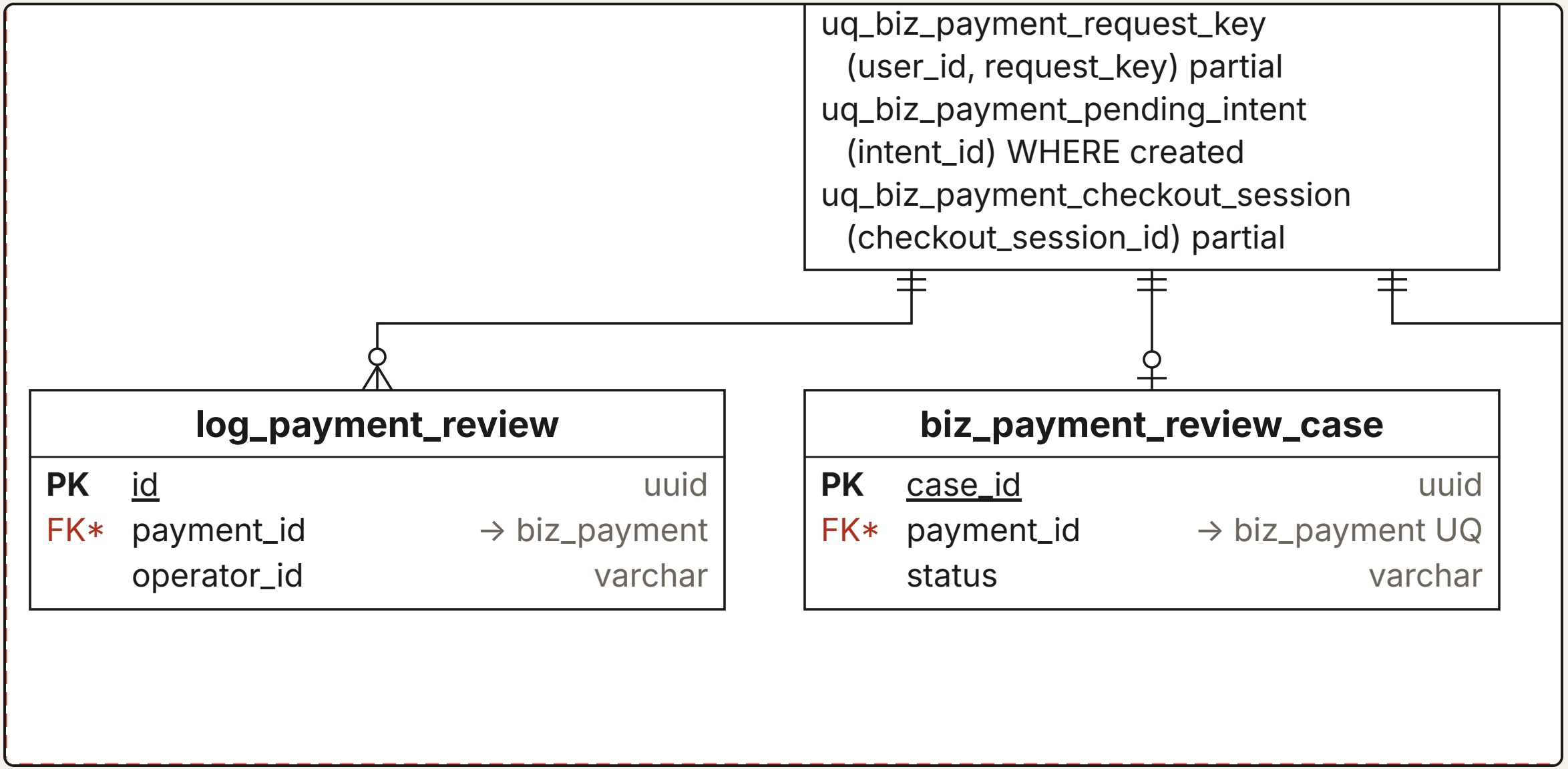
LJ Liu Jin

# ERD 1:1 · payment (right, part 2/3) · 14/21



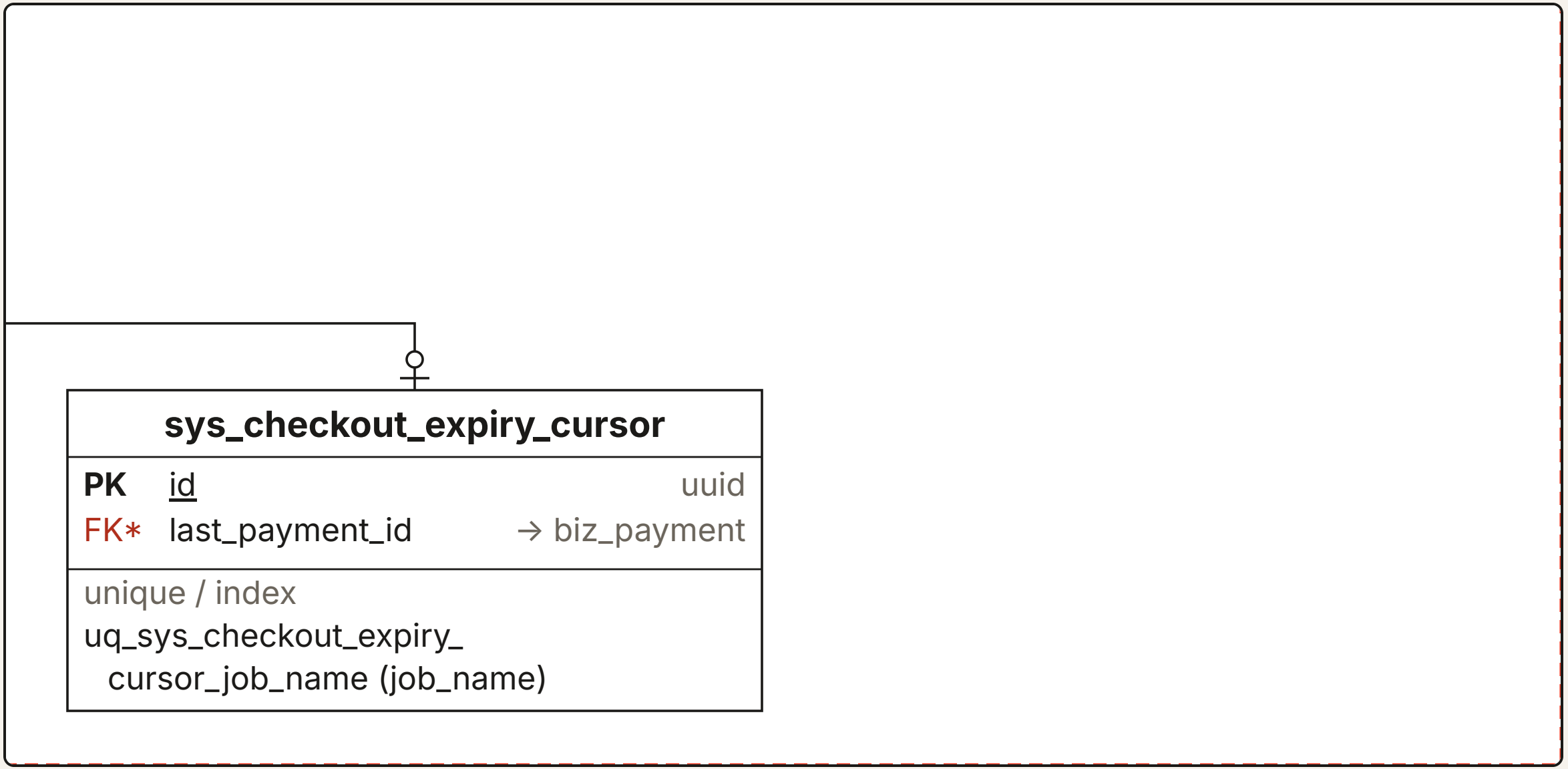
LJ Liu Jin

# ERD 1:1 · payment (left, part 3/3) · 15/21



LJ Liu Jin

# ERD 1:1 · payment (right, part 3/3) · 16/21



LJ Liu Jin

# ERD 1:1 · order (part 1/2) · 17/21

## order (commerce-svc)

### biz_purchase_intent

| | | |
|---|---|---|
| **PK** | id | uuid |
| FK* | user_id | → sys_user |
| FK* | deal_id | → biz_group_deal |
| FK* | product_id | → biz_product |
| FK* | fulfilled_payment_id | → biz_payment |
| FK* | sharer_user_id | → sys_user |
| | version | int |

unique / index

uq_biz_purchase_intent_fulfilled_payment (fulfilled_payment_id) partial

### biz_order

| | | |
|---|---|---|
| **PK** | id | uuid |
| FK* | payment_id | → biz_payment |
| FK* | user_id | → sys_user |
| FK* | deal_id | → biz_group_deal |
| FK* | product_id | → biz_product |
| | status | varchar |
| | version | int |

unique / index

uq_biz_order_order_no (order_no) partial

uq_biz_order_payment_id (payment_id)

uq_biz_order_deal_user

LJ Liu Jin

# ERD 1:1 · order (part 2/2) · 18/21

## biz_purchase_intent

| | | |
|---|---|---|
| PK | id | uuid |
| FK* | user_id | → sys_user |
| FK* | deal_id | → biz_group_deal |
| FK* | product_id | → biz_product |
| FK* | fulfilled_payment_id | → biz_payment |
| FK* | sharer_user_id | → sys_user |
| | version | int |

unique / index

uq_biz_purchase_intent_fulfilled_payment (fulfilled_payment_id) partial

## biz_order

| | | |
|---|---|---|
| PK | id | uuid |
| FK* | payment_id | → biz_payment |
| FK* | user_id | → sys_user |
| FK* | deal_id | → biz_group_deal |
| FK* | product_id | → biz_product |
| | status | varchar |
| | version | int |

unique / index

uq_biz_order_order_no (order_no) partial

uq_biz_order_payment_id (payment_id)

uq_biz_order_deal_user (deal_id, user_id) partial

LJ Liu Jin

# ERD 1:1 · notification · 19/21

## notification (notification-svc)

### biz_notification

| | | |
|---|---|---|
| **PK** | id | uuid |
| FK* | event_id | → event id |
| FK* | user_id | → sys_user |
| FK* | reference_id | → deal / payment |

unique / index

uq_biz_notification_event_user
(event_id, user_id)

### sys_consumed_message

| | | |
|---|---|---|
| **PK** | event_id | uuid |
| **PK** | consumer | varchar |

LJ Liu Jin

# ERD 1:1 · integration · scheduling (left) · 20/21

## integration · scheduling (shared)

### sys_outbox

| | | |
|---|---|---|
| **PK** | id | uuid = eventId |
| FK* | aggregate_id | → any aggregate |
| | published_at | timestamptz |
| | attempts | int |

unique / index

idx_sys_outbox_unpublished partial

idx_sys_outbox_published_retention partial

### sys_scheduler_lock

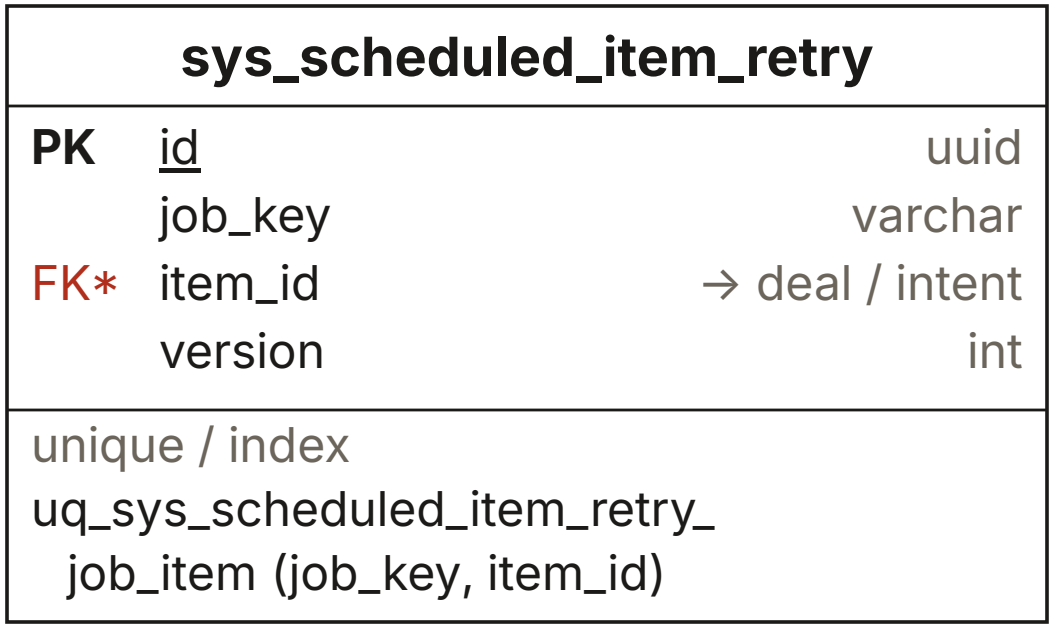
| | | |
|---|---|---|
| **PK** | id | uuid |
| | lock_key | varchar |
| | locked_until | timestamptz |
| | version | int |

unique / index

uq_sys_scheduler_lock_key (lock_key) partial

LJ Liu Jin

# ERD 1:1 · integration · scheduling (right) · 21/21



# Diagram details, 1:1: one page per crop

Slide 9 · Logical architecture: COLA layers acr...
D9-1 D9-2 D9-3

Slide 12 · One strategy pair: analysis objects b...
D12-1 D12-2 D12-3

Slide 13 · Use case diagram: in scope, out of s...
D13-1 D13-2 D13-3 D13-4 D13-5

Slide 14 · UC4 join: the analysis model
D14-1 D14-2 D14-3

Slide 15 · UC4 join: the design class diagram
D15-1 D15-2 D15-3

Slide 16 · UC4 join: the design sequence, paid ...
D16-1 D16-2 D16-3

Slide 17 · Team-formation statechart
D17-1 D17-2

Slide 18 · UC3 start a team: Factory Method
D18-1 D18-2 D18-3

Slide 19 · UC5 to UC6 team outcome: Observer
D19-1 D19-2

Slide 20 · Notifications: Template Method for o...
D20-1 D20-2

Slide 21 · UC1 pricing: Strategy
D21-1 D21-2

Slide 22 · Design problem: listing rules that de...
D22-1 D22-2

Slide 23 · After: the State pattern, and why the...
D23-1 D23-2

Slide 24 · Database schema: core entities and ...
D24-1 D24-2

Slide 25 · Testing on the Agile Testing Quadrants
D25-1 D25-2 D25-3 D25-4

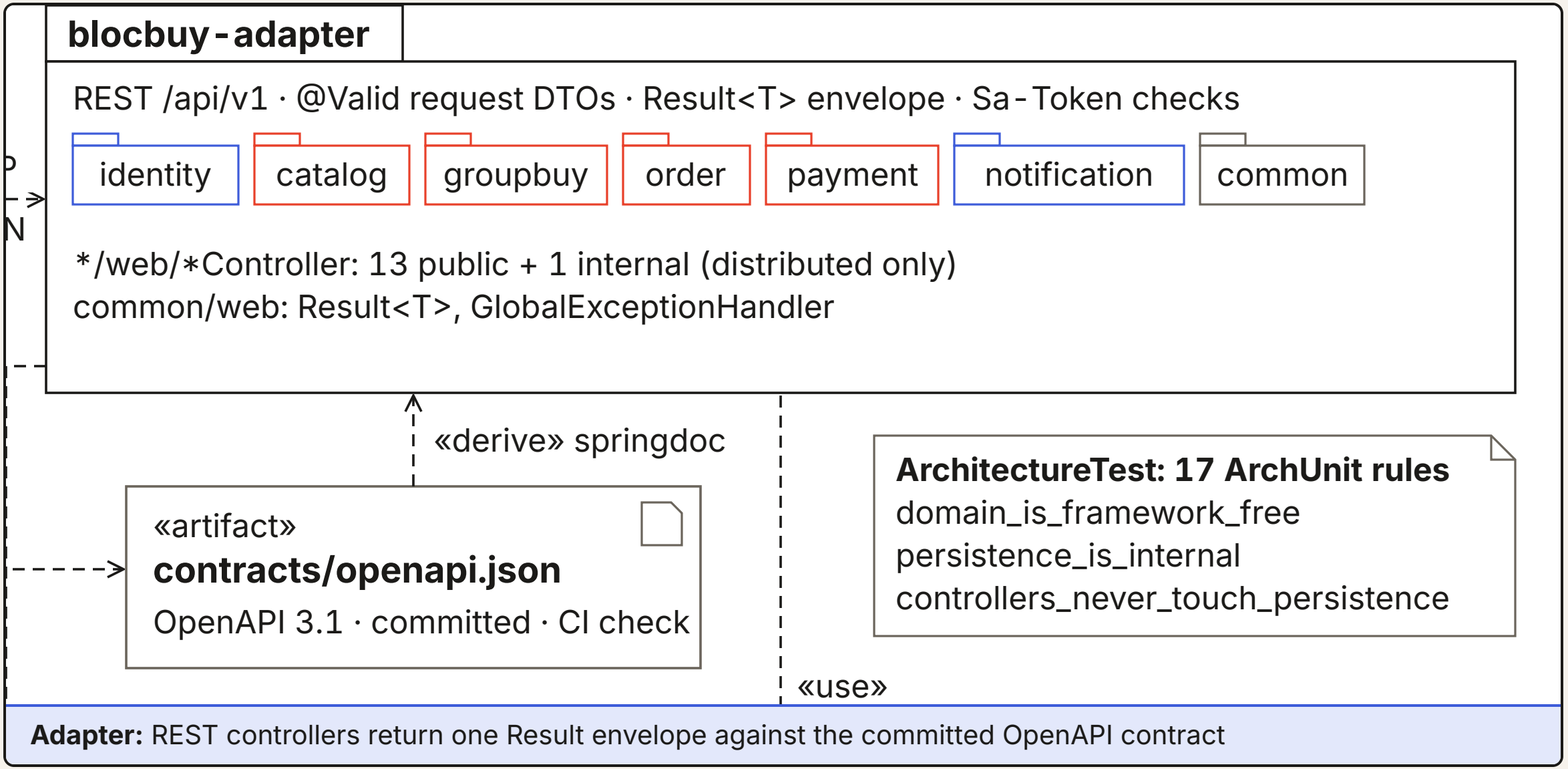
Slide 29 · Continuous integration: pull-request...
D29-1 D29-2 D29-3 D29-4

Slide 31 · Delivery: one environment, the code ...
D31-1 D31-2 D31-3

*Each page is one crop at 1:1 with its caption: the same view the slide steps to while presenting.*

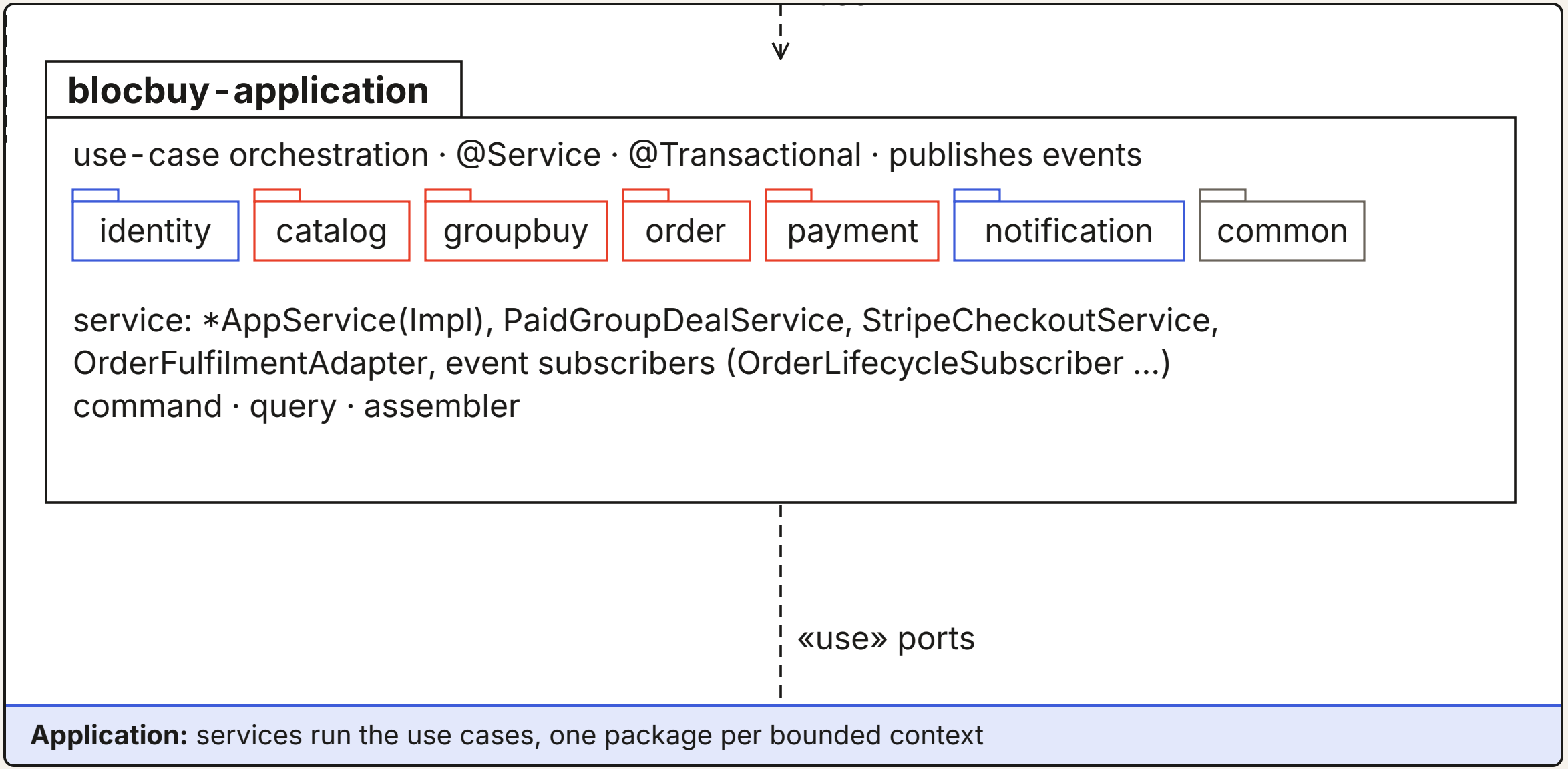
WZ Eric

# Slide 9 · detail 1/3: adapter · contract



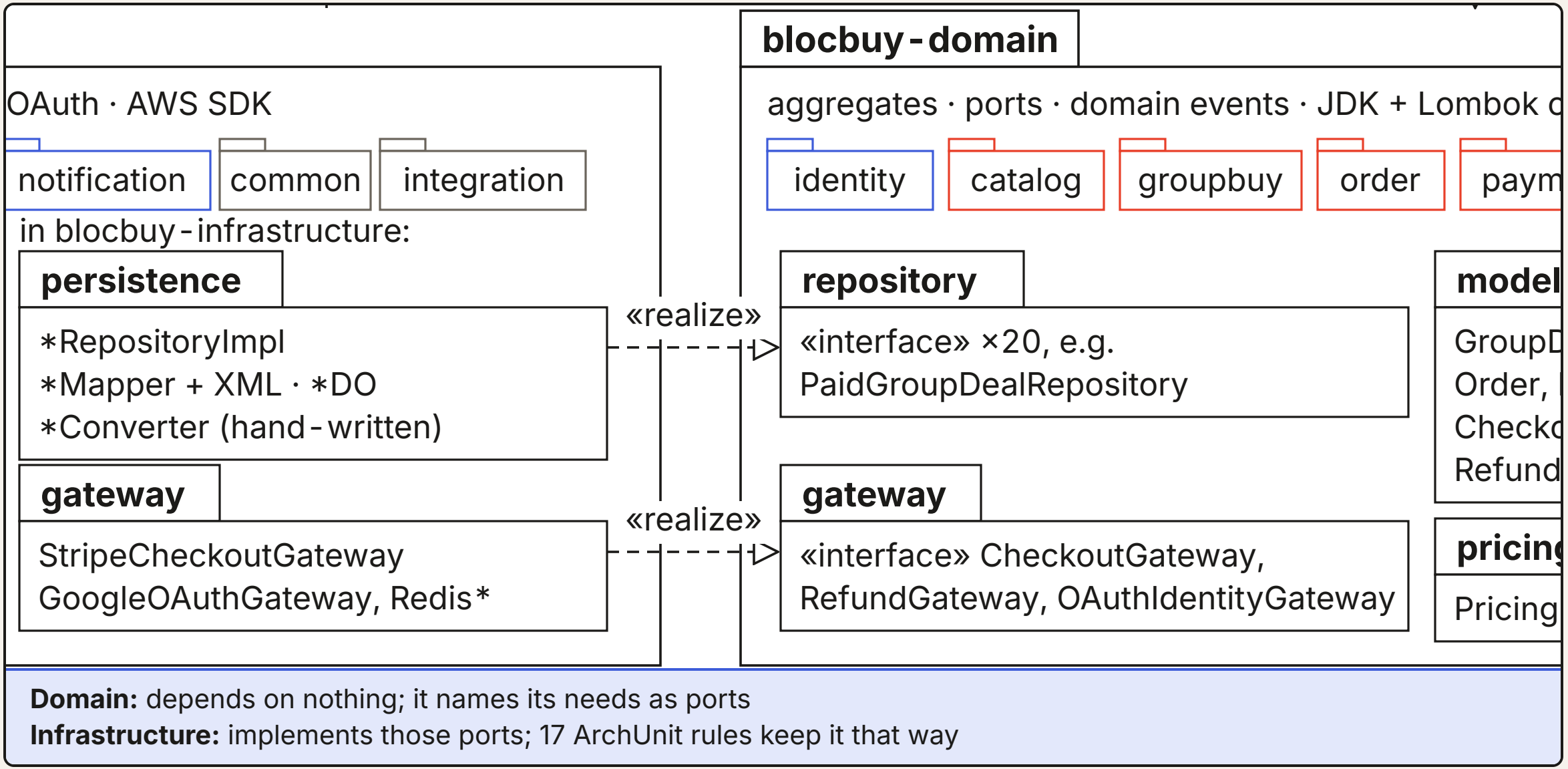
WZ Eric

# Slide 9 · detail 2/3: application



WZ Eric

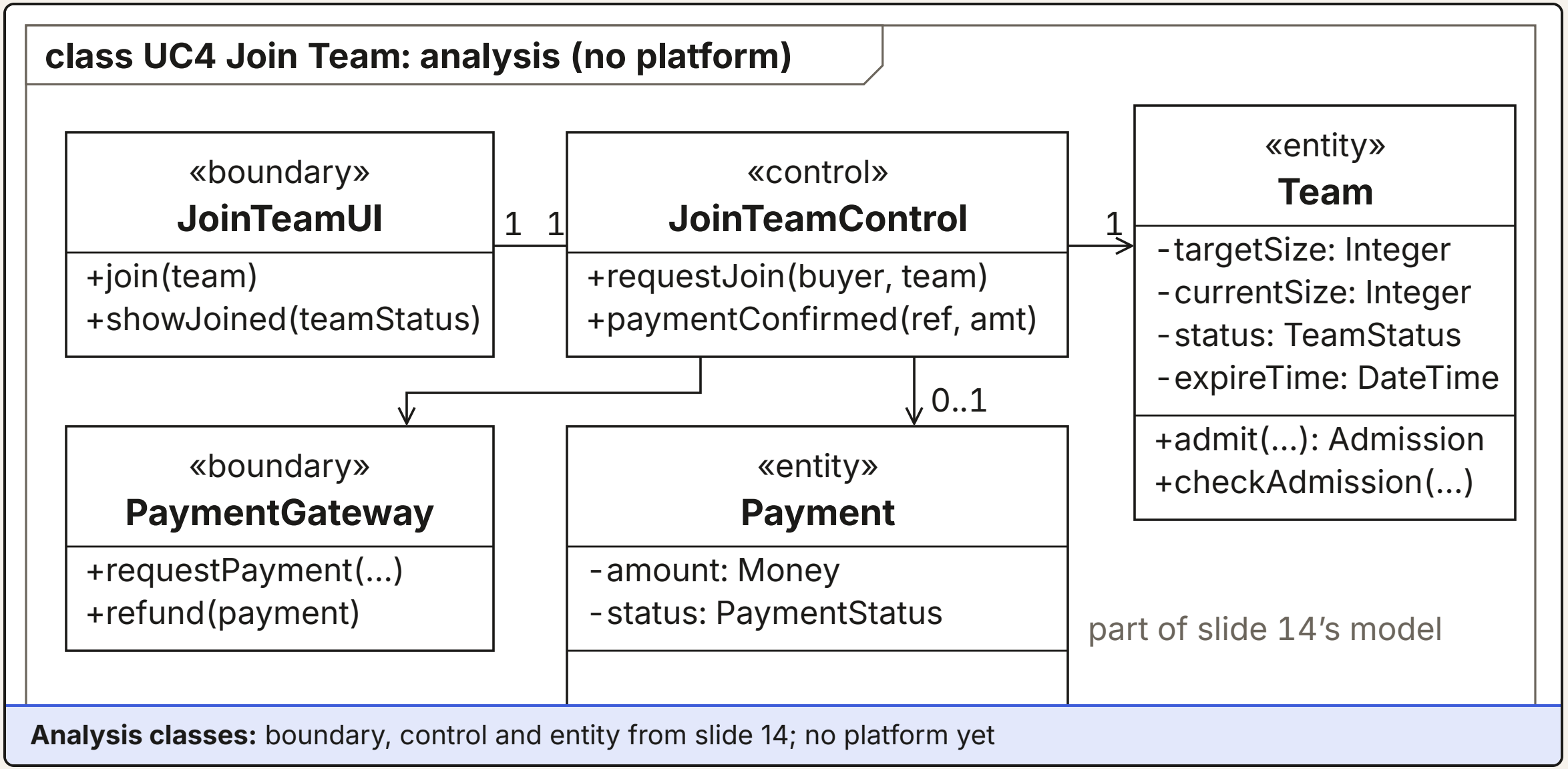
# Slide 9 · detail 3/3: infrastructure realises the domain ports




**Domain:** depends on nothing; it names its needs as ports
**Infrastructure:** implements those ports; 17 ArchUnit rules keep it that way

WZ Eric

# Slide 12 · detail 1/3: analysis: UC4




**Analysis classes:** boundary, control and entity from slide 14; no platform yet

WZ Eric

# Slide 12 · detail 2/3: strategy key

## Transition strategies applied (tag = strategy)

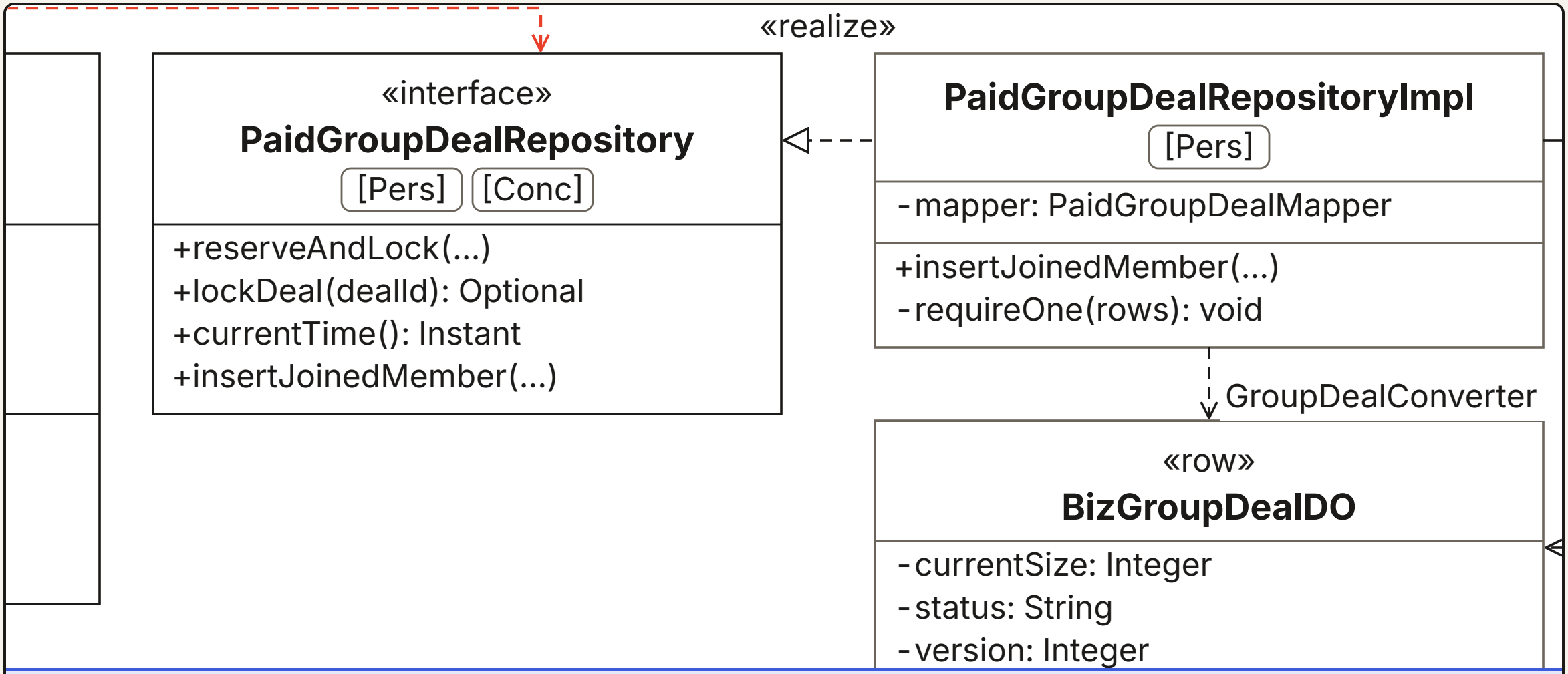
| | | |
|---|---|---|
| [Pres] | Presentation | JoinTeamUI → route + Query hook |
| [Dist] | Distribution | control interface → stub + @RestController |
| [Pers] | Persistence | Team → GroupDeal + port ← MyBatis impl |
| [Ext] | External interface | PaymentGateway → CheckoutGateway ← adapter |
| [Conc] | Concurrency | requestJoin → lock order + guarded UPDATE |
| [Evt] | Domain events | team full → TeamSucceeded published |
| [Sec] | Security | CSRF header, rate limit, session check |
| [Lang] | Language & framework | Spring beans, immutable Java aggregate |

Payment entity → CheckoutPayment + CheckoutPaymentRepository
(payment context, same [Pers] shape; not drawn)

**Strategy key:** each tag names the transition strategy that produced a design class

WZ Eric

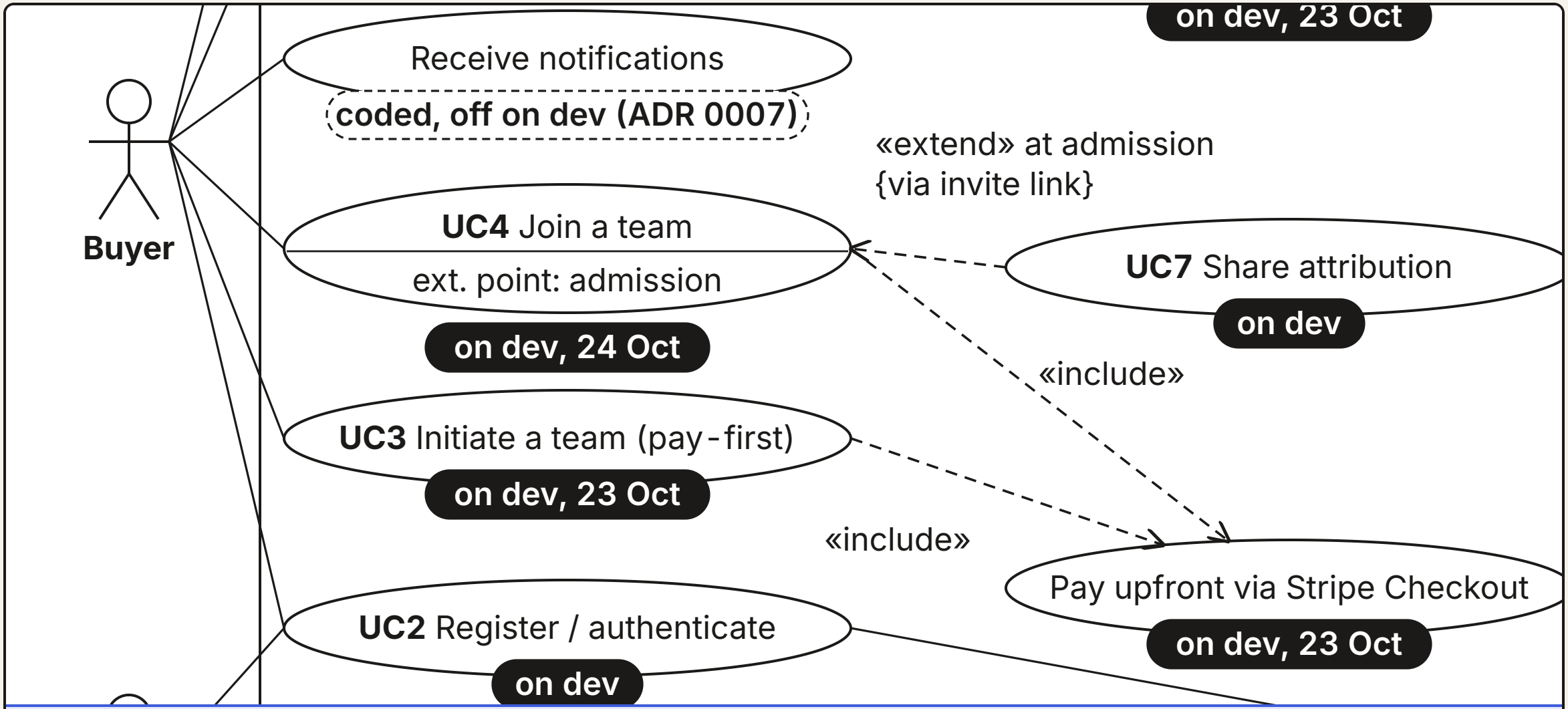
# Slide 12 · detail 3/3: design classes · persistence




**Design classes:** the Team becomes the GroupDeal aggregate, reached through its repository port
◁ **means "realises":** MyBatis classes in infrastructure implement that port

WZ Eric

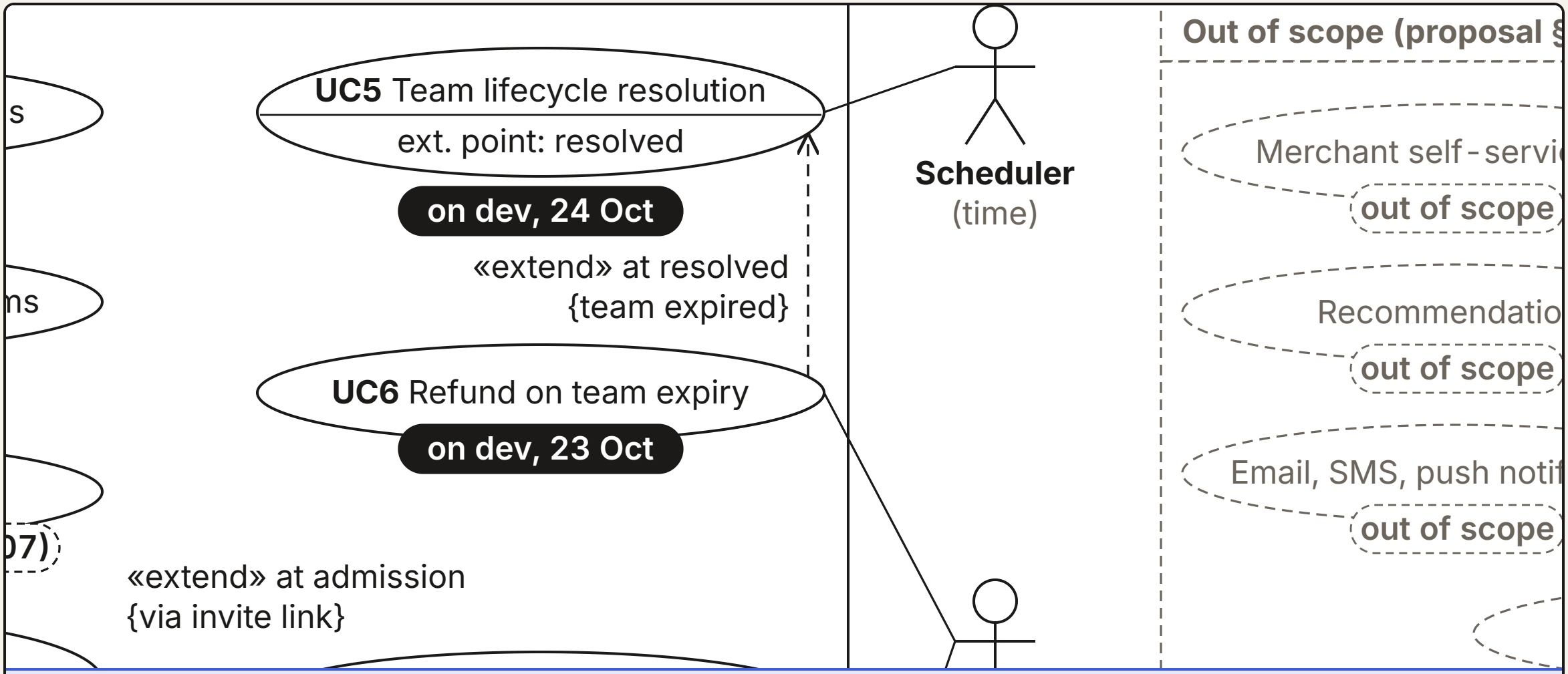
# Slide 13 · detail 1/5: Buyer and the money path




**In scope:** start and join «include» paying upfront; invite attribution «extend»s join

WZ Eric

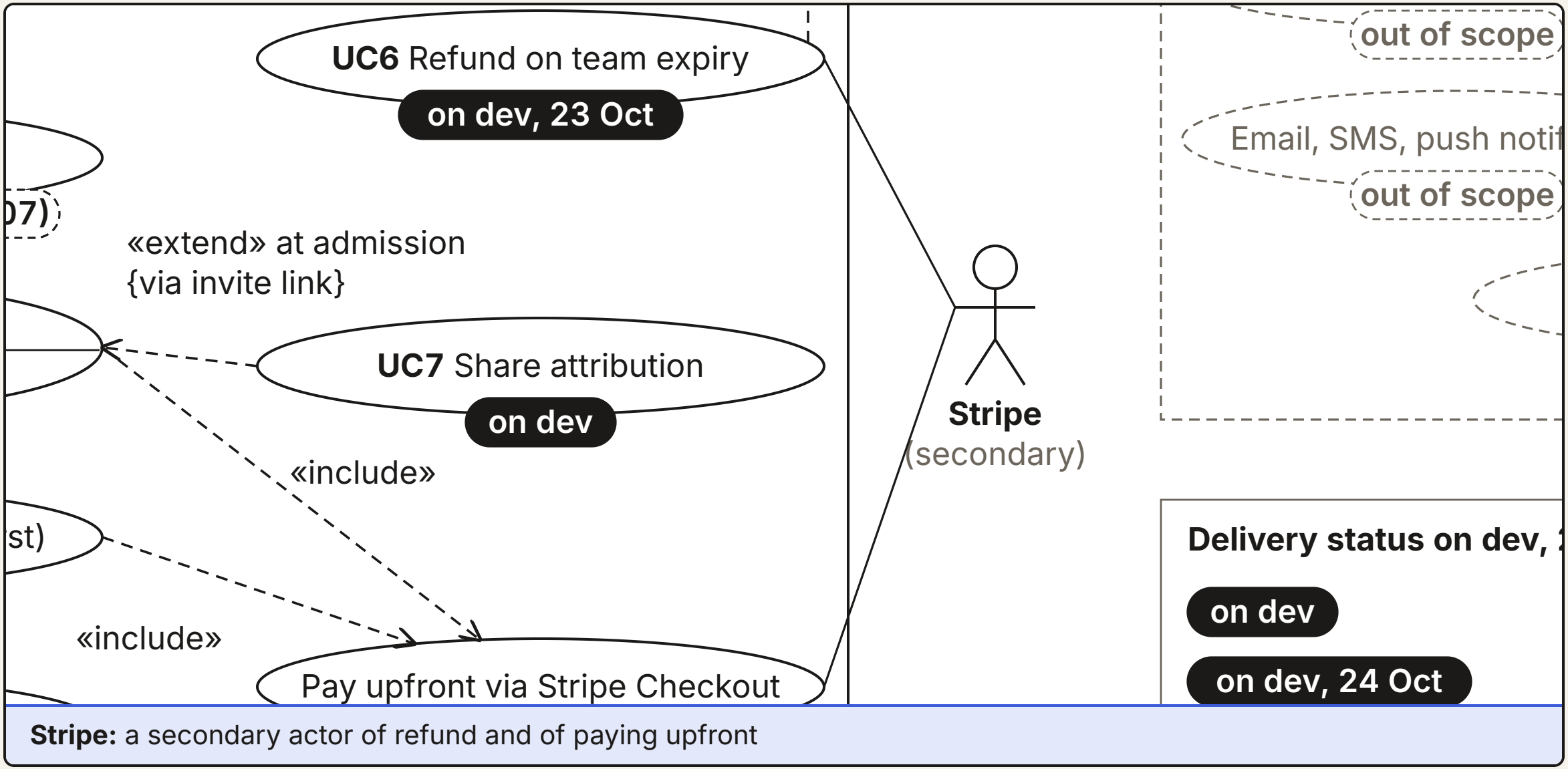
# Slide 13 · detail 2/5: resolution: Scheduler




**Scheduler:** a time-driven actor resolves teams
**«extend»:** refund extends resolution when a team expires

WZ Eric

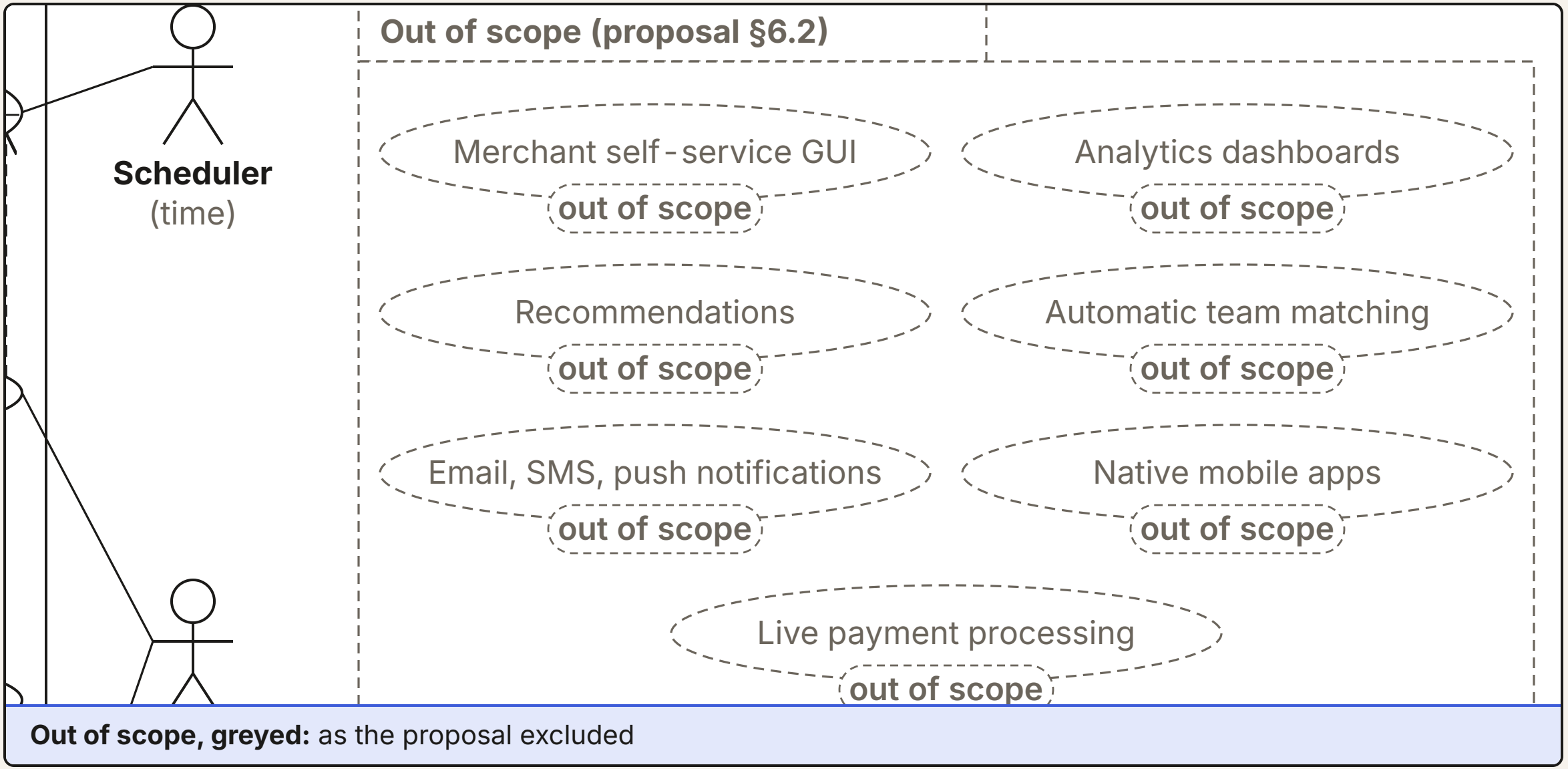
# Slide 13 · detail 3/5: refund, payment: Stripe




**Stripe:** a secondary actor of refund and of paying upfront

WZ Eric

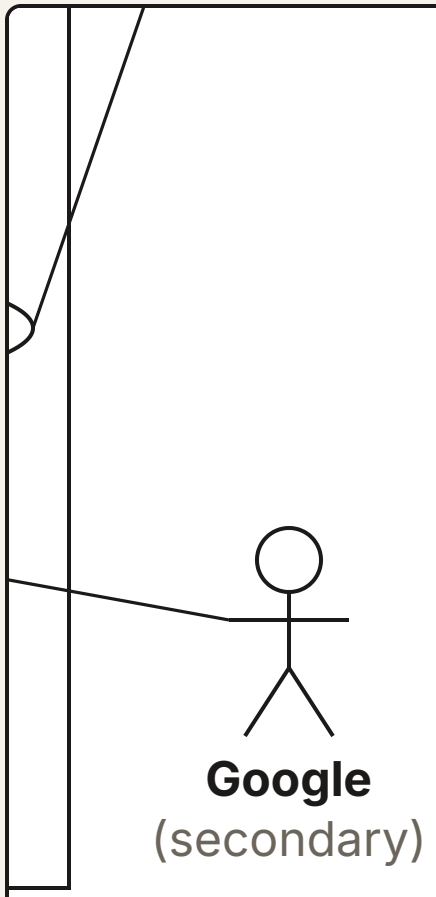
# Slide 13 · detail 4/5: out of scope




**Out of scope, greyed:** as the proposal excluded

WZ Eric

# Slide 13 · detail 5/5: delivery status




## Delivery status on dev, 24 Oct (team ruling)

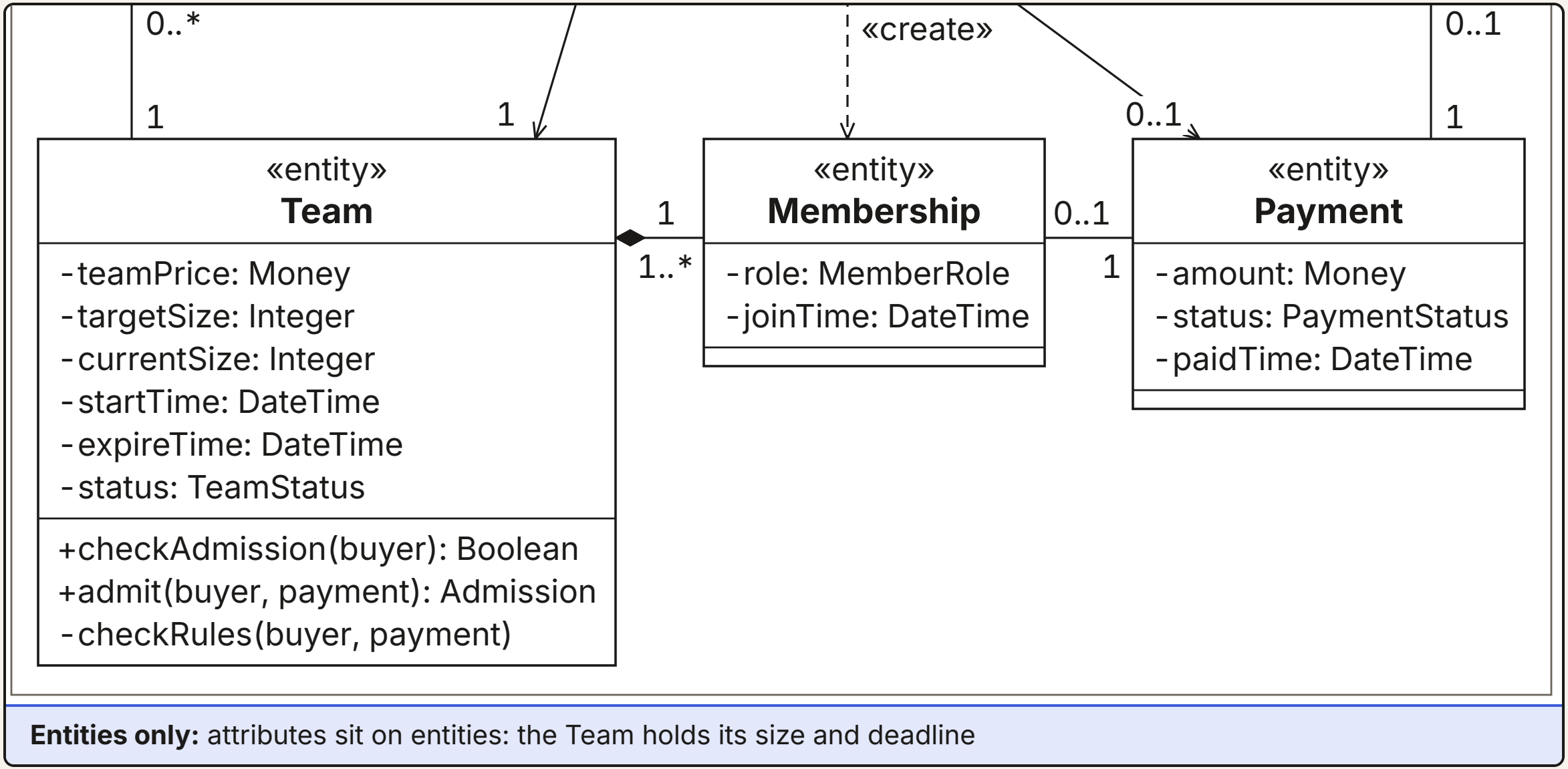
| | |
|---|---|
| **on dev** | deployed and usable on dev |
| **on dev, 24 Oct** | paid step first shown on that day |
| **coded, off on dev (ADR 0007)** | SQS consumer runs on SWE5001 only |
| **out of scope** | excluded by the proposal |

----------> «include»: base → included
«extend»: extending → base

**Status badges:** paid start and expiry refund on dev since 23 Oct; join and team resolution since 24 Oct

WZ Eric

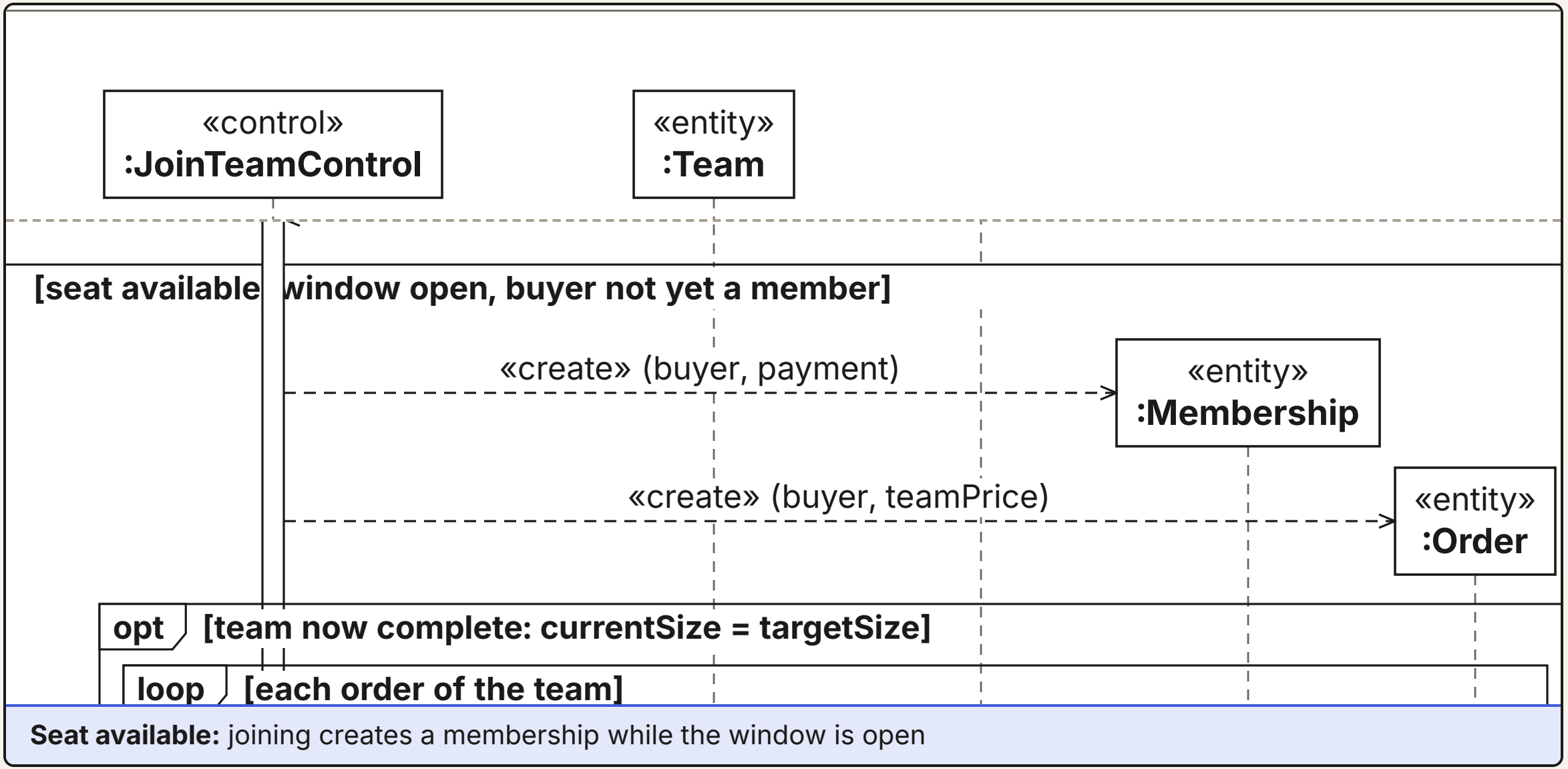
# Slide 14 · detail 1/3: entities




**Entities only:** attributes sit on entities: the Team holds its size and deadline

WZ Eric

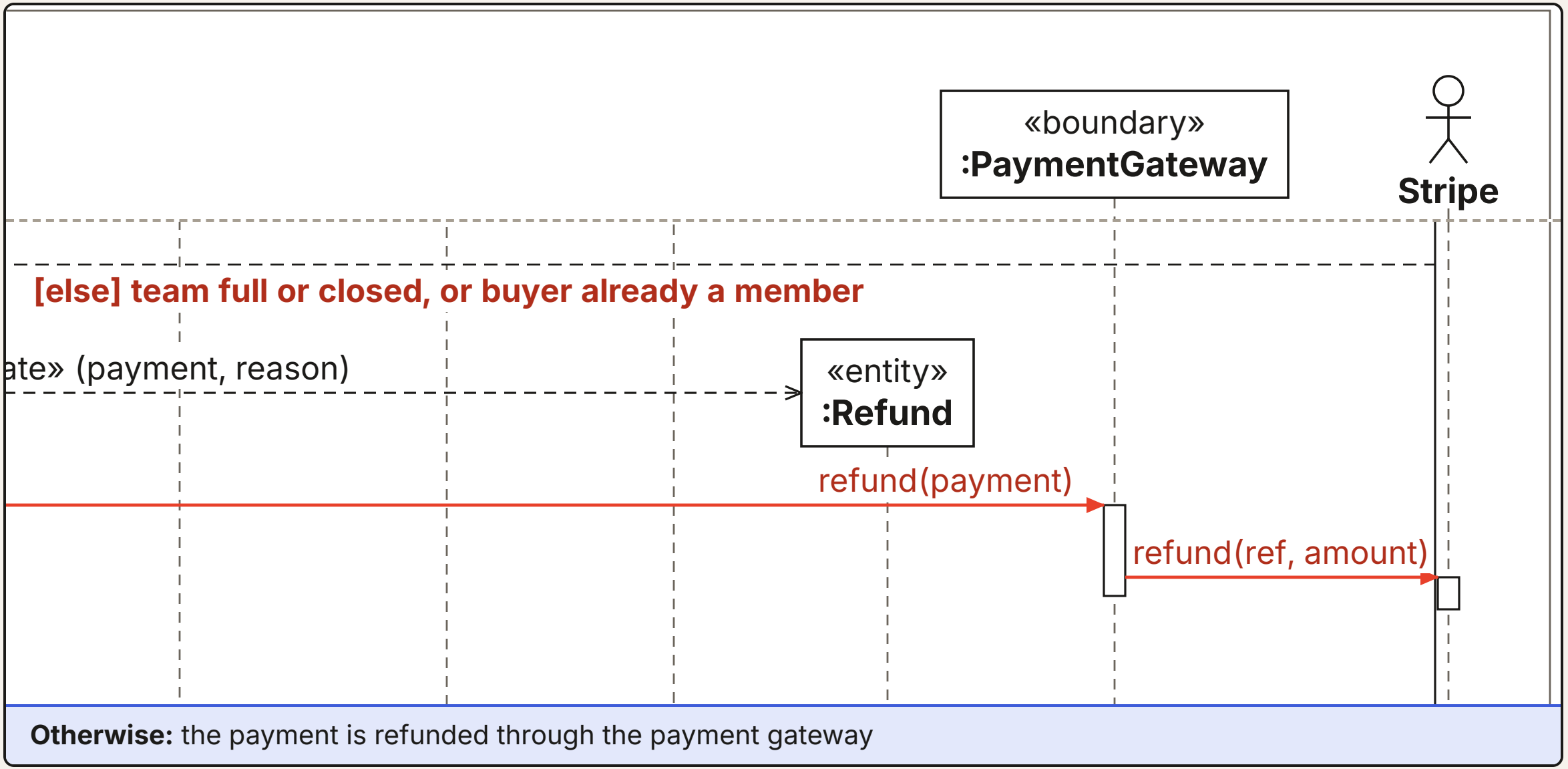
# Slide 14 · detail 2/3: seat available: join




**Seat available:** joining creates a membership while the window is open

WZ Eric

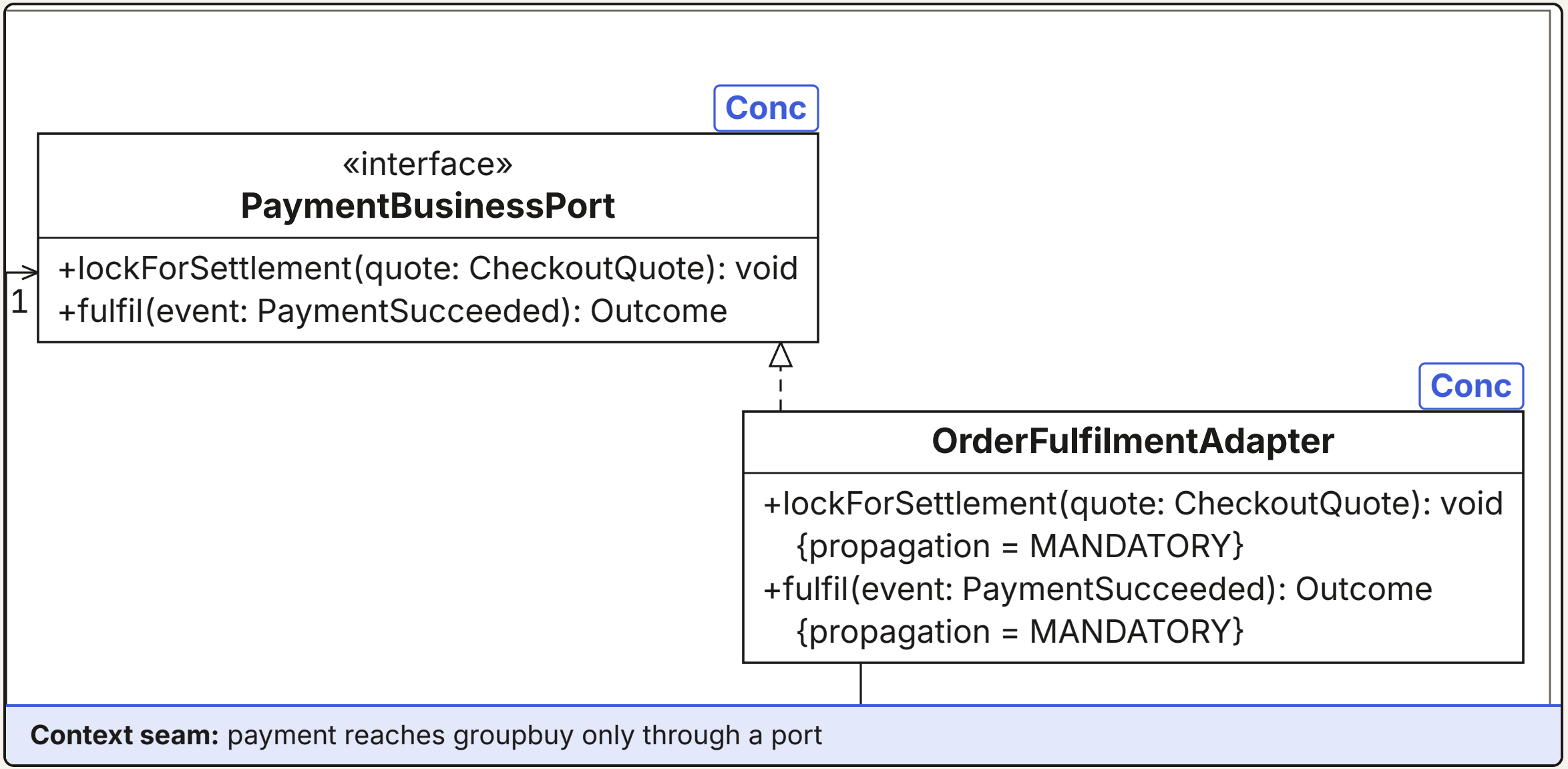
# Slide 14 · detail 3/3: else: refund through the gateway




**Otherwise:** the payment is refunded through the payment gateway

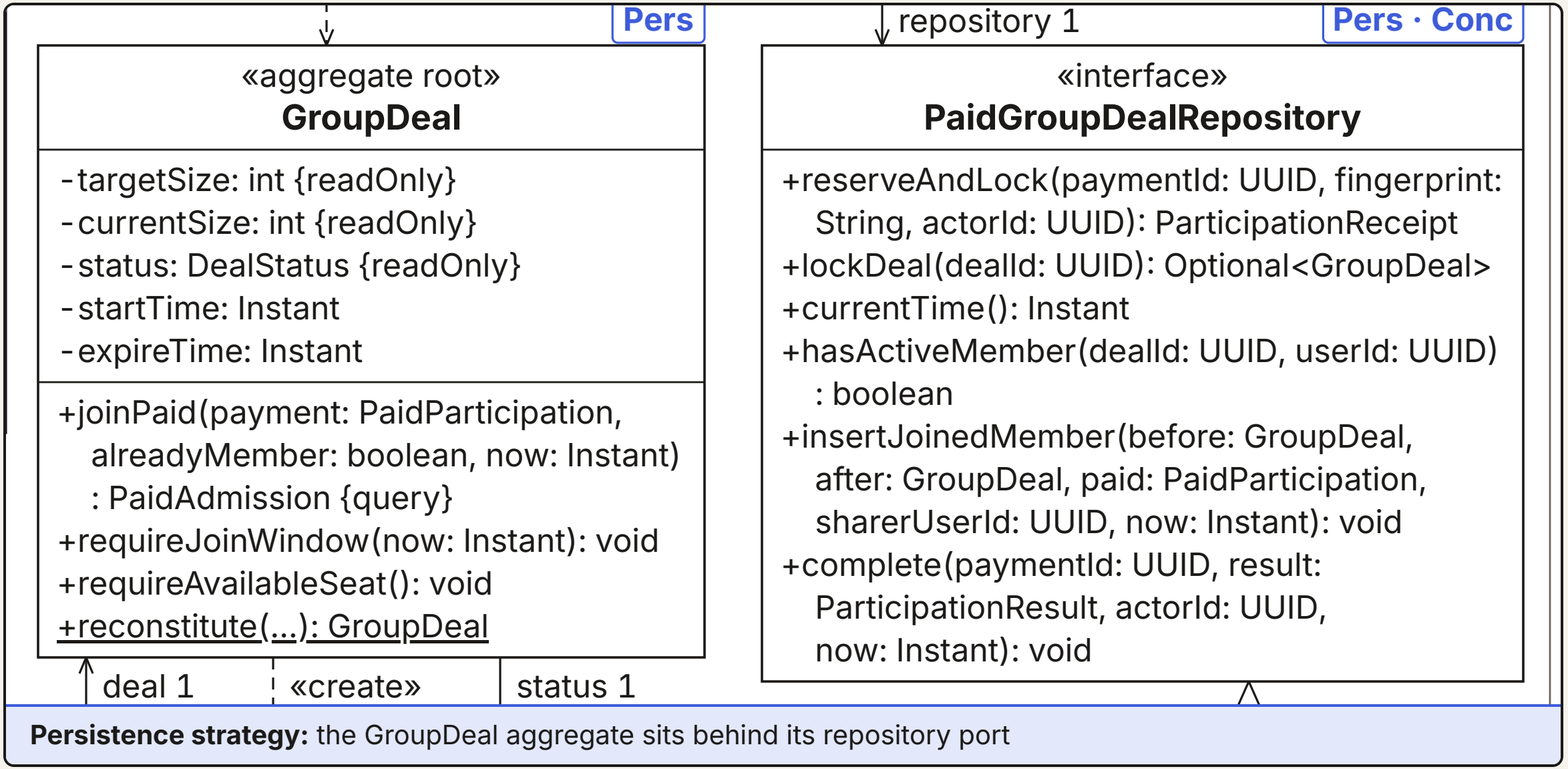
WZ Eric

# Slide 15 · detail 1/3: payment meets groupbuy through a port



WZ Eric

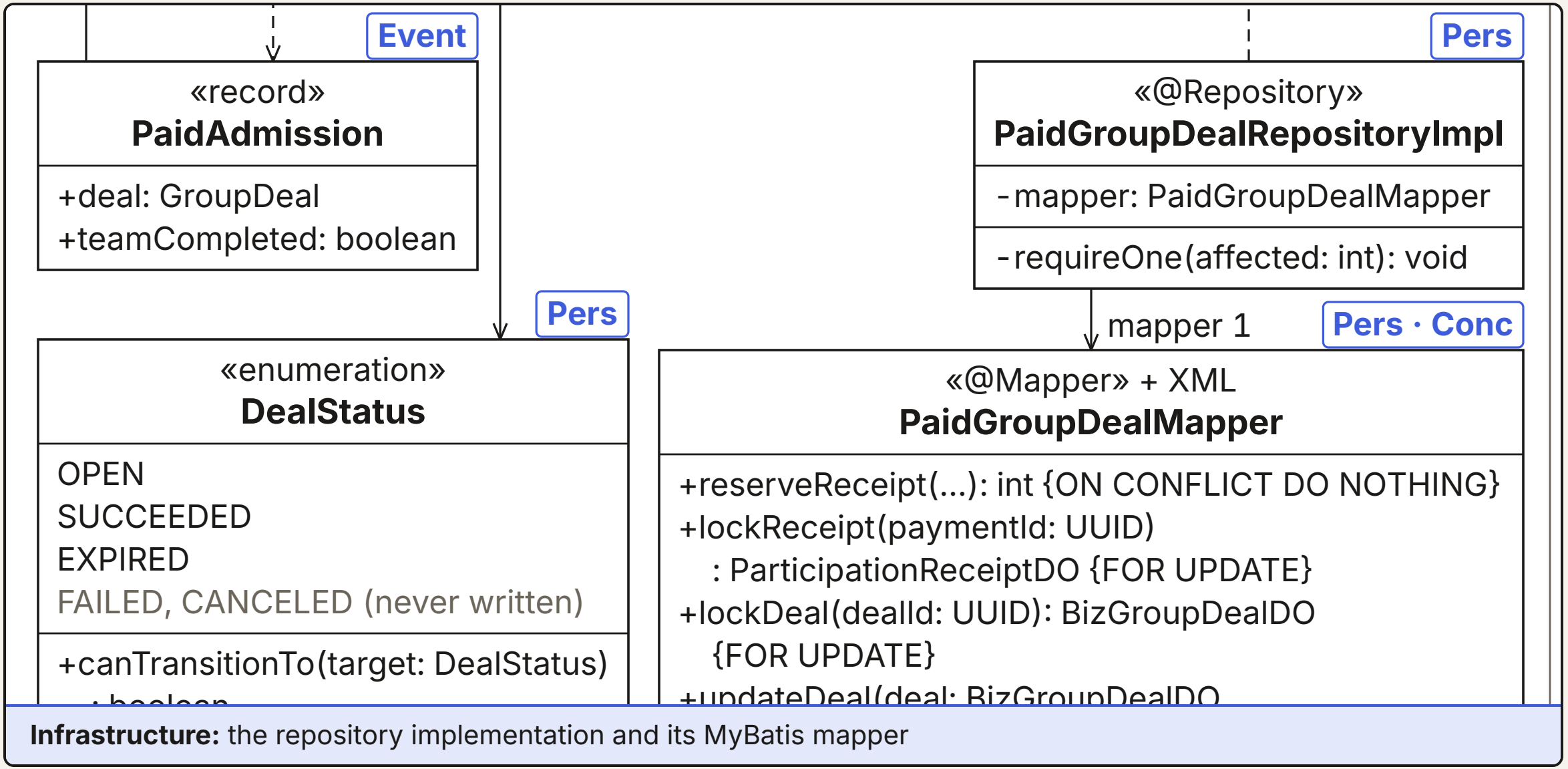
# Slide 15 · detail 2/3: aggregate behind its repository port




**Persistence strategy:** the GroupDeal aggregate sits behind its repository port

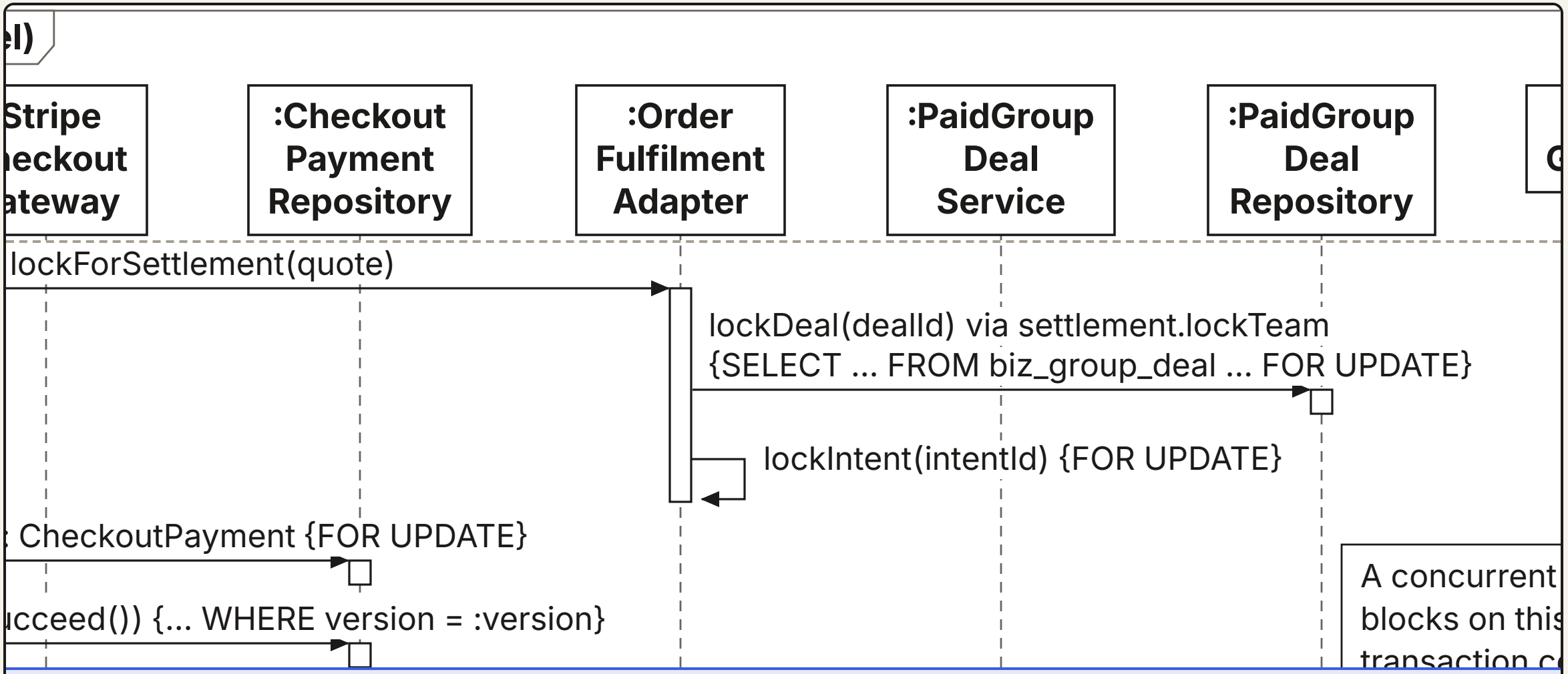
WZ Eric

# Slide 15 · detail 3/3: MyBatis implementation of the port



WZ Eric

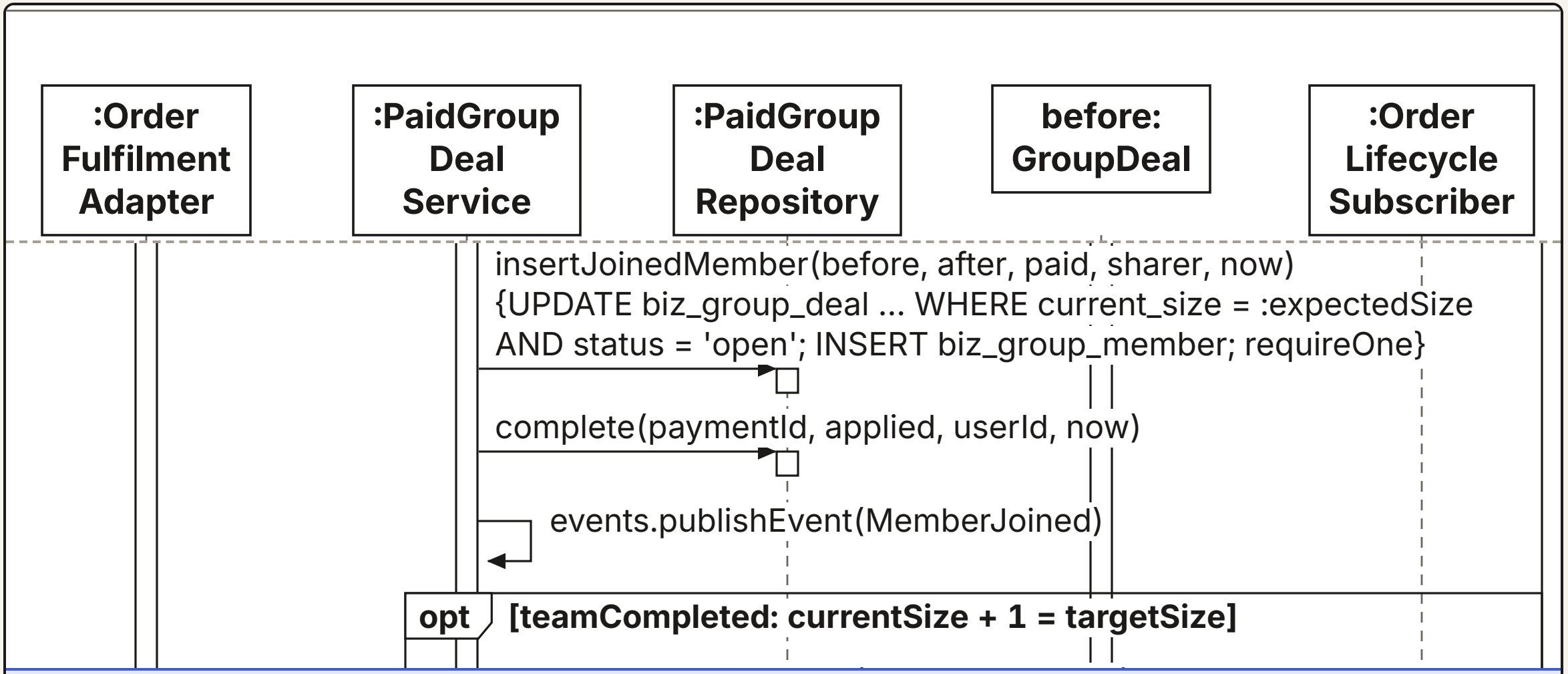
# Slide 16 · detail 1/3: team row locked first




**One transaction:** after the webhook's signature check; drawn as critical, as UML has no transaction fragment
**Lock order:** team row, intent, then payment; a racing join waits on the team row

WZ Eric

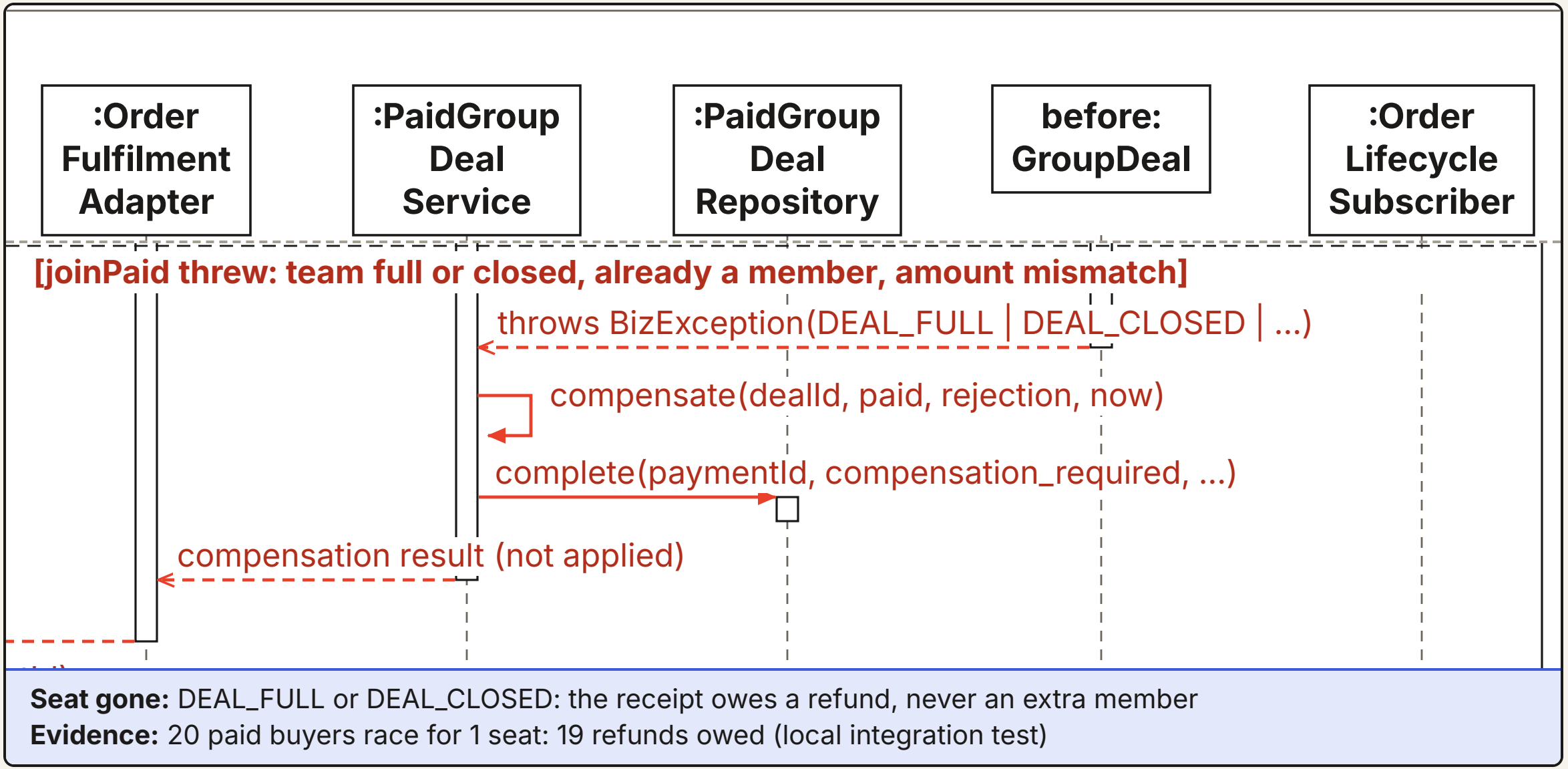
# Slide 16 · detail 2/3: guarded update · last seat




**Guarded update:** `UPDATE … WHERE current_size = expected AND status = open`
**Last seat:** the opt [teamCompleted] branch publishes `TeamSucceeded`

WZ Eric

# Slide 16 · detail 3/3: loser: compensation, refund owed




**Seat gone:** DEAL_FULL or DEAL_CLOSED: the receipt owes a refund, never an extra member
**Evidence:** 20 paid buyers race for 1 seat: 19 refunds owed (local integration test)

WZ Eric

# Slide 17 · detail 1/2: OPEN and its joins

/ currentSize := 1; leader := payer;
startTime := now

created by a confirmed paid START (UC3); no trigger or guard here

UC3

**OPEN**

paidJoinConfirmed(paid)
[admissible ∧ currentSize + 1 < targetSize]
/ currentSize := currentSize + 1;
publish MemberJoined

UC4

paidJoinConfirmed(paid) [¬admissible]
/ compensation_required → refund owed (UC6)

admissible ≡ startTime ≤ now < expireTime
∧ buyer not leader or member ∧ amount = price
(now read after the FOR UPDATE row lock)
last join vs expiry: first lock wins, the other sees DEAL_CLOSED

FAILED

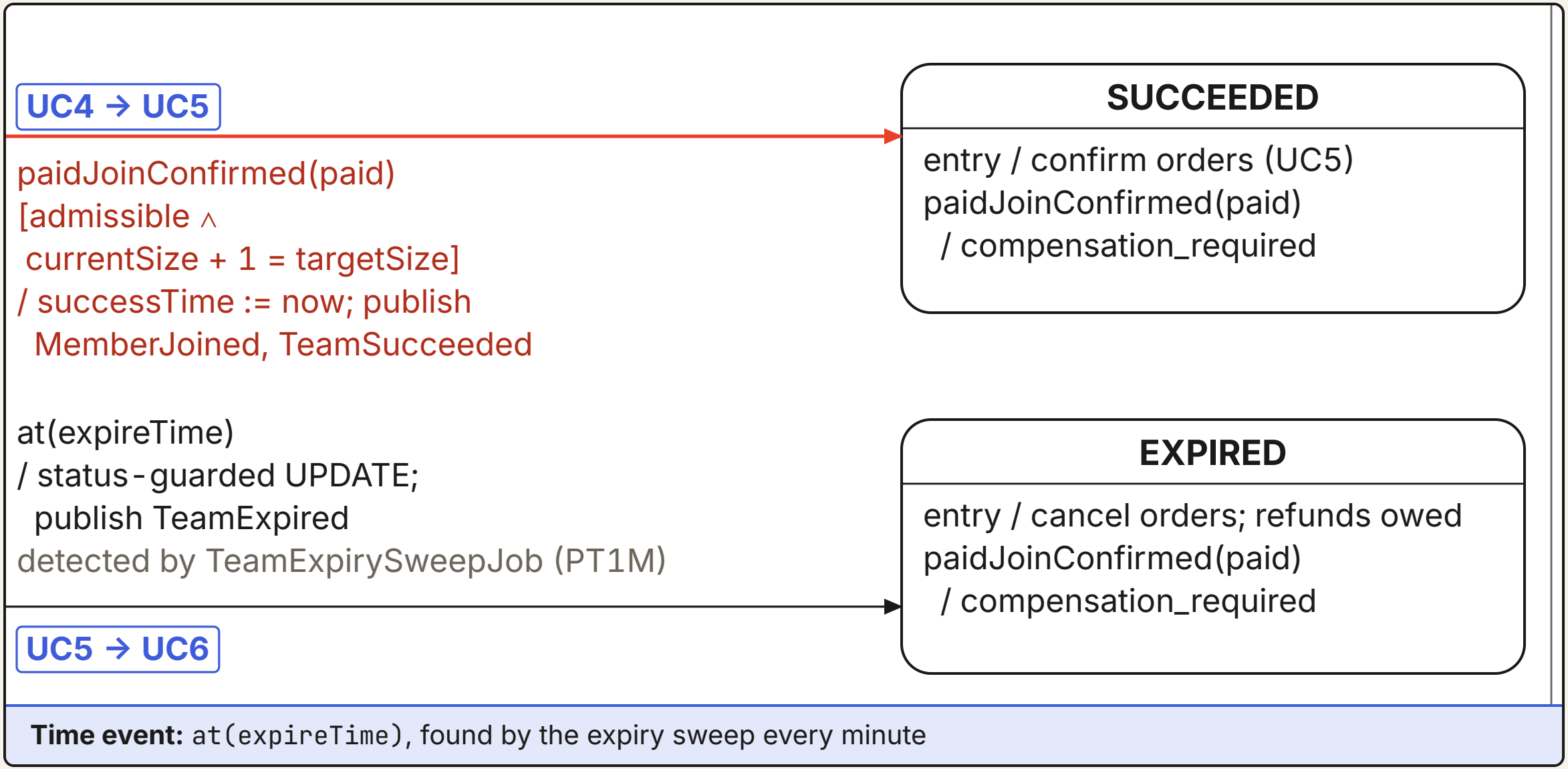
CANCELED

declared in DealStatus and allowed by canTransitionTo, but never written

**Guards:** price → new member → window → seat (`GroupDeal.joinPaid`)
**Code it matches:** `DealStatus.canTransitionTo`: only OPEN moves, never to itself
**Greyed:** FAILED and CANCELED are declared but never written

WZ Eric

# Slide 17 · detail 2/2: succeeded or expired

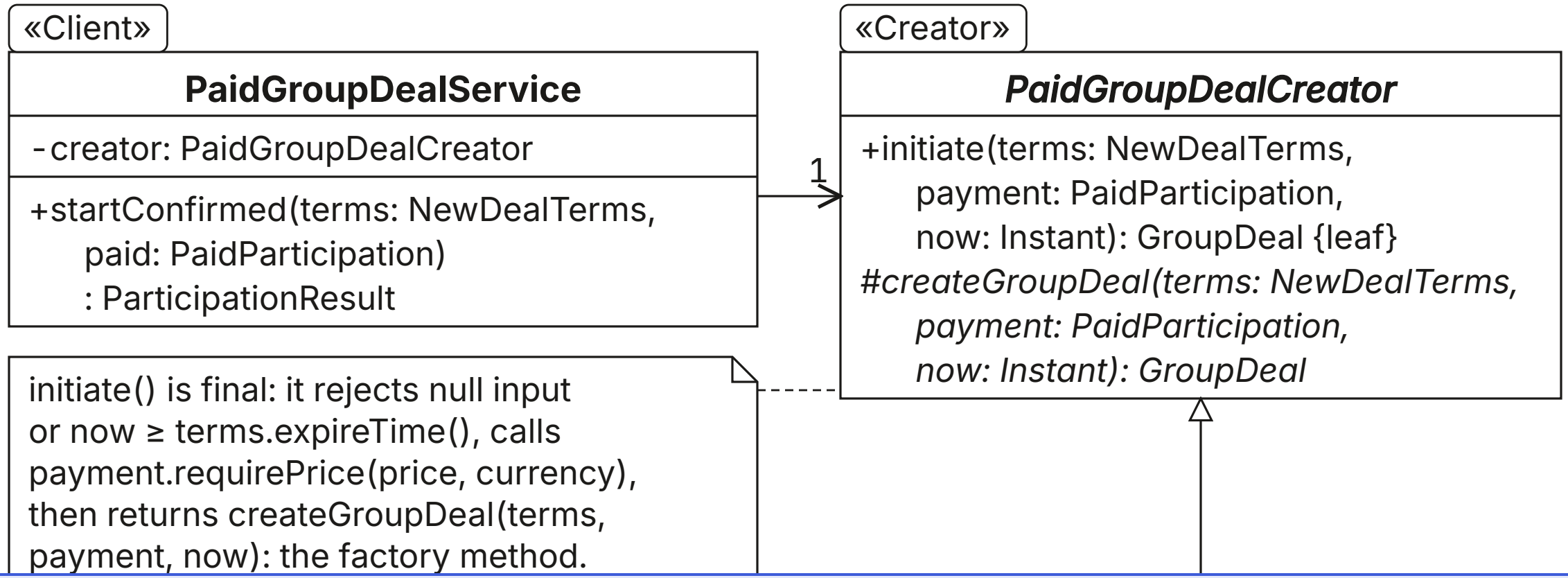



**Time event:** `at(expireTime)`, found by the expiry sweep every minute

WZ Eric

# Slide 18 · detail 1/3: Client · Creator

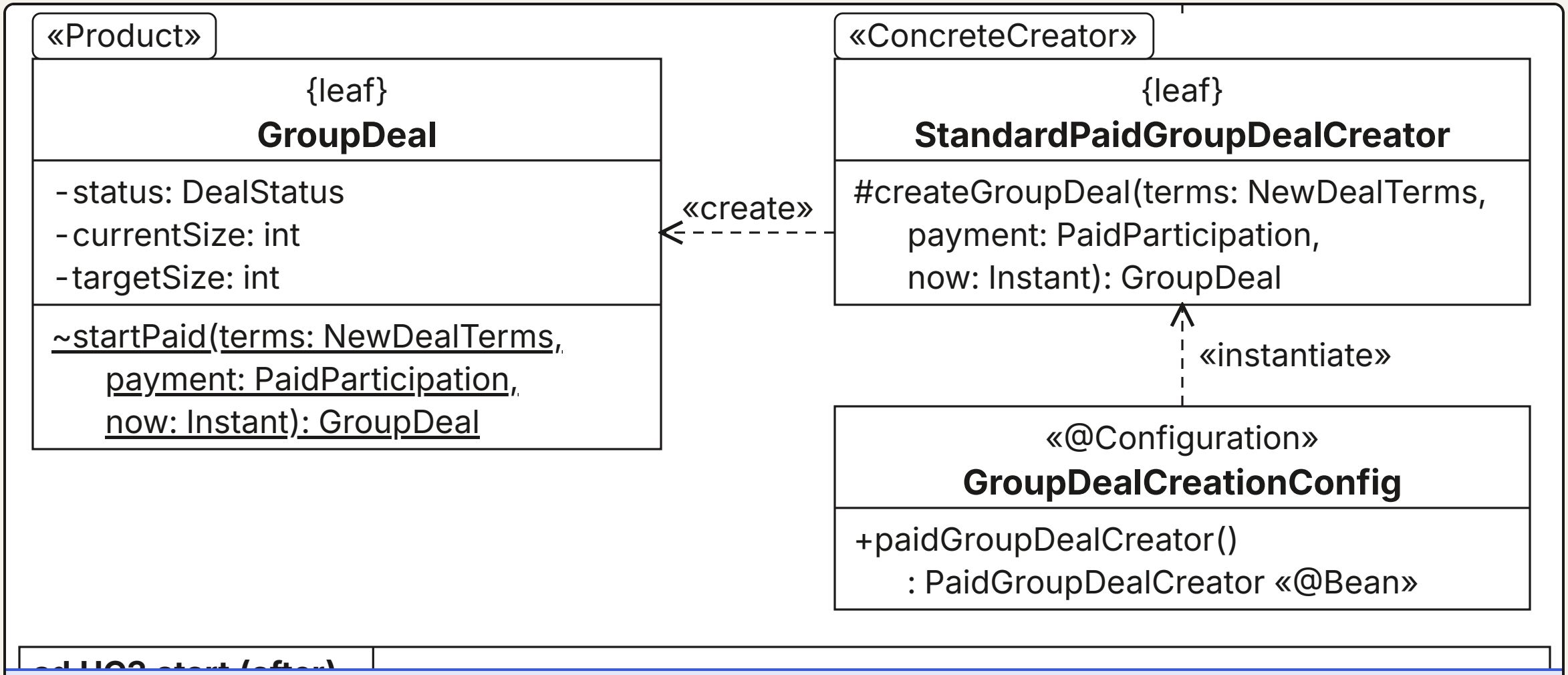
## Factory Method: UC3 start creates the team (GroupDeal)




**Client:** the service depends only on the abstract creator
**Needed because:** the proposal makes new kinds of team, without editing the service, a design driver

WZ Eric

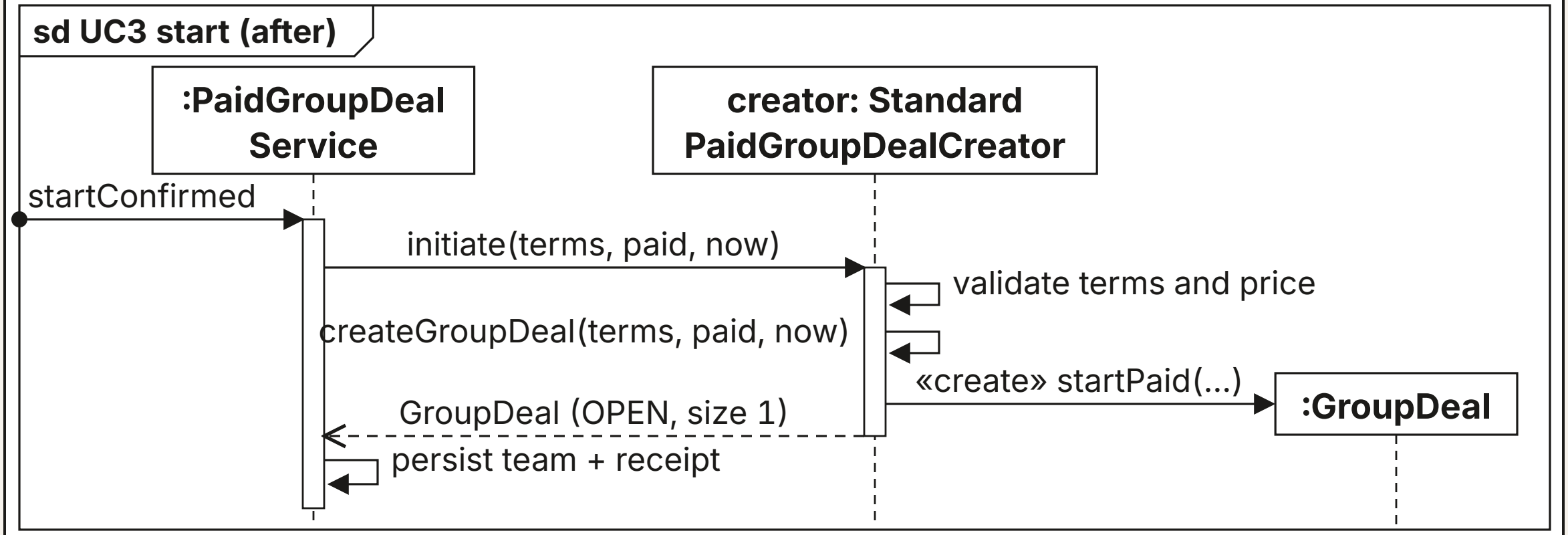
# Slide 18 · detail 2/3: ConcreteCreator · Product




**Participants:** Creator `PaidGroupDealCreator`, ConcreteCreator `StandardPaidGroupDealCreator`, Product `GroupDeal`
**Candidates:** Factory Method, static factory, Builder, Abstract Factory

WZ Eric

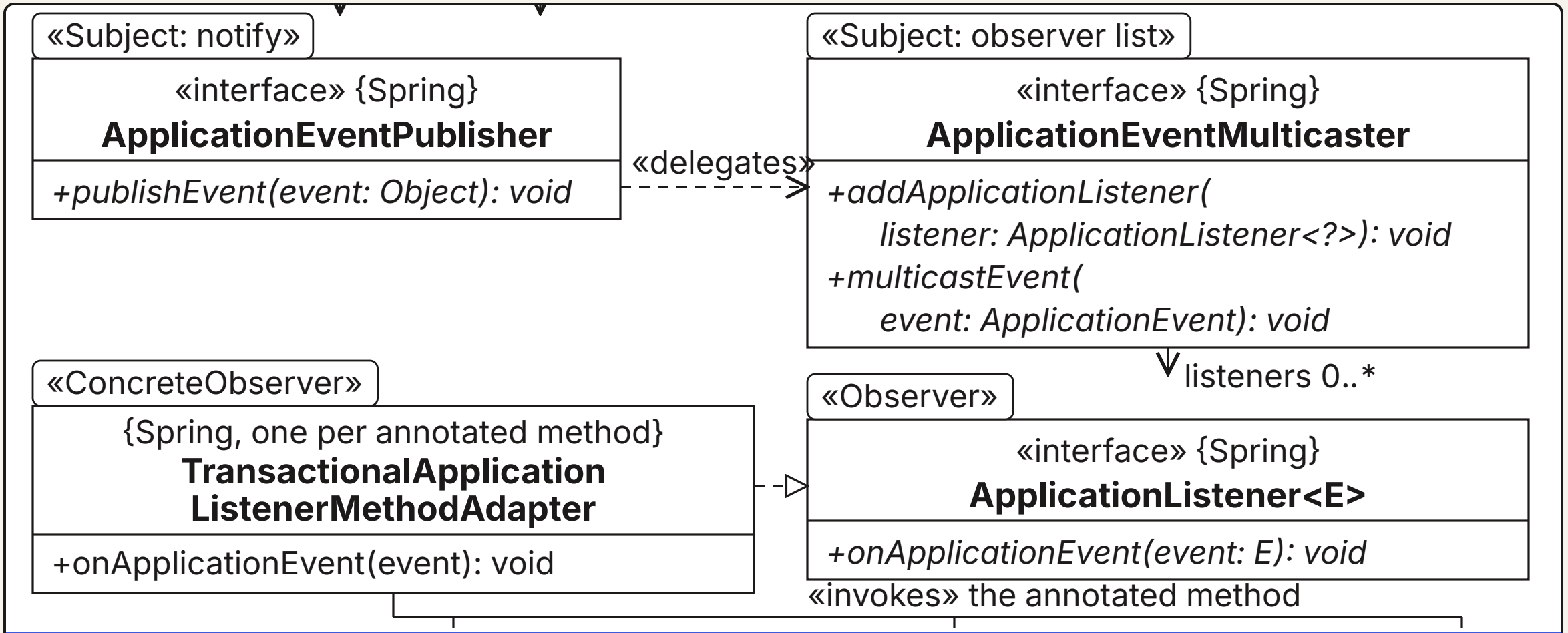
# Slide 18 · detail 3/3: initiate → createGroupDeal




**Why:** the final `initiate` validates, then calls `createGroupDeal`: every creator keeps the rules
**Liability:** one creator and one team class today
**Tested by:** `PaidGroupDealServiceTest.startUsesTheInjectedCreator`

YQ Yorik

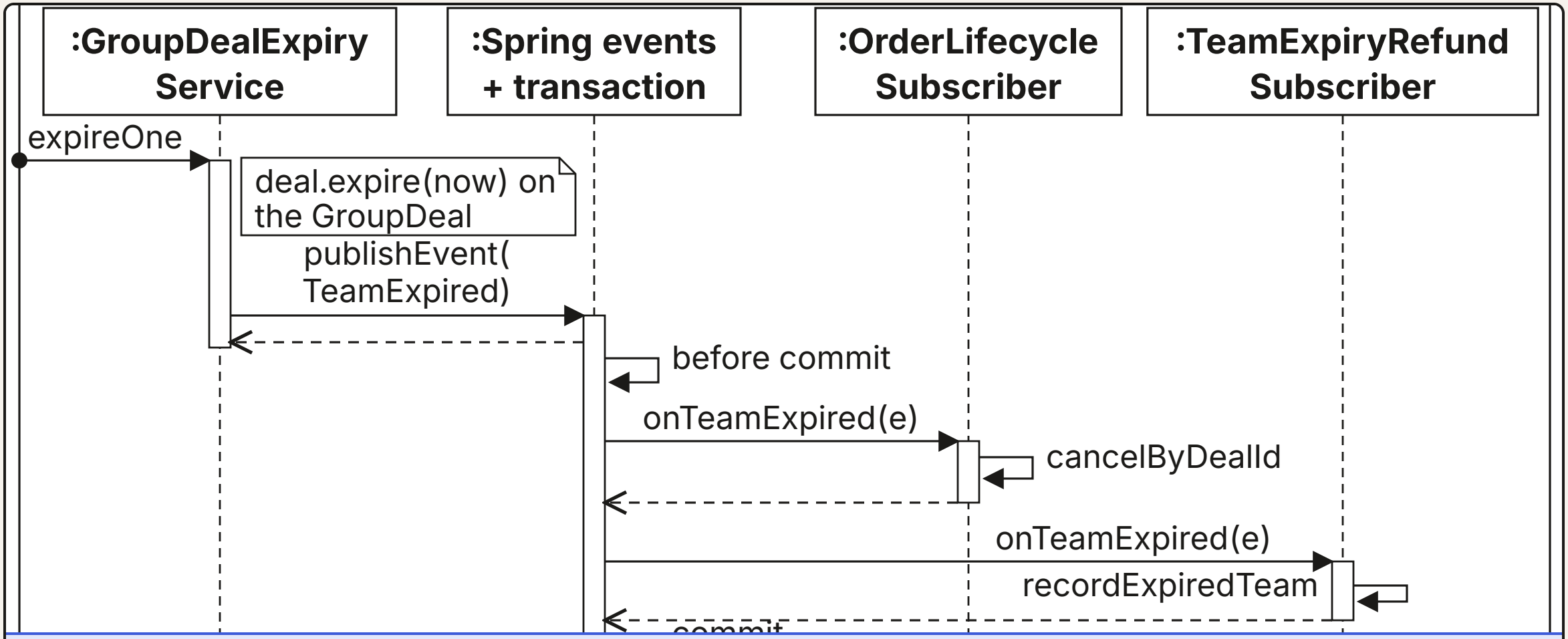
# Slide 19 · detail 1/2: Subject · Observer · ConcreteObserver adapter




**Subject:** Spring's `ApplicationEventPublisher` and its multicaster
**ConcreteObserver:** Spring's adapter, one per subscriber method
**Candidates:** Observer, Mediator, direct calls, a message broker

YQ Yorik

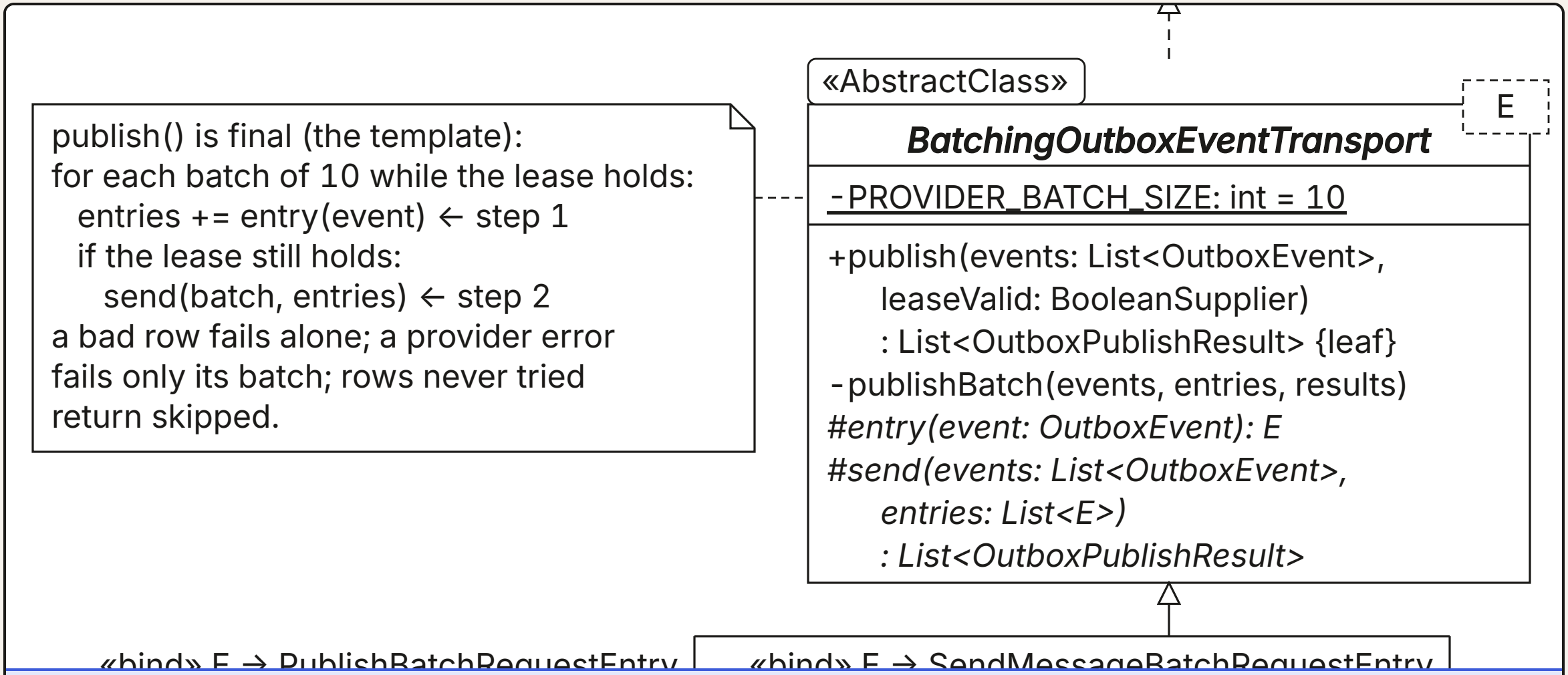
# Slide 19 · detail 2/2: publish → before commit, in order




**Before commit, in order:** `OrderLifecycleSubscriber` (10), then `TeamExpiryRefundSubscriber` (20)
**Liability:** implicit control flow; a wrong phase is a silent bug
**Tested by:** `TeamExpiryRefundSubscriberTest`: a failure rolls the expiry back

PS Pan

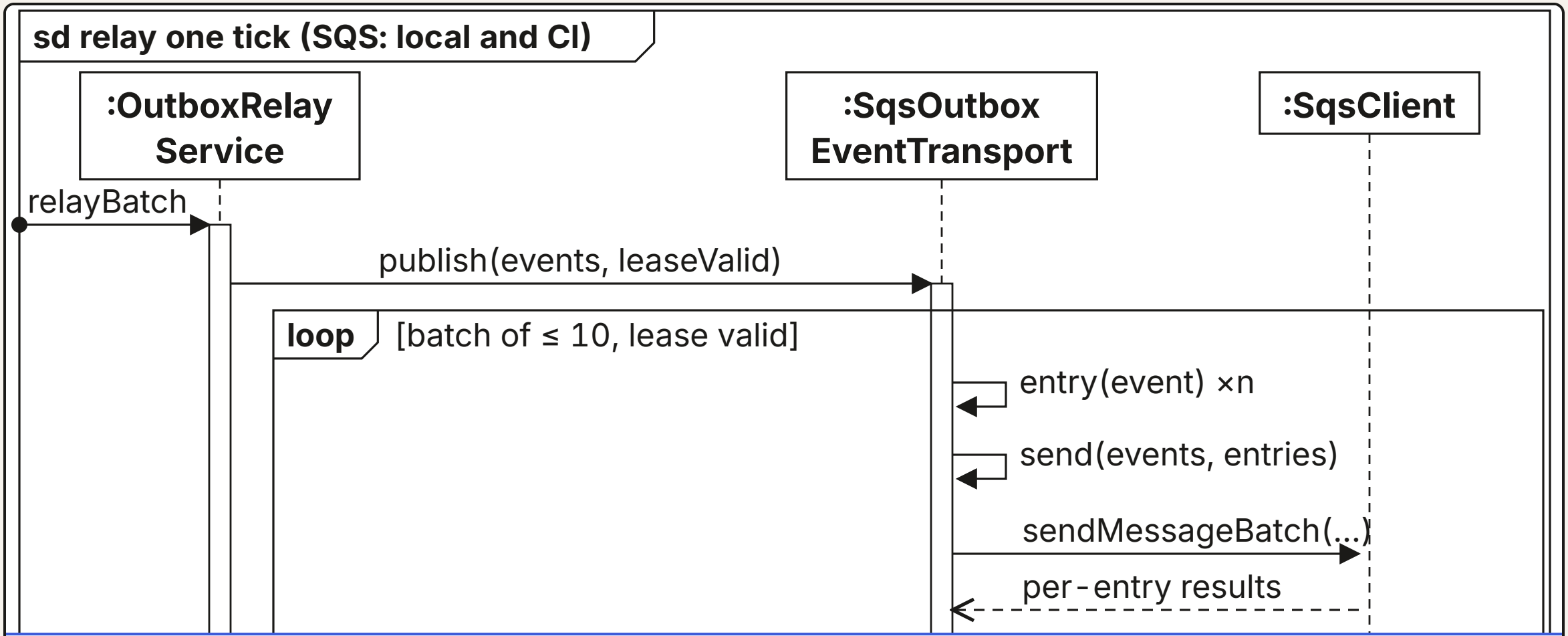
# Slide 20 · detail 1/2: template method




**AbstractClass:** `BatchingOutboxEventTransport<E>`: the final `publish` runs the loop
**Candidates:** Template Method; Strategy (the whole algorithm does not vary); an adapter per SDK

PS Pan

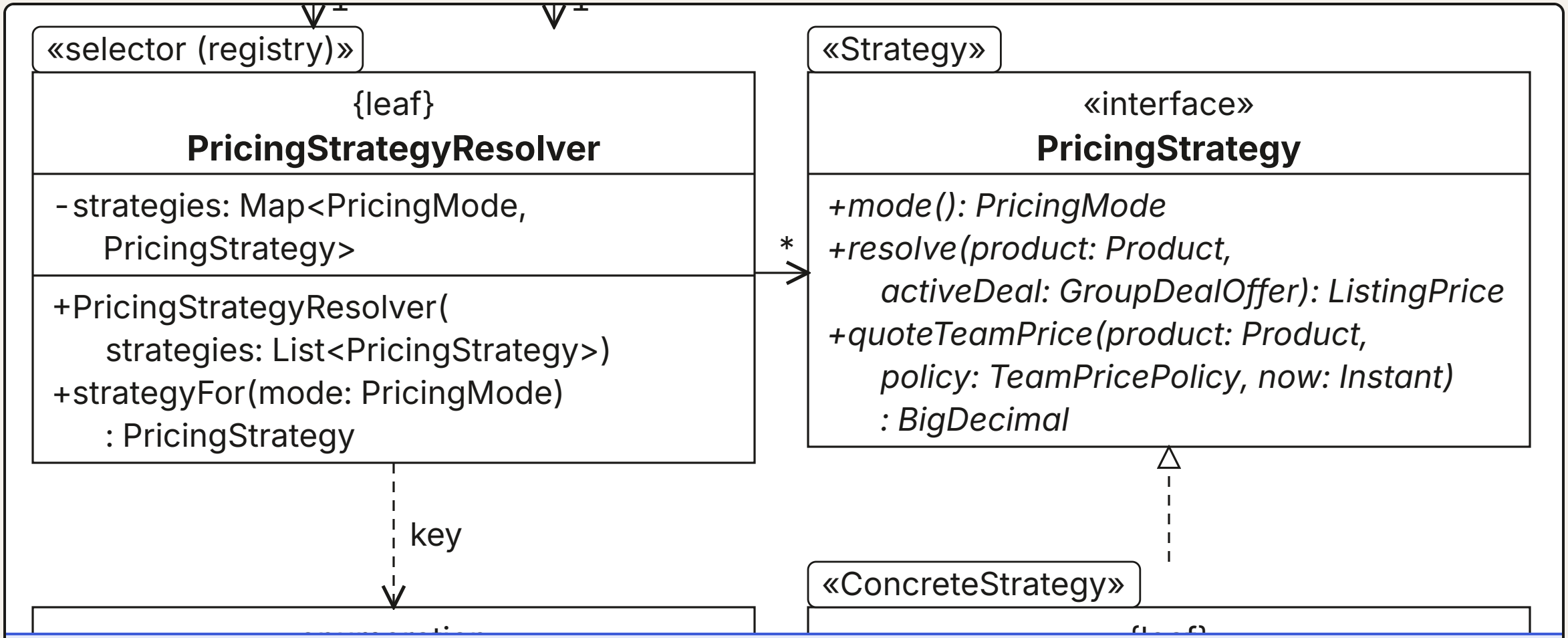
# Slide 20 · detail 2/2: publish calls entry, send




**Primitive operations:** `entry` and `send`: each transport implements only these
**Liability:** a subclass cannot change the loop
**Tested by:** `OutboxBatchFailureTest`, both providers

JC Lim

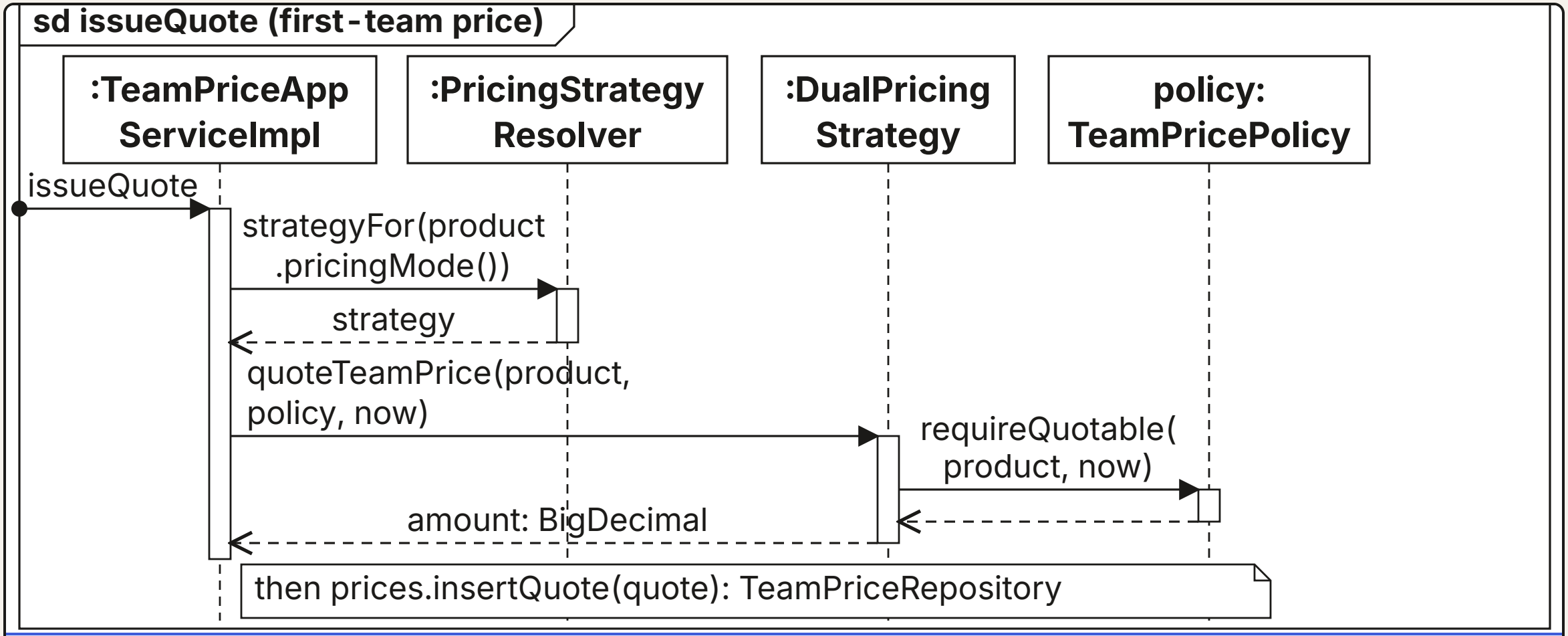
# Slide 21 · detail 1/2: resolver · Strategy




**Participants:** Strategy `PricingStrategy`, picked by a resolver
**Contexts:** the product and team-price services
**Candidates:** Strategy, Template Method (no shared skeleton), Decorator

JC Lim

# Slide 21 · detail 2/2: quote → resolve → strategy

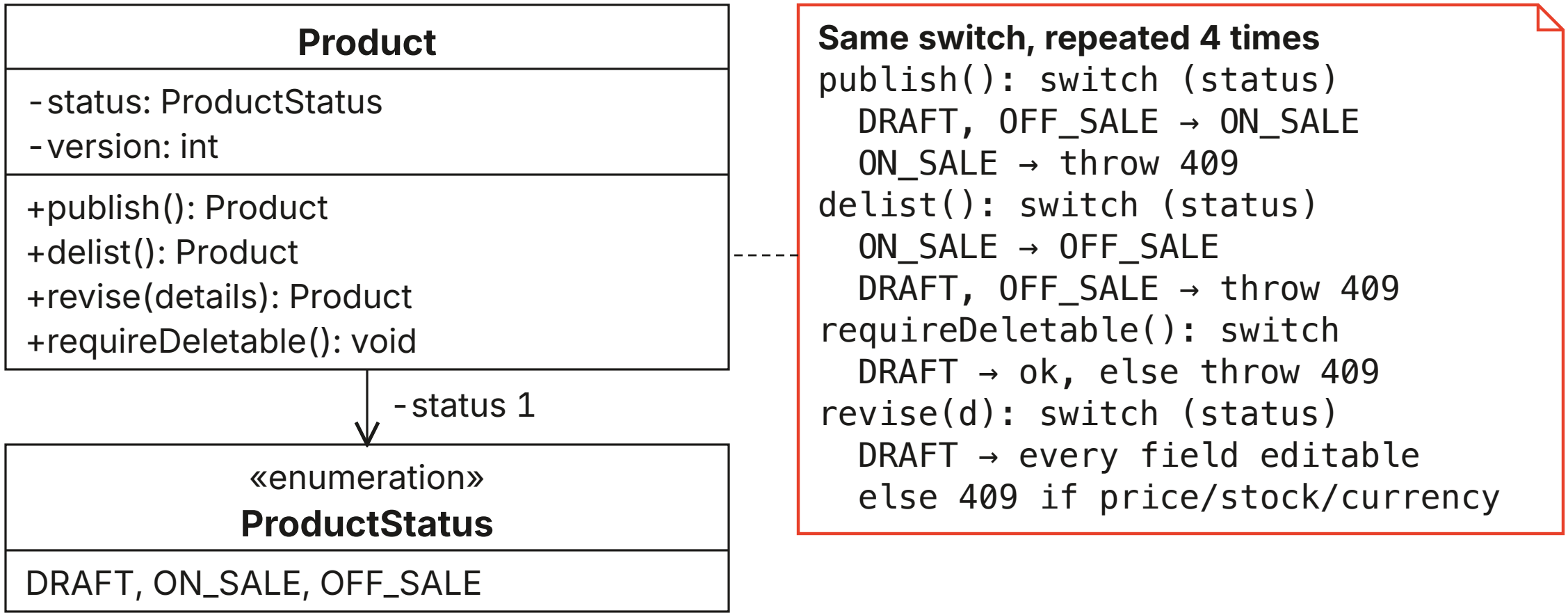



**Delegation:** each context hands the arithmetic to the ConcreteStrategy `DualPricingStrategy`
**Liability:** one strategy today; new pricing schemes are a design driver
**Tested by:** `PricingStrategyResolverTest`, `DualPricingStrategyTest`

LJ Liu Jin

# Slide 22 · detail 1/2: brute force: class

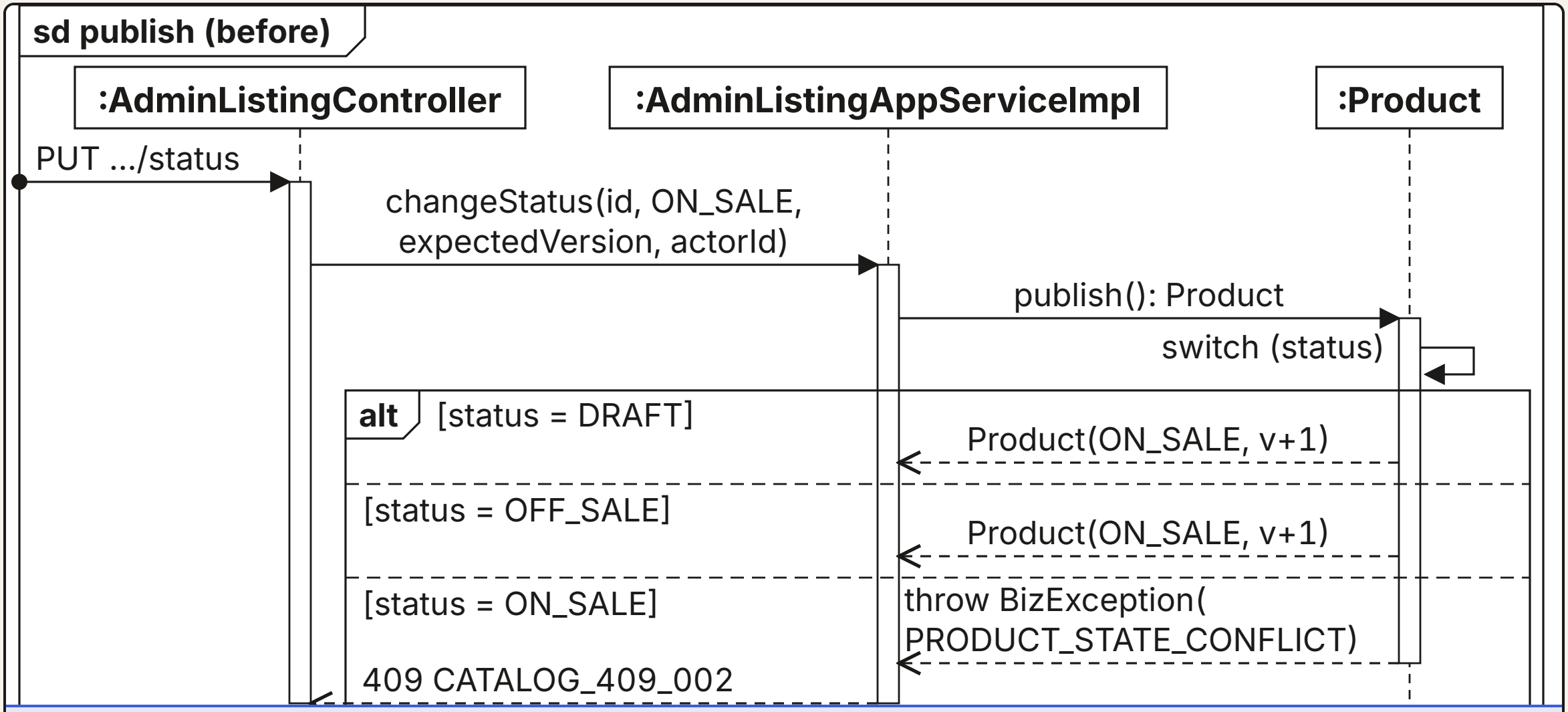
## Before (brute force): a status switch in every operation




**Brute force:** the same `switch (status)` in four operations of `Product`, in red

LJ Liu Jin

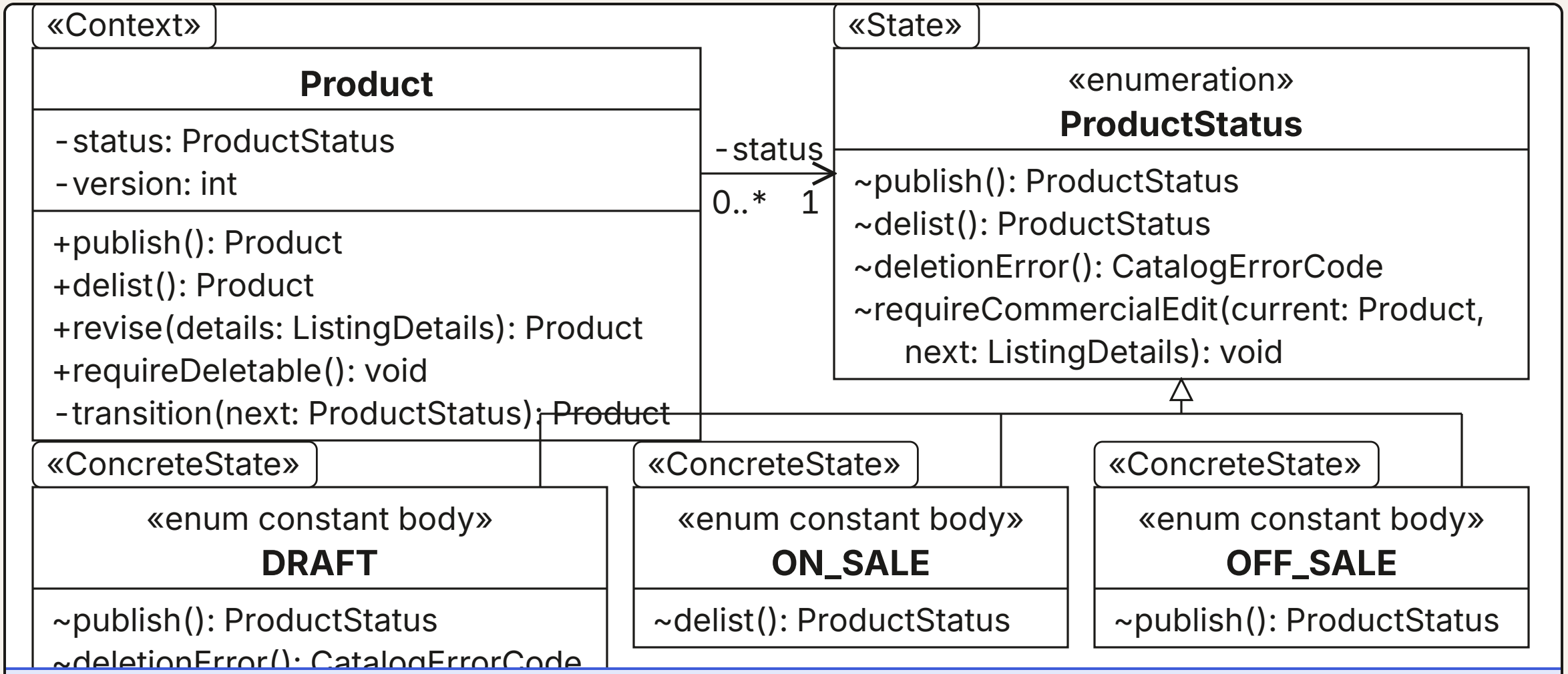
# Slide 22 · detail 2/2: brute force: sequence




**In the sequence:** `publish()` branches on the listing's status

LJ Liu Jin

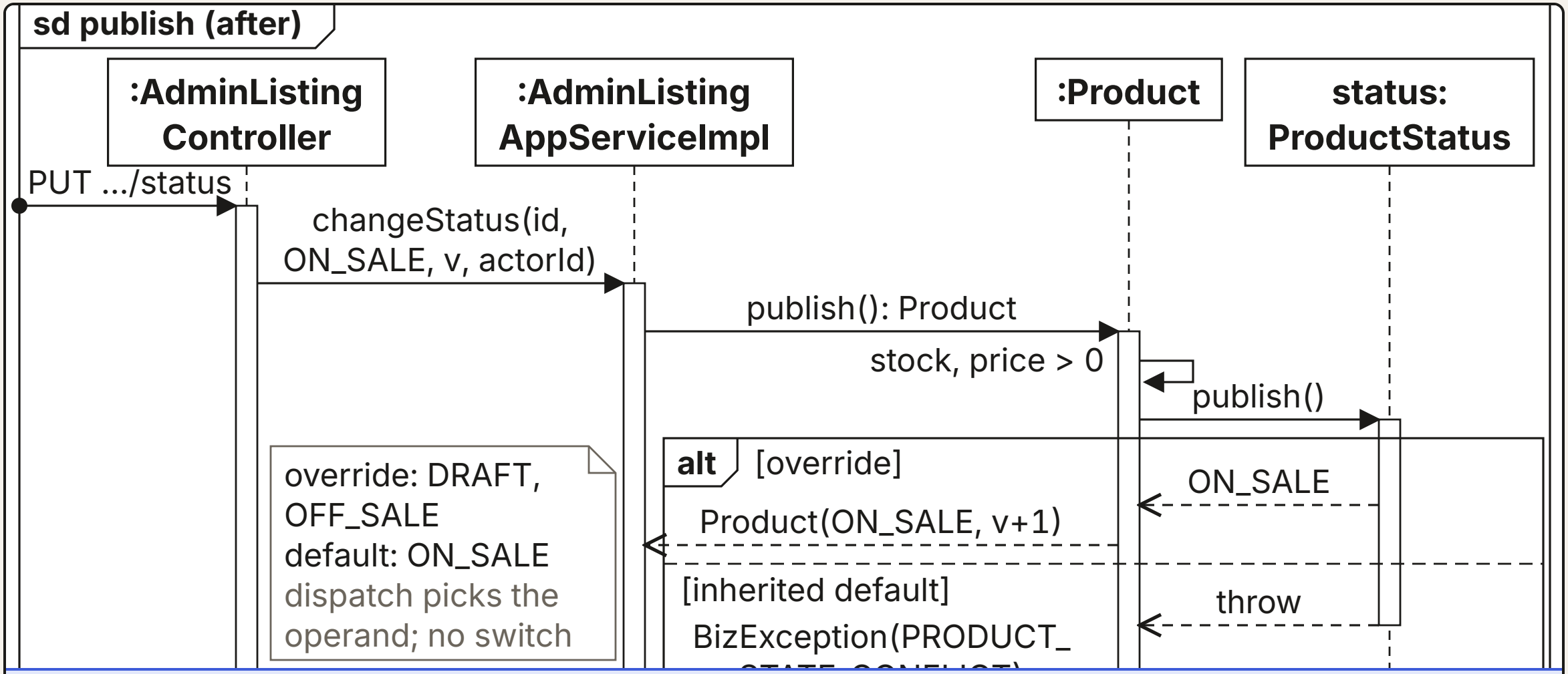
# Slide 23 · detail 1/2: State · ConcreteStates




**Participants:** Context `Product`, State `ProductStatus`
**ConcreteStates:** DRAFT, ON_SALE, OFF_SALE: enum constants with bodies, one subclass each

LJ Liu Jin

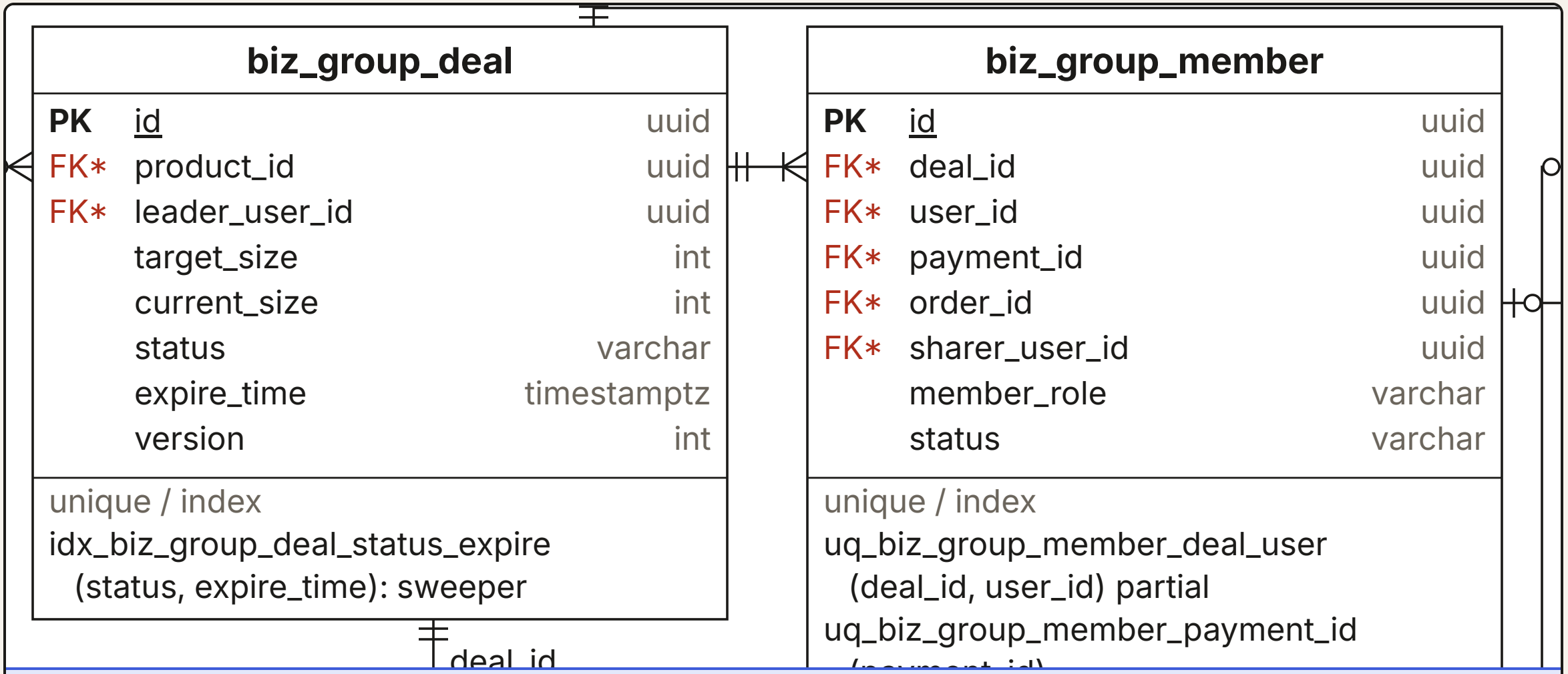
# Slide 23 · detail 2/2: after: publish delegates




**No switch:** each state answers publish its own way
**Liability:** a new state still needs a constant, its overrides and tests

LJ Liu Jin

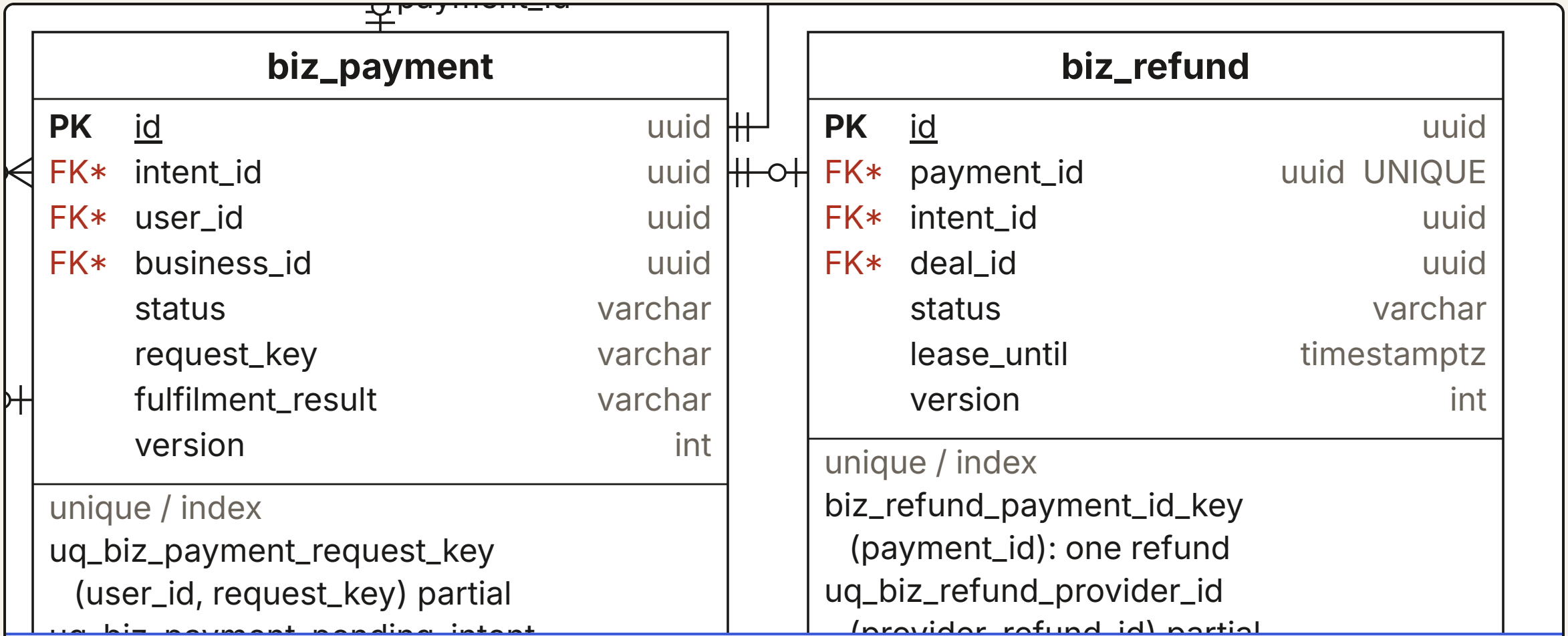
# Slide 24 · detail 1/2: deal · membership




**Keys:** UUIDv7 surrogate primary keys from PostgreSQL 18
**Race backstop:** one membership per buyer per team, by a unique index

LJ Liu Jin

# Slide 24 · detail 2/2: payment · refund




**Race backstop:** one refund per payment, by a unique key
**Associations:** logical FK columns checked by the application: our DDL convention, not a course rule
**Aggregates:** one table per aggregate root; nothing inherited, so no inheritance mapping

LJ Liu Jin

# Slide 25 · detail 1/4: Q1 · technology-facing · supporting the team

**Q1** technology-facing · supporting the team — **automated**

### Unit, component `unit`

JUnit: 162 of 185 test classes
Vitest: 77 files

### Integration `integration`

Testcontainers: 45 of 49 classes, on real PostgreSQL, Redis and ElasticMQ

### Contract, ArchUnit `integration`

openapi-diff on the committed contract
17 architecture rules

Runs in `build-quality`, `frontend-check` (required), `contract-check`

**Caught:** two red tests reproduced the join-quote defect before its fix

**Complex classes under test**

| | |
|---|---|
| **Team state machine** | `DealStatusTest`, `PaidGroupDealTest` |
| **Last-seat capacity** | `StripeLocalIntegrationTest.twentyBuyersRaceForLastSeat…` (integration) |
| **Refund idempotency** | `RefundServiceTest.lostResponseRetriesSameDurableKey` |

LJ Liu Jin

# Slide 25 · detail 2/4: Q2 · business-facing · supporting the team

**Q2** business-facing · supporting the team — **automated**

### Story acceptance (system)

`StripeRolloutAcceptanceIT`: real HTTP, scheduler, signed webhooks

### End-to-end (system)

Playwright: 8 specs, on the preview build and on dev after each deploy

Runs in `build-quality`, `e2e-preview`, `e2e-dev`

**Caught:** a 502 on e2e-dev during a deploy (3–4 Oct): one host

LJ Liu Jin

# Slide 25 · detail 3/4: Q3 · business-facing · critiquing the product

**Q3** business-facing · critiquing the product — **manual**

**Exploratory, acceptance** system

By hand, logged
Stripe Sandbox START and JOIN in a browser (20 Sep, local)
Sign-in cookie Secure and HttpOnly on dev (8 Oct)

**Caught:** no defect logged from these runs

Not done: acceptance testing with outside users

LJ Liu Jin

# Slide 25 · detail 4/4: Q4 · technology-facing · critiquing the product

**Q4** technology-facing · critiquing the product **tools**

**Performance, load** system

k6 on the SWE5001 ECS environment, by hand (next slide)

**Security scanning** types

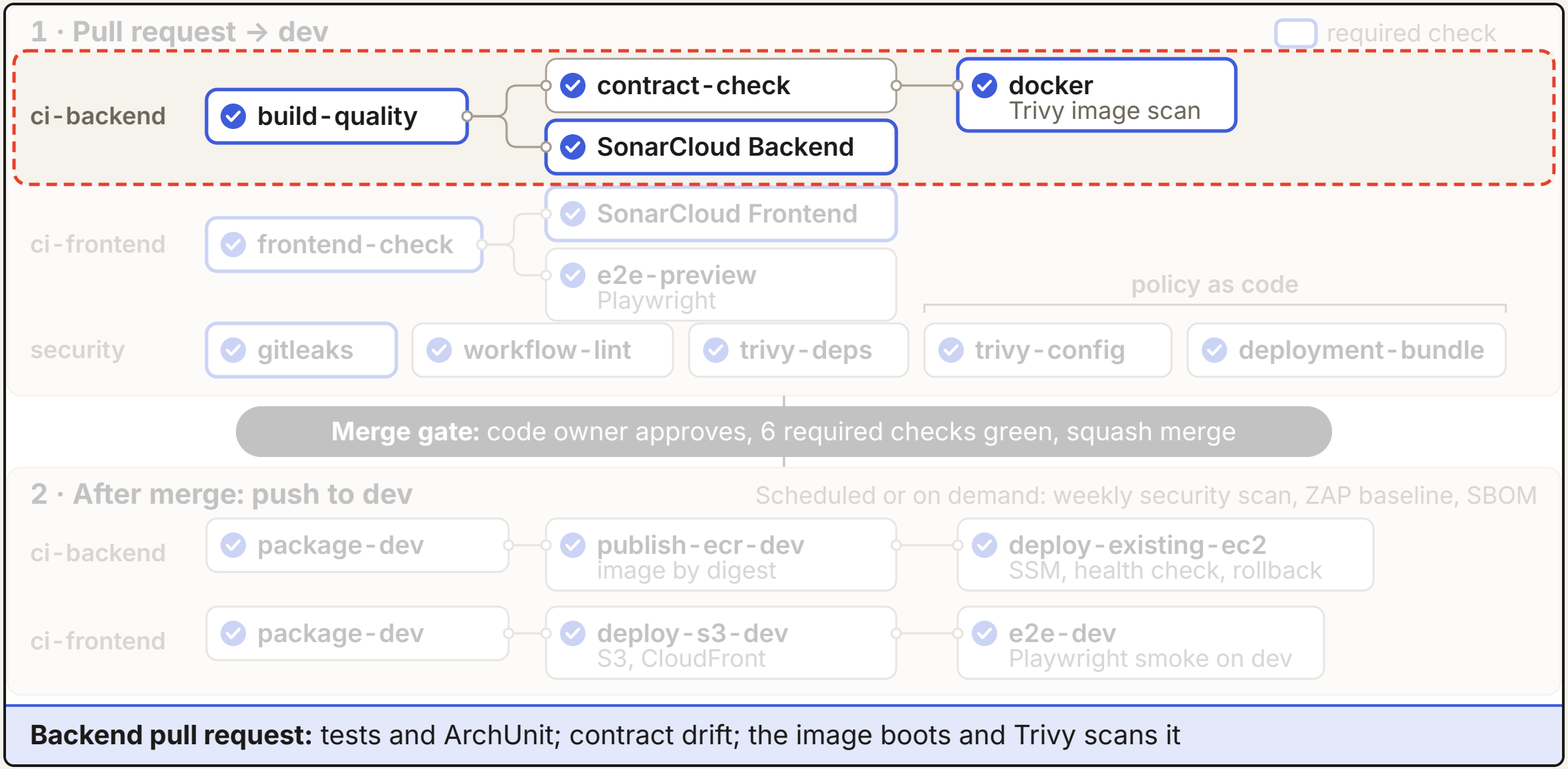
Static analysis (SAST)
Dependencies (SCA)
Image and IaC configuration
Secrets
Dynamic, passive (DAST): OWASP ZAP on anonymous dev pages, advisory

Runs in `security.yml`, `build-quality`, `docker` gate the pipeline; `dast-baseline` runs weekly, advisory (slide 30)

**Caught:** full-history gitleaks: 24 matches, 4 badge tokens still open (slide 32); ZAP on dev: 2 Medium, 4 Low CSP and header findings (22 Oct)

PS Pan

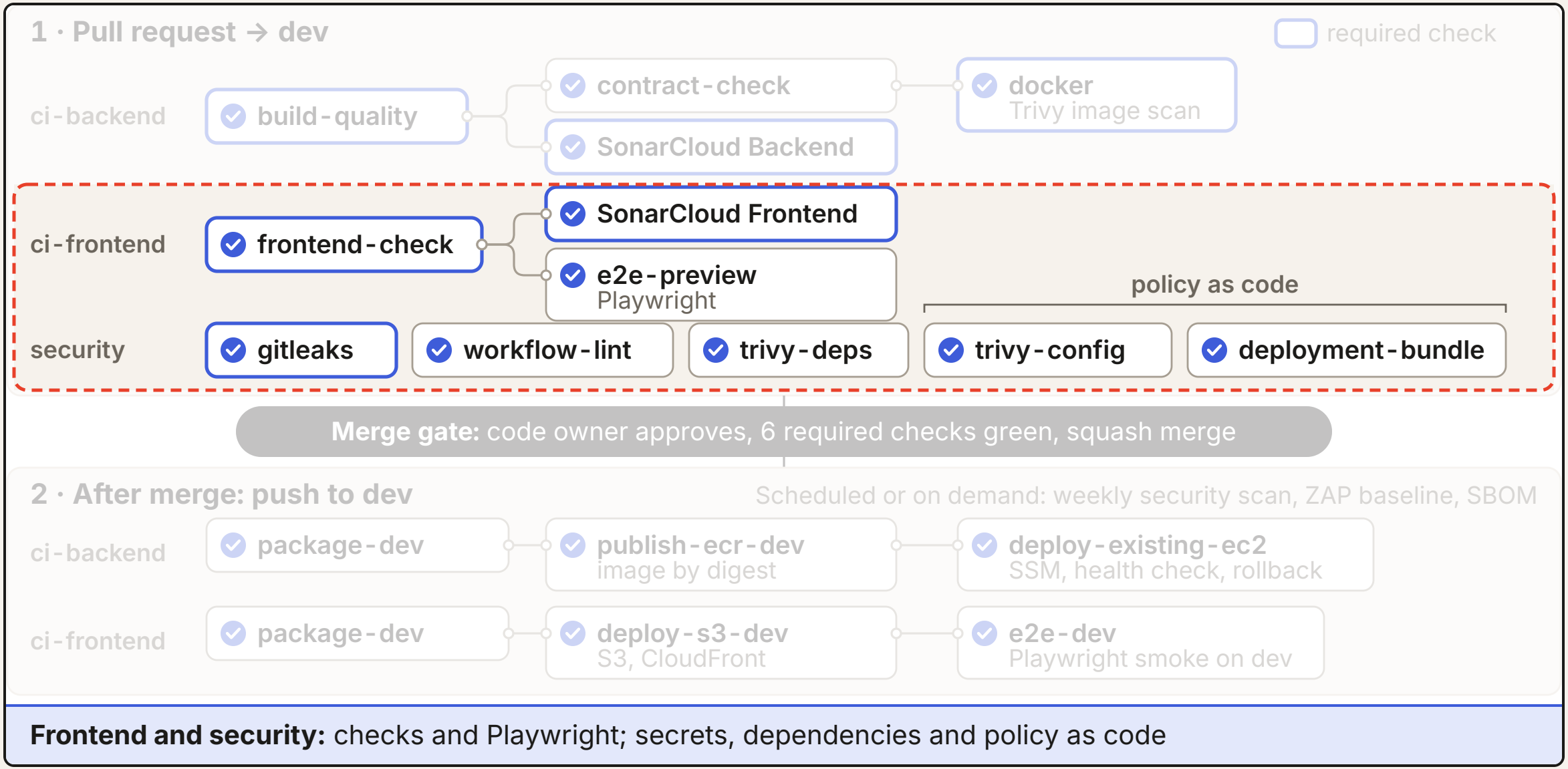
# Slide 29 · detail 1/4: PR · backend




**Backend pull request:** tests and ArchUnit; contract drift; the image boots and Trivy scans it

PS Pan

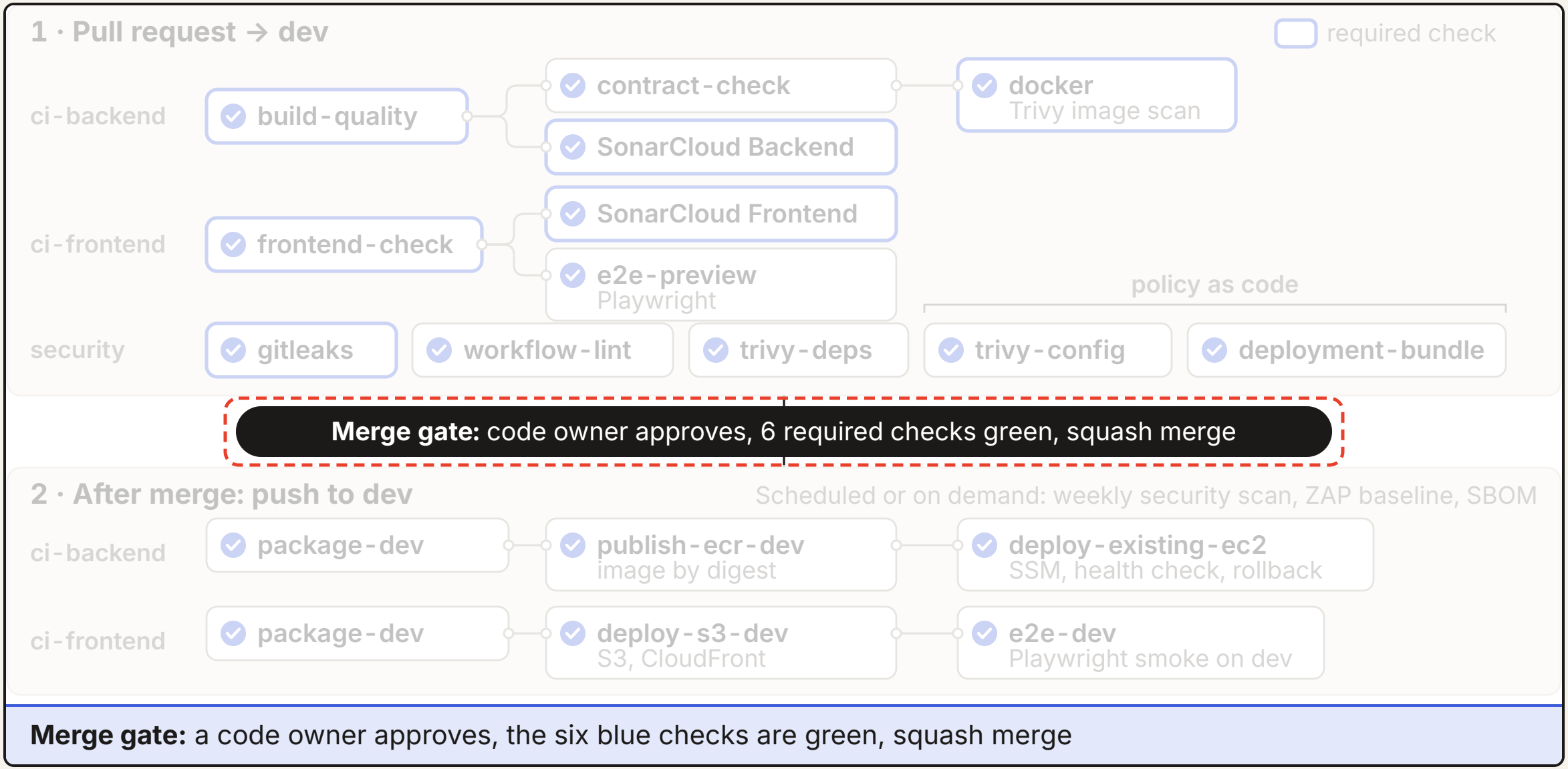
# Slide 29 · detail 2/4: PR · frontend, security




**Frontend and security:** checks and Playwright; secrets, dependencies and policy as code

PS Pan

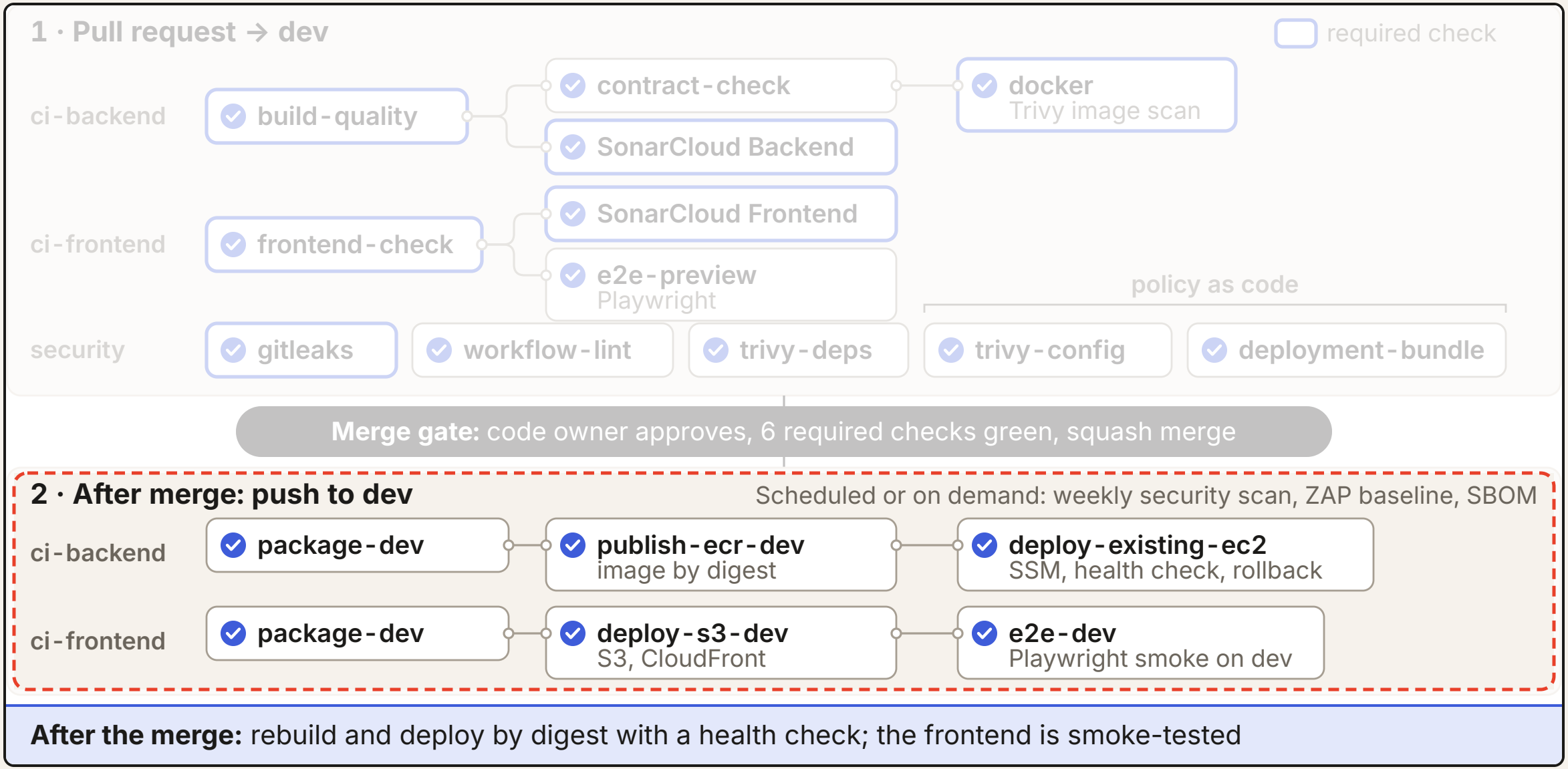
# Slide 29 · detail 3/4: merge gate




**Merge gate:** a code owner approves, the six blue checks are green, squash merge

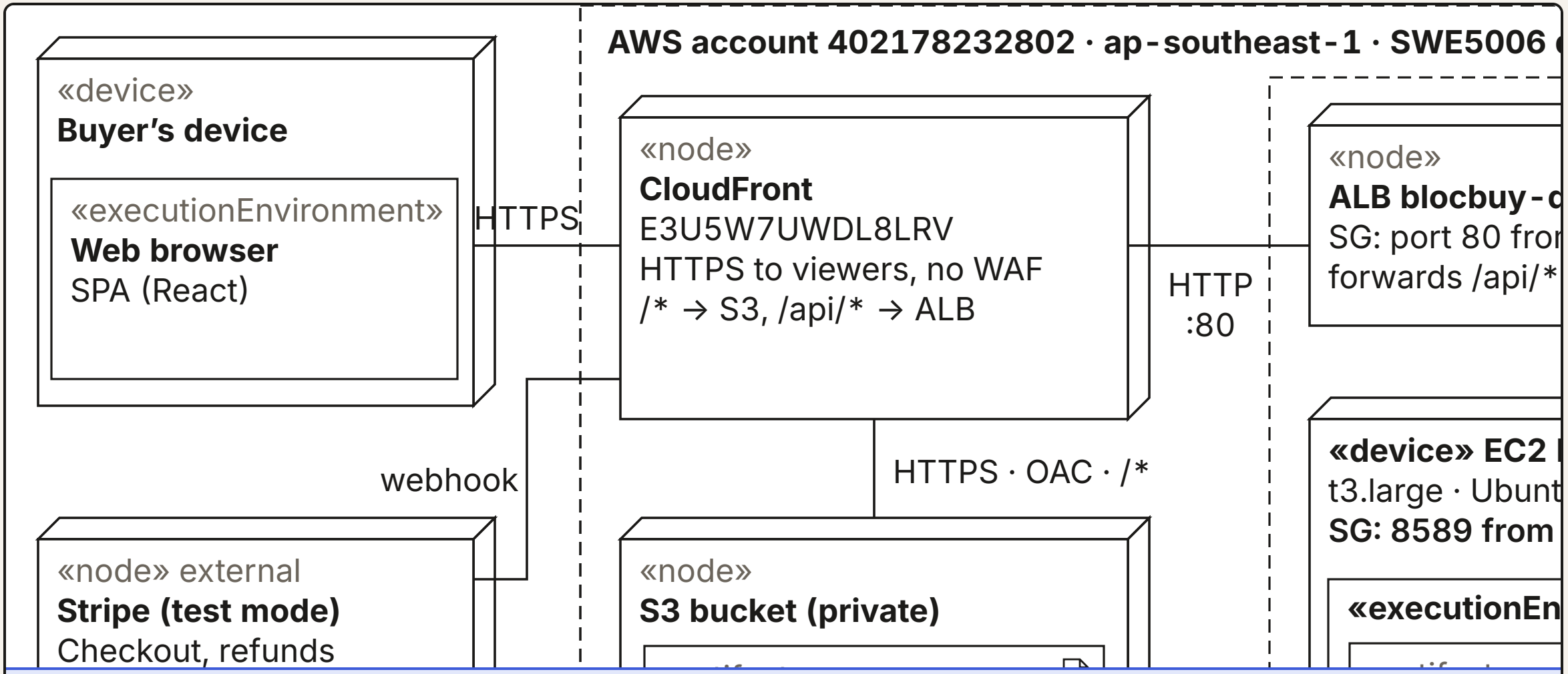
PS Pan

# Slide 29 · detail 4/4: after merge



PS Pan

# Slide 31 · detail 1/3: request path

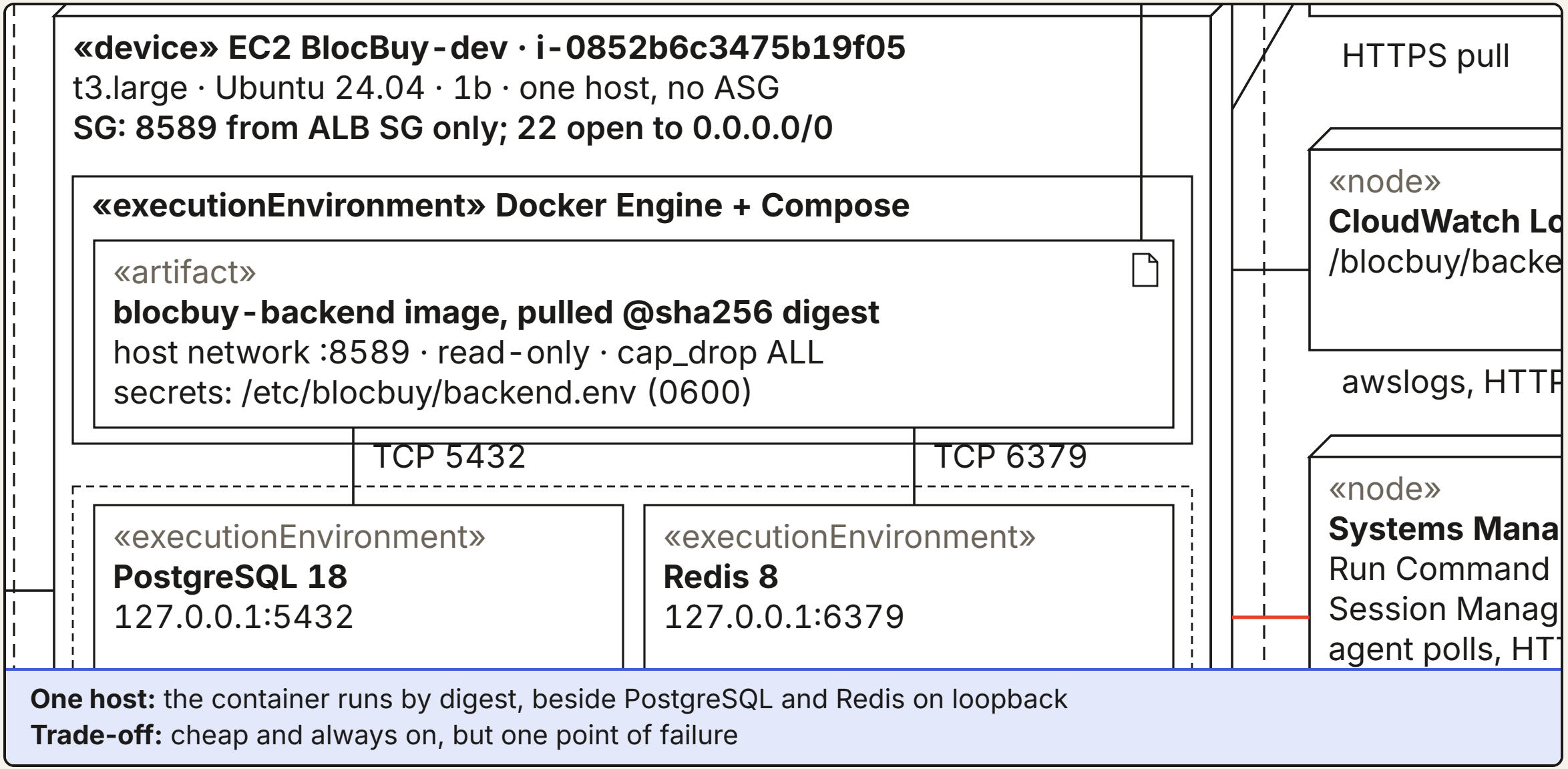



**One environment:** dev; the SWE5006 production stack was planned, never applied

**Request path:** the browser talks only to CloudFront: files from a private bucket, API calls to the load balancer

PS Pan

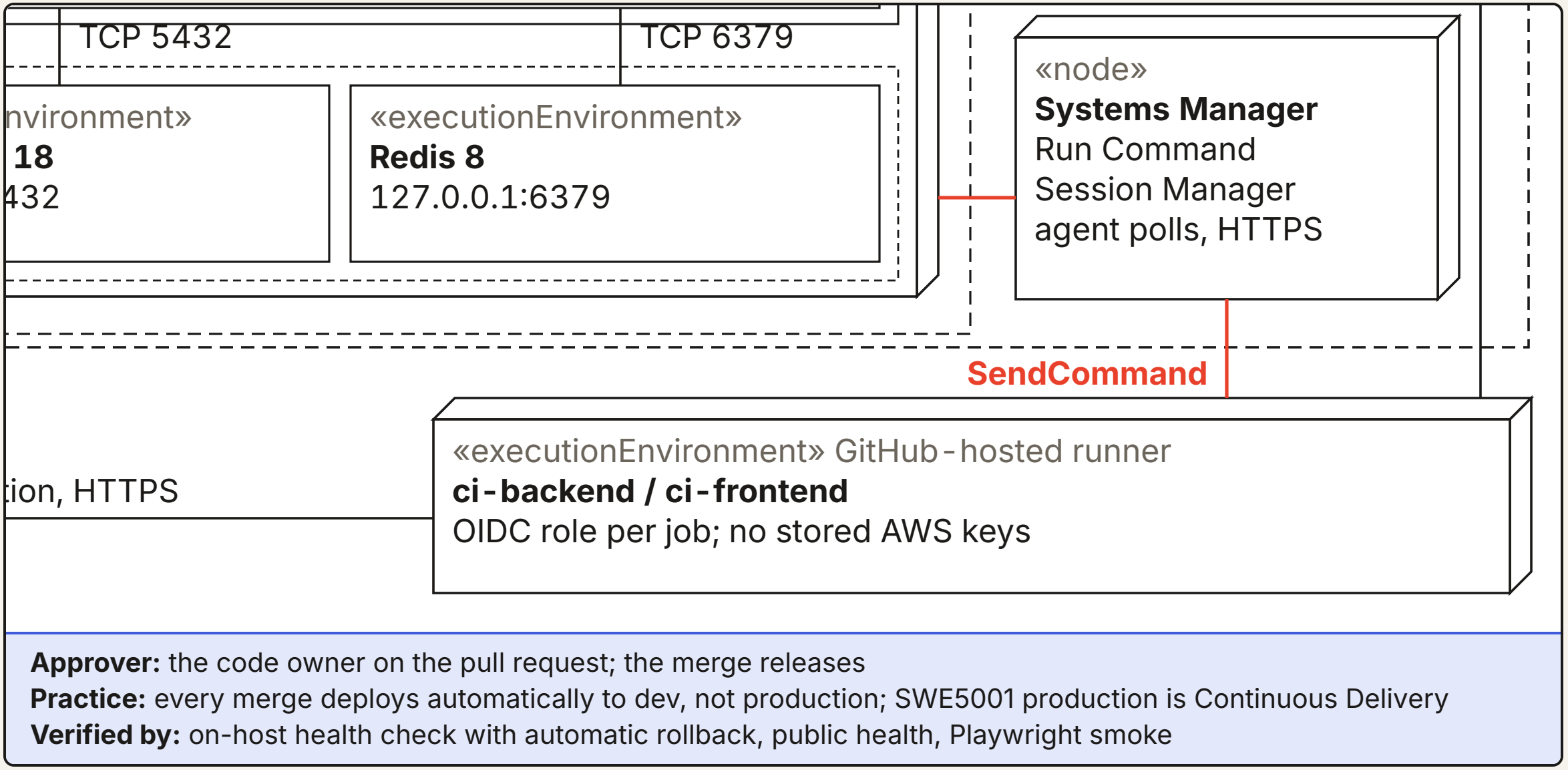
# Slide 31 · detail 2/3: one host




**One host:** the container runs by digest, beside PostgreSQL and Redis on loopback
**Trade-off:** cheap and always on, but one point of failure

PS Pan

# Slide 31 · detail 3/3: deploy path



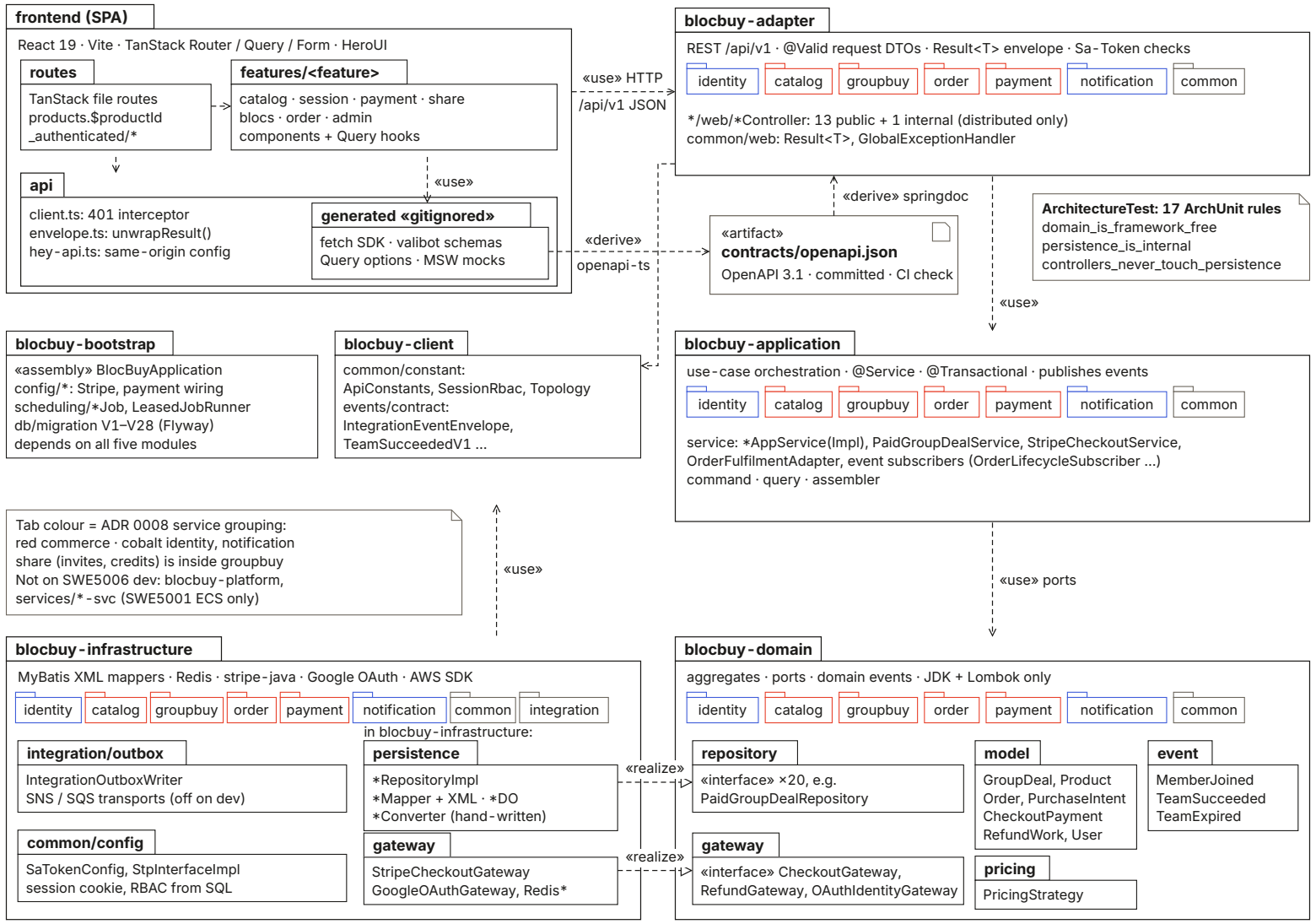

**Approver:** the code owner on the pull request; the merge releases
**Practice:** every merge deploys automatically to dev, not production; SWE5001 production is Continuous Delivery
**Verified by:** on-host health check with automatic rollback, public health, Playwright smoke

# Diagrams at full size

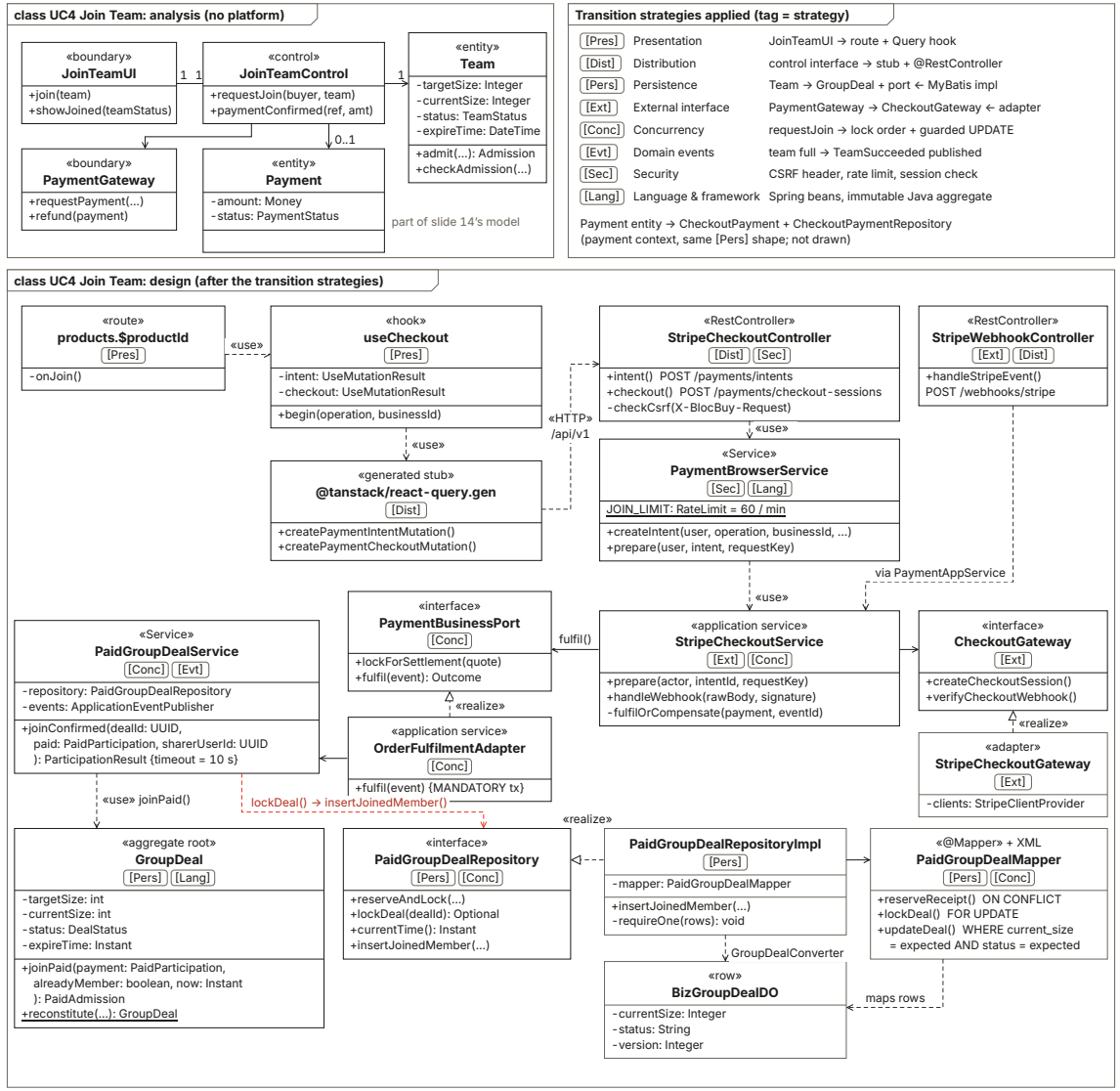
| | | | | | |
|---|---|---|---|---|---|
| F1 | **Logical view: COLA layers and packages**<br>slide 9 | F2 | **Transition strategies: the UC4 pair**<br>slide 12 | F3 | **Use case diagram**<br>slide 13 |
| F4 | **UC4 analysis class diagram**<br>slide 14 | F5 | **UC4 analysis sequence**<br>slide 14 | F6 | **UC4 design class diagram**<br>slide 15 |
| F7 | **UC4 design sequence, paid join by webhook**<br>slide 16 | F8 | **Team-formation statechart**<br>slide 17 | F9 | **Factory Method: start a team**<br>slide 18 |
| F10 | **Observer: team outcome**<br>slide 19 | F11 | **Template Method: outbox transports**<br>slide 20 | F12 | **Strategy: pricing**<br>slide 21 |
| F13 | **Listing lifecycle before State (brute force)**<br>slide 22 | F14 | **Listing lifecycle after State**<br>slide 23 | F15 | **Core schema**<br>slide 24 |
| F16 | **CI/CD in detail: every step, trigger and path filter**<br>slide 29 | F17 | **Dev deployment**<br>slide 31 | F18 | **Expiry and refund sequence (ECS)**<br>B5 |
| F19 | **SWE5001 production topology**<br>B10 | F20 | **UC9 listing statechart**<br>B13 | F21 | **Entity-relationship diagram, all tables**<br>B16 |

*Click a diagram. It opens fitted; the toolbar switches to actual size or zooms; scroll with the trackpad or mouse wheel (arrow keys still change slides). Press B for the backup index.*

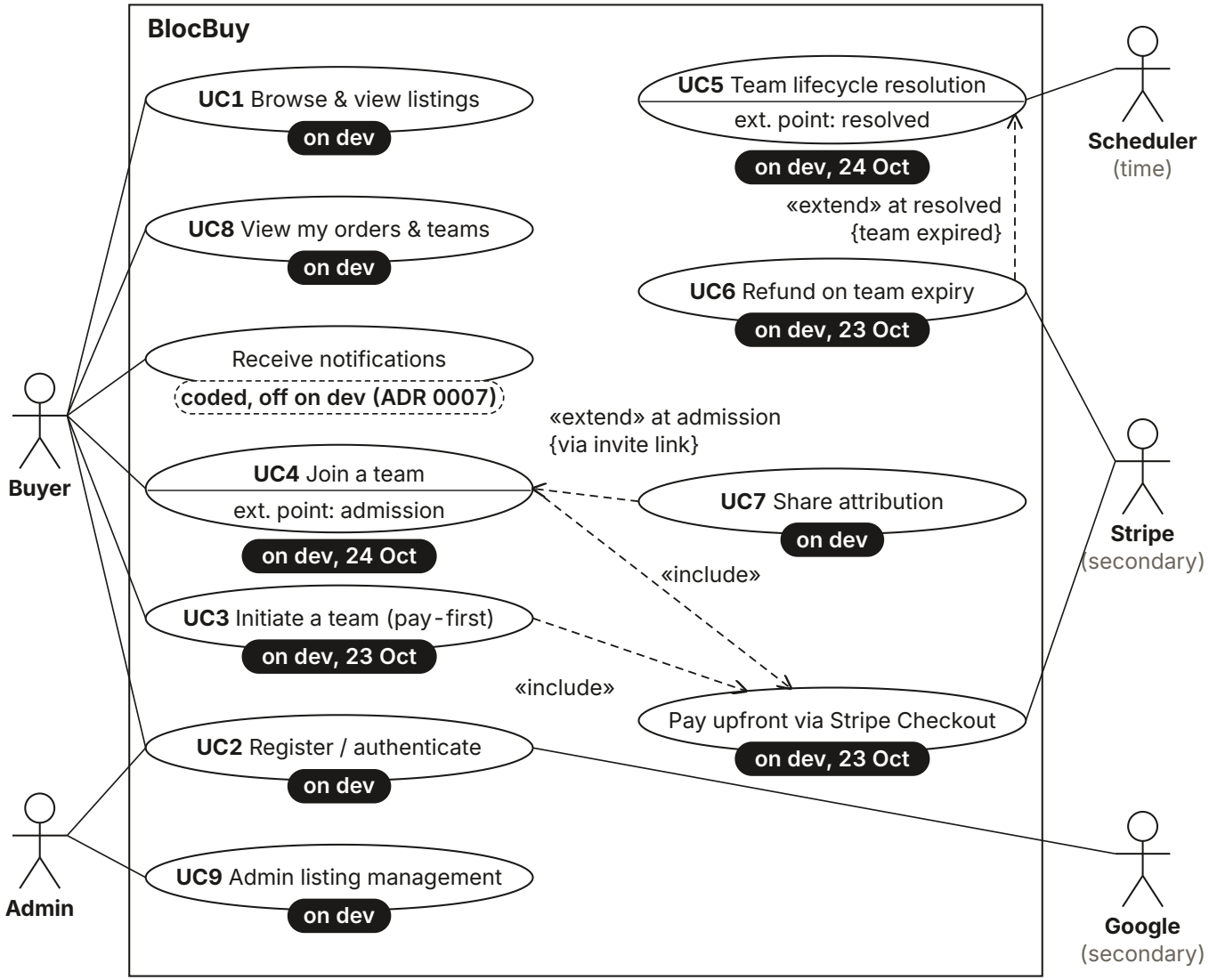
# Logical view: COLA layers and packages

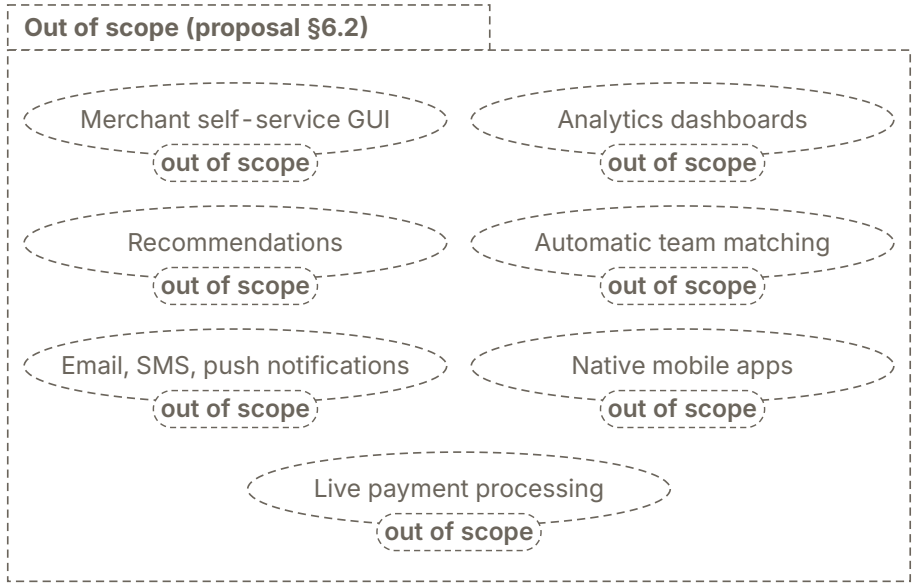


# Transition strategies: the UC4 pair



# Use case diagram



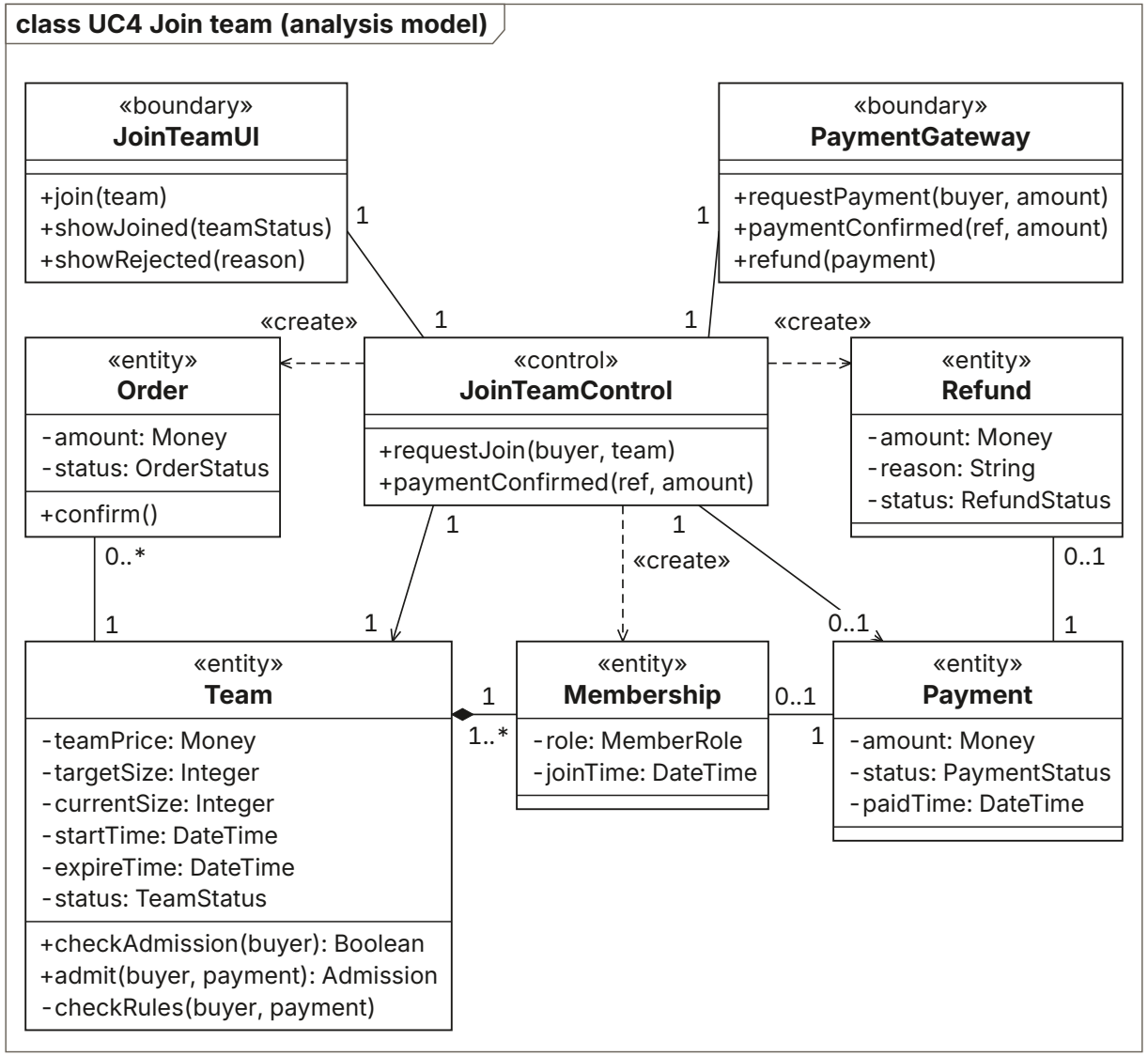

**Delivery status on dev, 24 Oct (team ruling)**

| | |
|---|---|
| on dev | deployed and usable on dev |
| on dev, 24 Oct | paid step first shown on that day |
| coded, off on dev (ADR 0007) | SQS consumer runs on SWE5001 only |
| out of scope | excluded by the proposal |

----> «include»: base → included
«extend»: extending → base

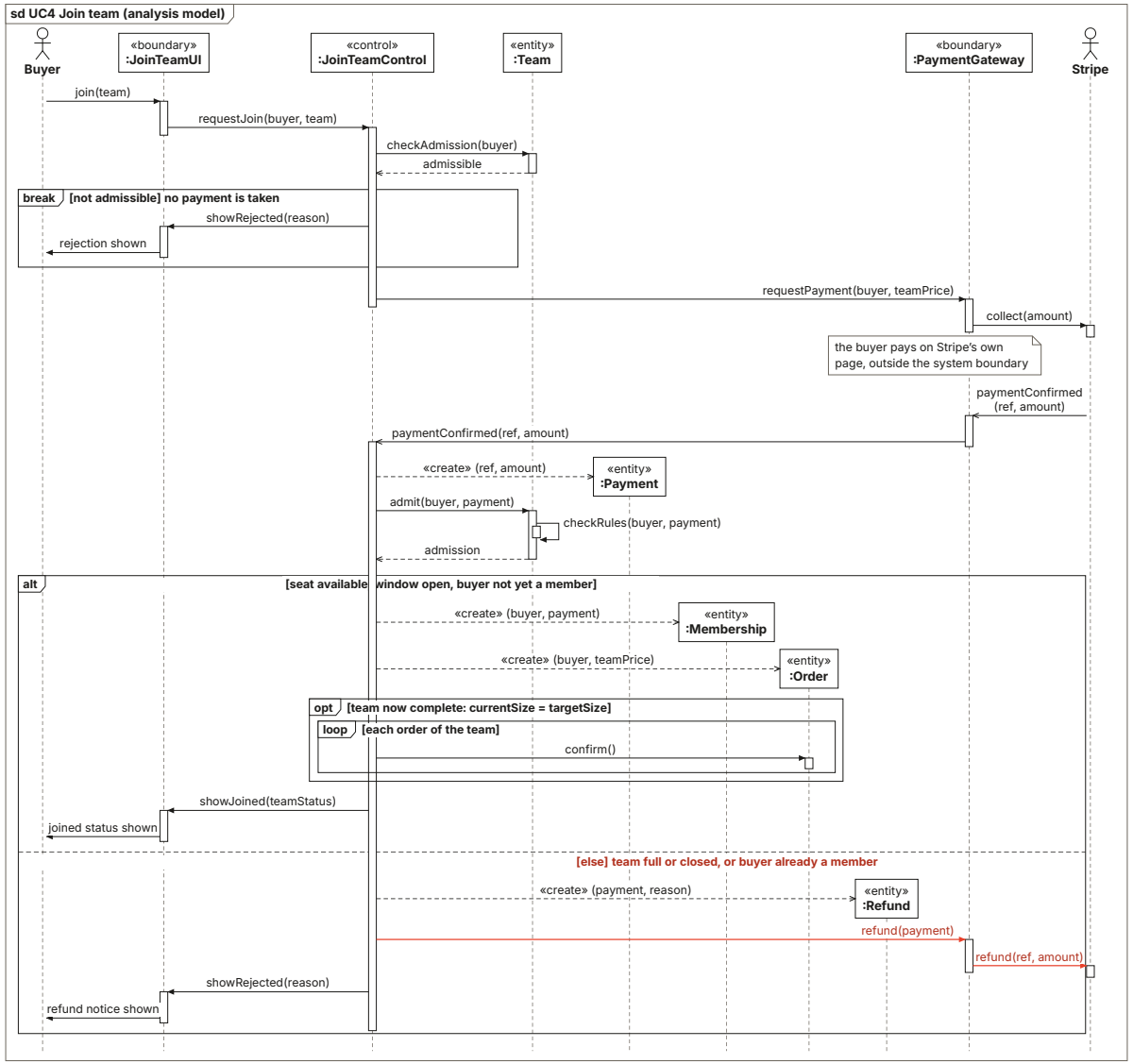
BACKUP · Q&A ONLY F4

# UC4 analysis class diagram

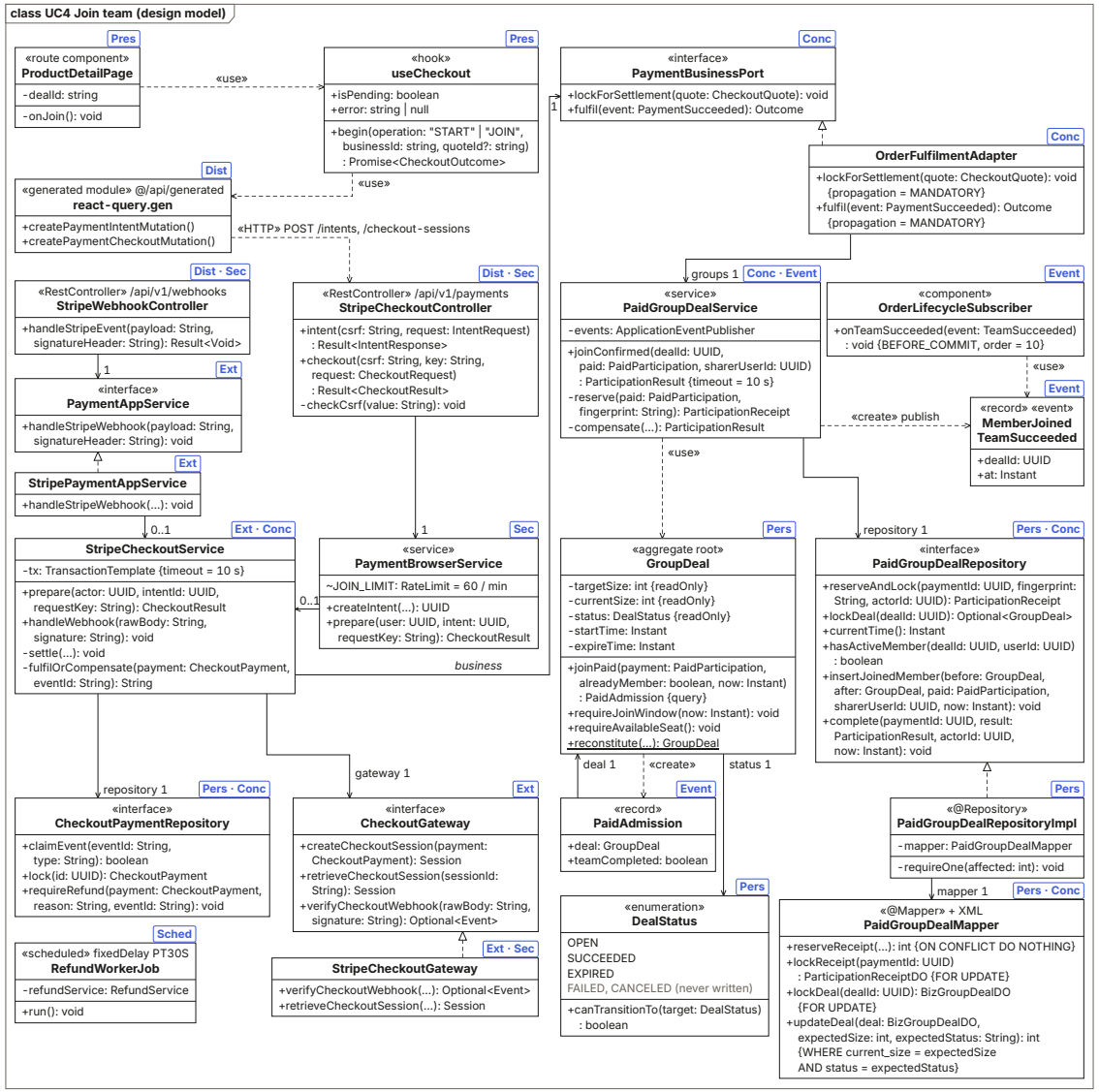


BACKUP · Q&A ONLY F5

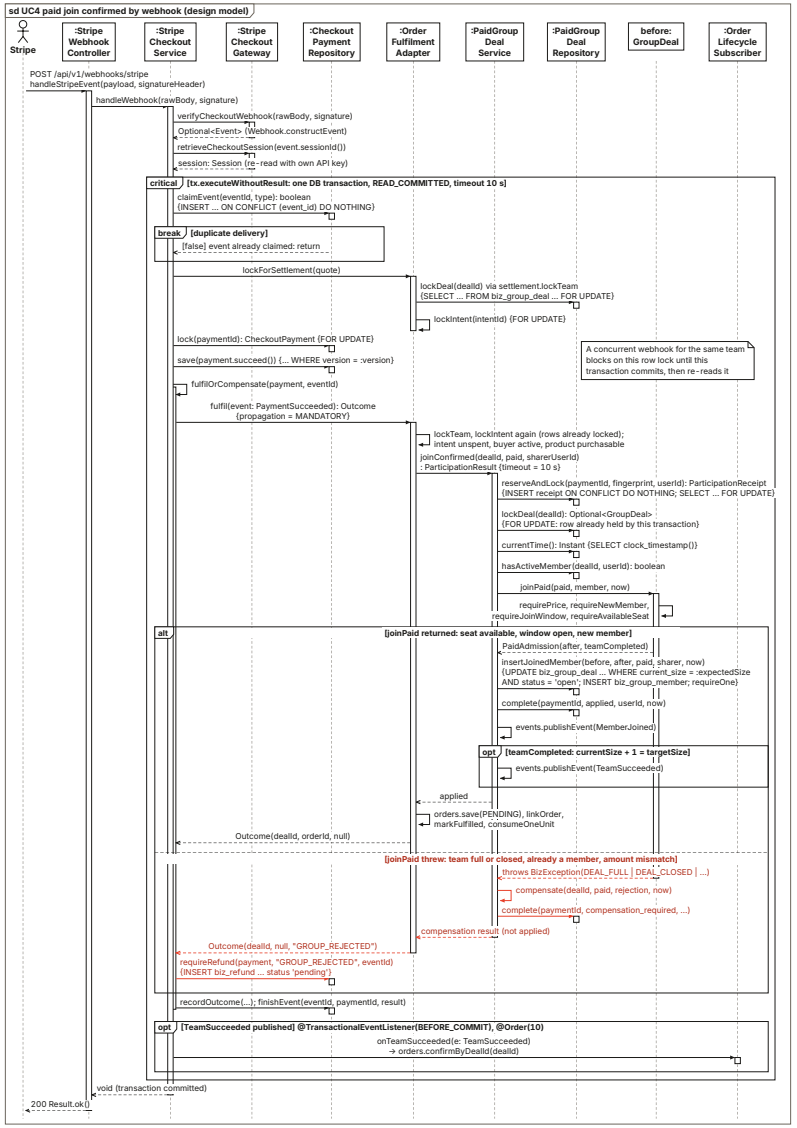
# UC4 analysis sequence



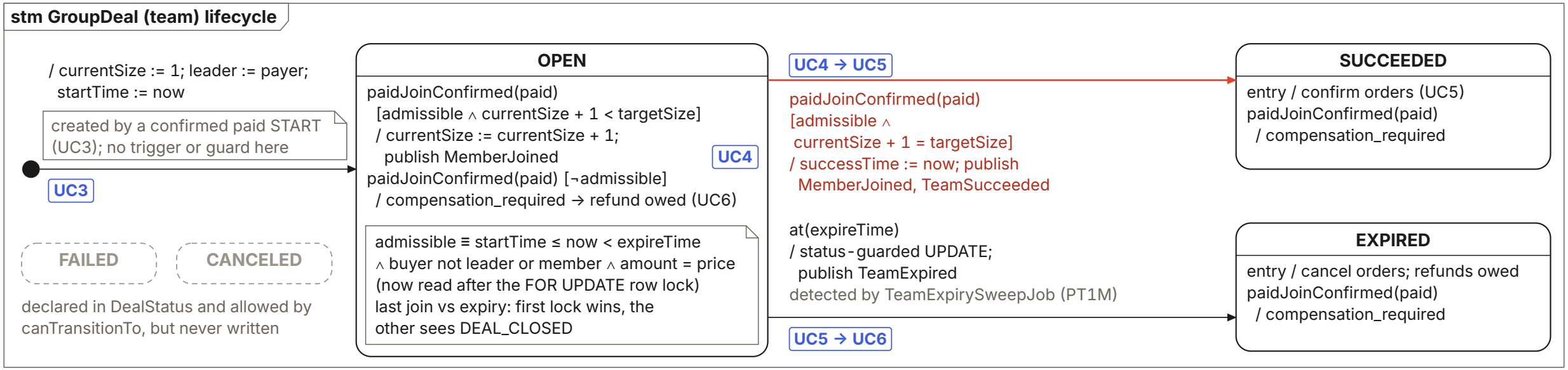
# UC4 design class diagram



# UC4 design sequence, paid join by webhook

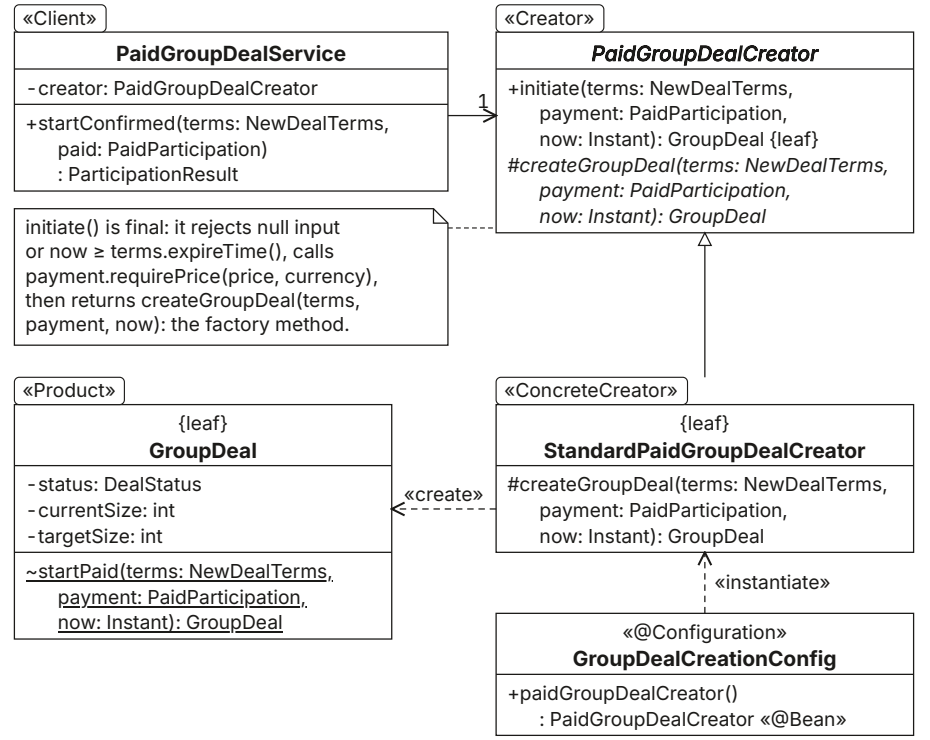


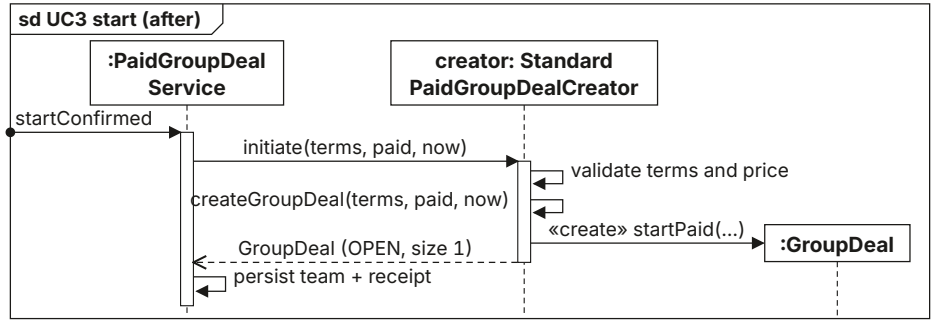
# Team-formation statechart



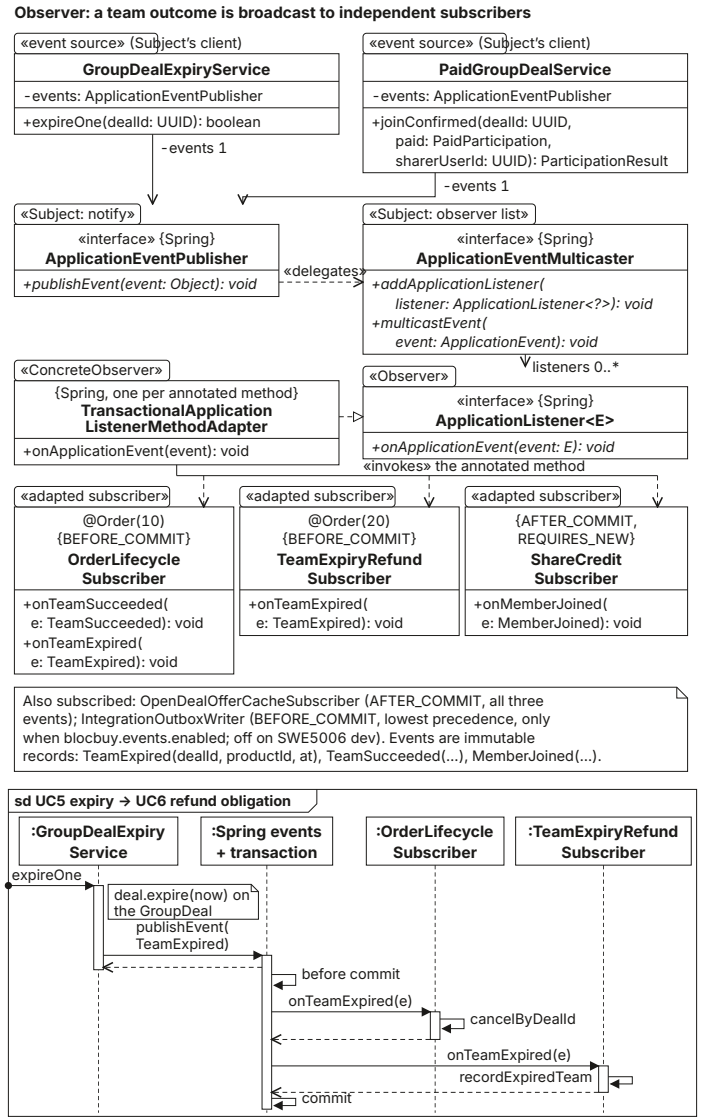
# Factory Method: start a team

**Factory Method: UC3 start creates the team (GroupDeal)**

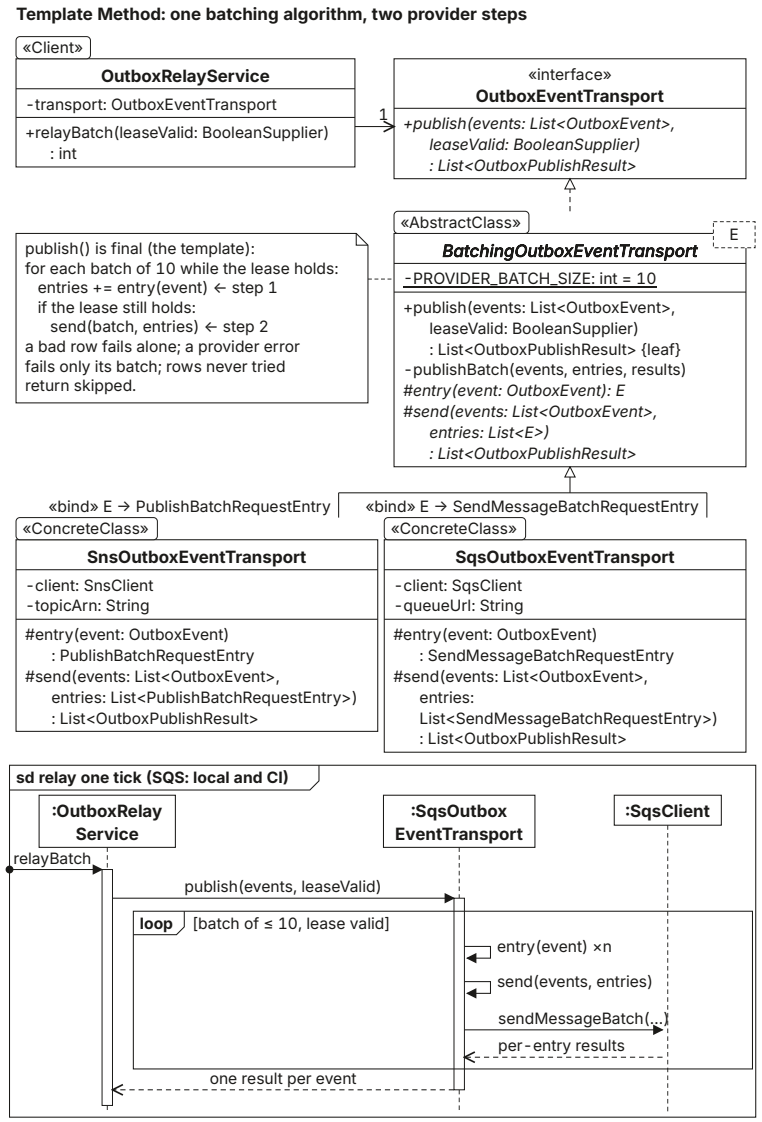


# Observer: team outcome

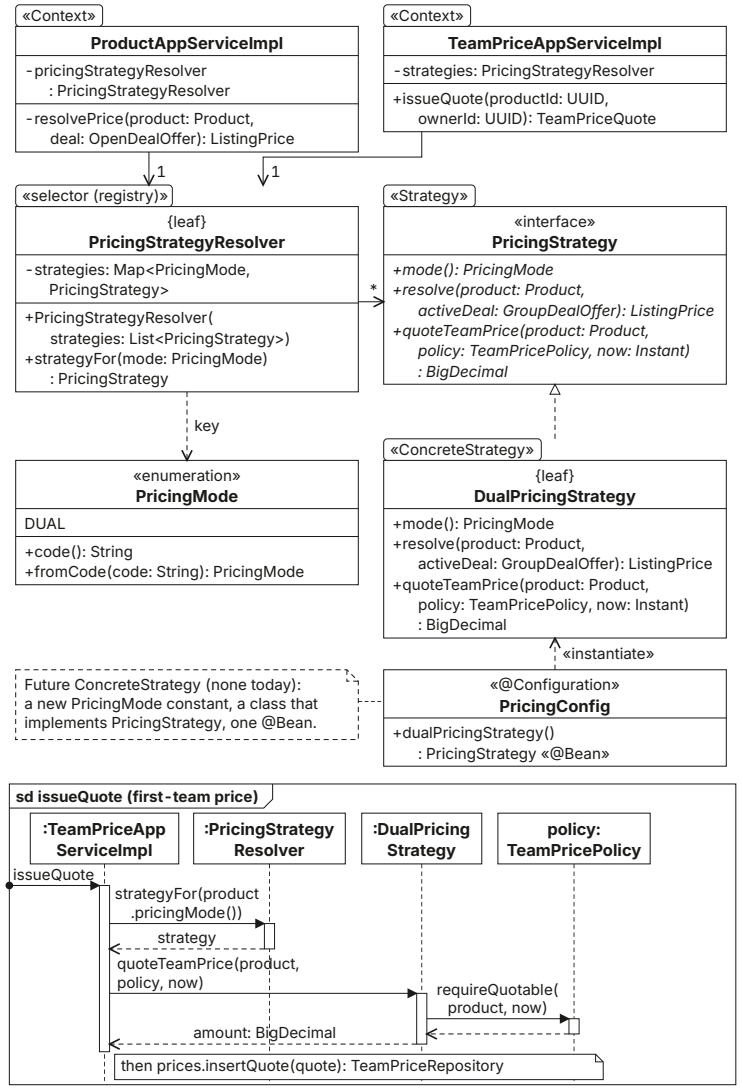


# Template Method: outbox transports



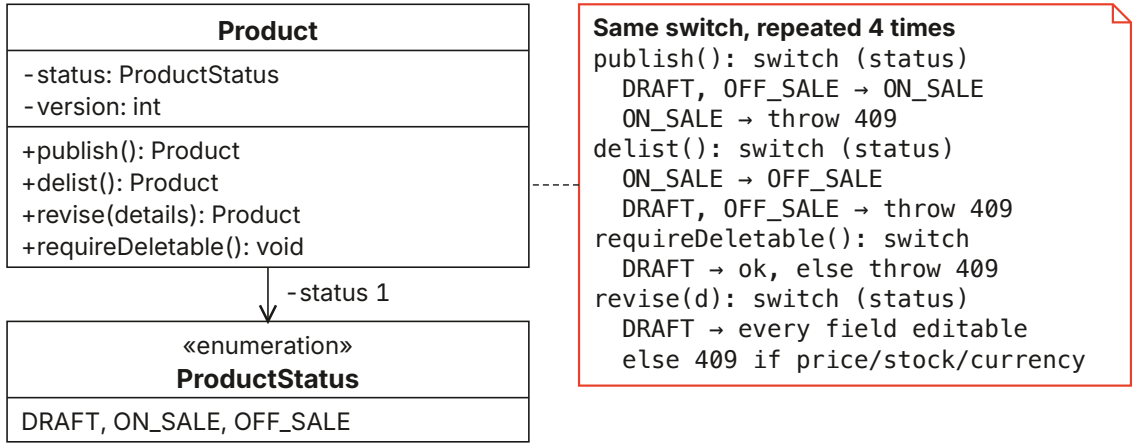
# Strategy: pricing

**Strategy: the pricing algorithm is chosen by the product's pricing mode**

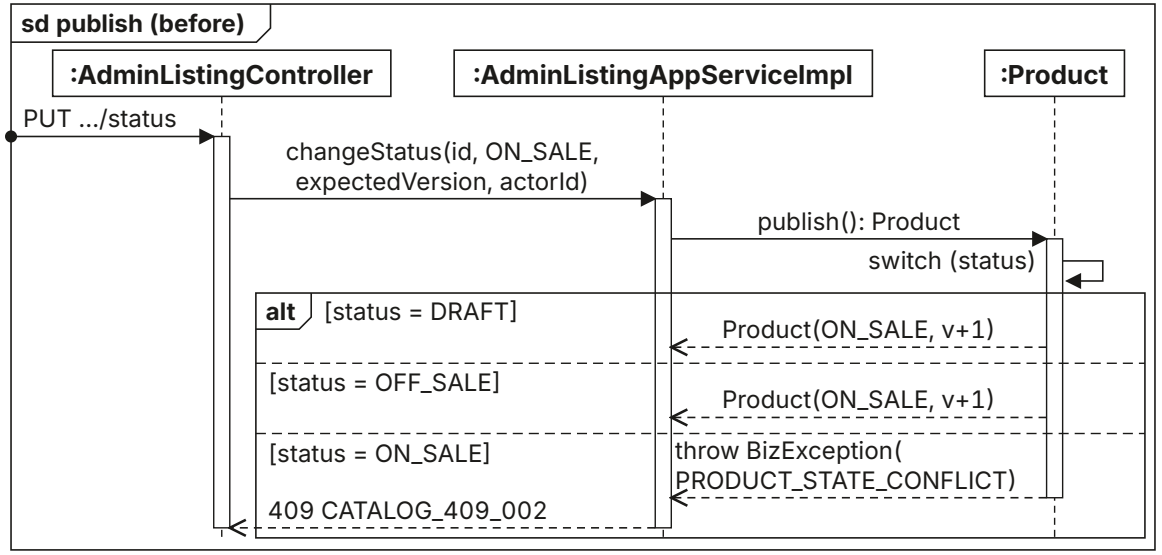


# Listing lifecycle before State (brute force)

**Before (brute force): a status switch in every operation**

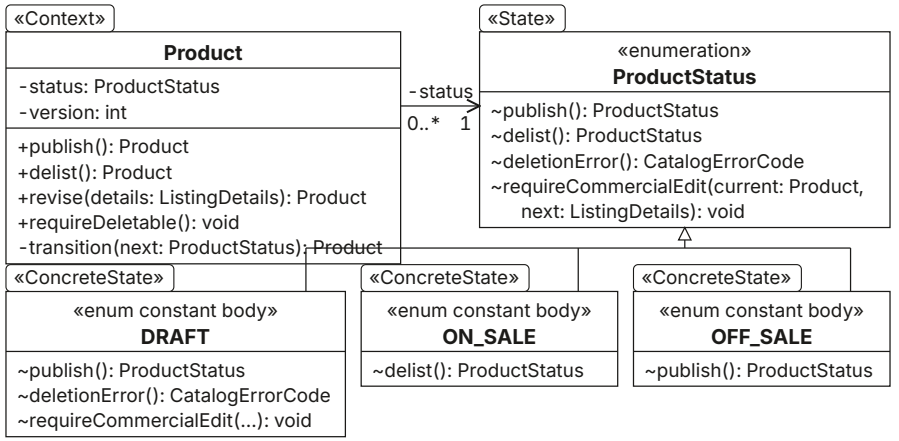



Problem: each operation repeats the status test; a new state edits all four.

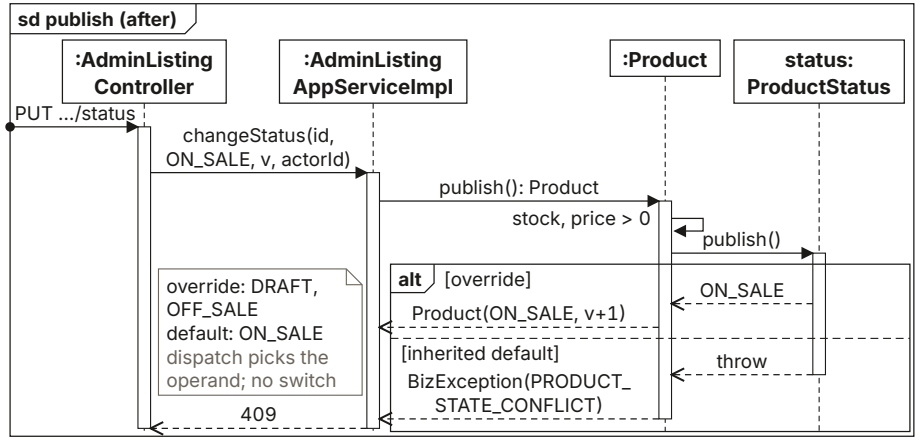


# Listing lifecycle after State

**After: State — Product delegates lifecycle rules to its ProductStatus**

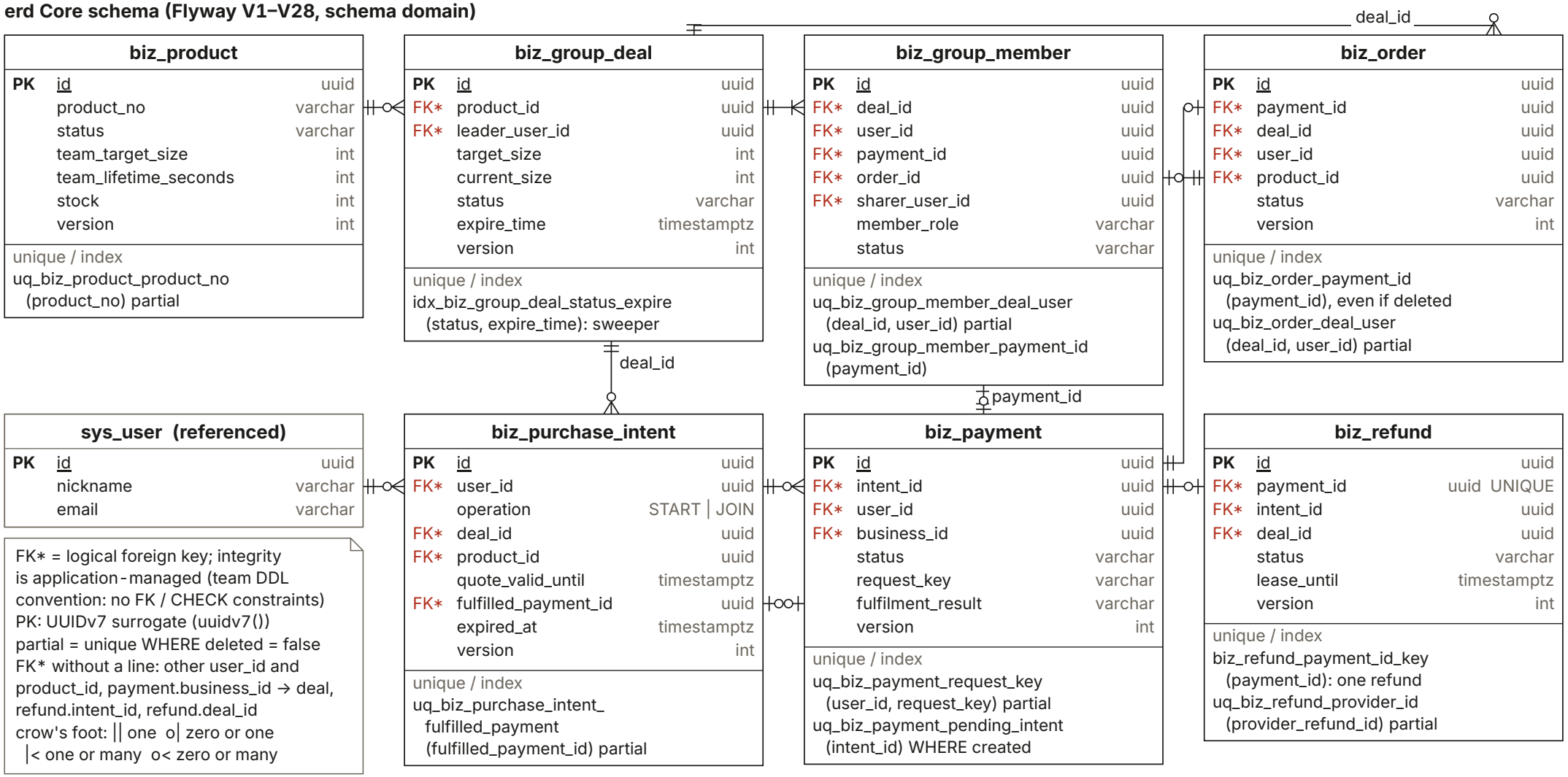



**Implementation (ProductStatus.java, Product.java)**
Each constant body compiles to an anonymous final subclass (ProductStatus$1, $2, $3) that overrides the State operations.
Inherited defaults: publish(), delist() throw PRODUCT_STATE_CONFLICT (409).
Product.publish(): stock > 0 and price > 0, then transition(status.publish()).



# Core schema

**erd Core schema (Flyway V1–V28, schema domain)**



# CI/CD in detail: every step, trigger and path filter

## 1 · On every pull request into dev

trigger: pull_request → dev (ci-backend also on label changes); jobs skip when their paths are untouched

### ci-backend.yml

backend/**, contracts/**

- **build-quality** (required)
  Spotless + Checkstyle
  mvnw verify: unit tests, ArchUnit, integration tests, SpotBugs, JaCoCo
  OpenAPI candidate from JAR
  verified boot JAR
- **contract-check**
  openapi.json drift
  openapi-diff: breaking
- **SonarCloud Backend** (required)
  gate: ≥ 80 % new code
- **docker** (required)
  image ci-<sha>
  boot with PG 18 + Redis 8.10;
  /api/actuator/health UP within 60 s;
  Trivy image scan

### ci-frontend.yml

frontend/**

- **frontend-check** (required)
  pnpm check: gen:api, tsc, oxlint, oxfmt, Vitest, knip;
  build dist/
- **SonarCloud Frontend** (required)
  ≥ 80 % new code
- **e2e-preview**
  Playwright on the PR's dist/

### security.yml

- **gitleaks** (required)
  PR commits
- **workflow-lint**
  actionlint
- **trivy-deps**
  Maven + pnpm
- **trivy-config**
  IaC, Docker
- **deployment-bundle**
  ShellCheck; Compose must hold only the backend service; SSM bundle file allowlist and size

### Other PR workflows, path-filtered

typesafe-smoke (live search contract) · perf-tooling (k6 tooling)
infra-swe5001: terraform test, Checkov, Trivy config (SWE5001 IaC)

### Approval gate

ruleset dev-require-review
- 1 approval incl.
- code-owner review
- stale approvals dismissed
- threads resolved
- branch up to date
- 6 required checks green
- squash merge only

## 2 · After squash merge: push to dev

automatic deploy to dev (not production), no manual step

backend/**, deployment/backend/** changed and ENABLE_EC2_DEPLOY = true

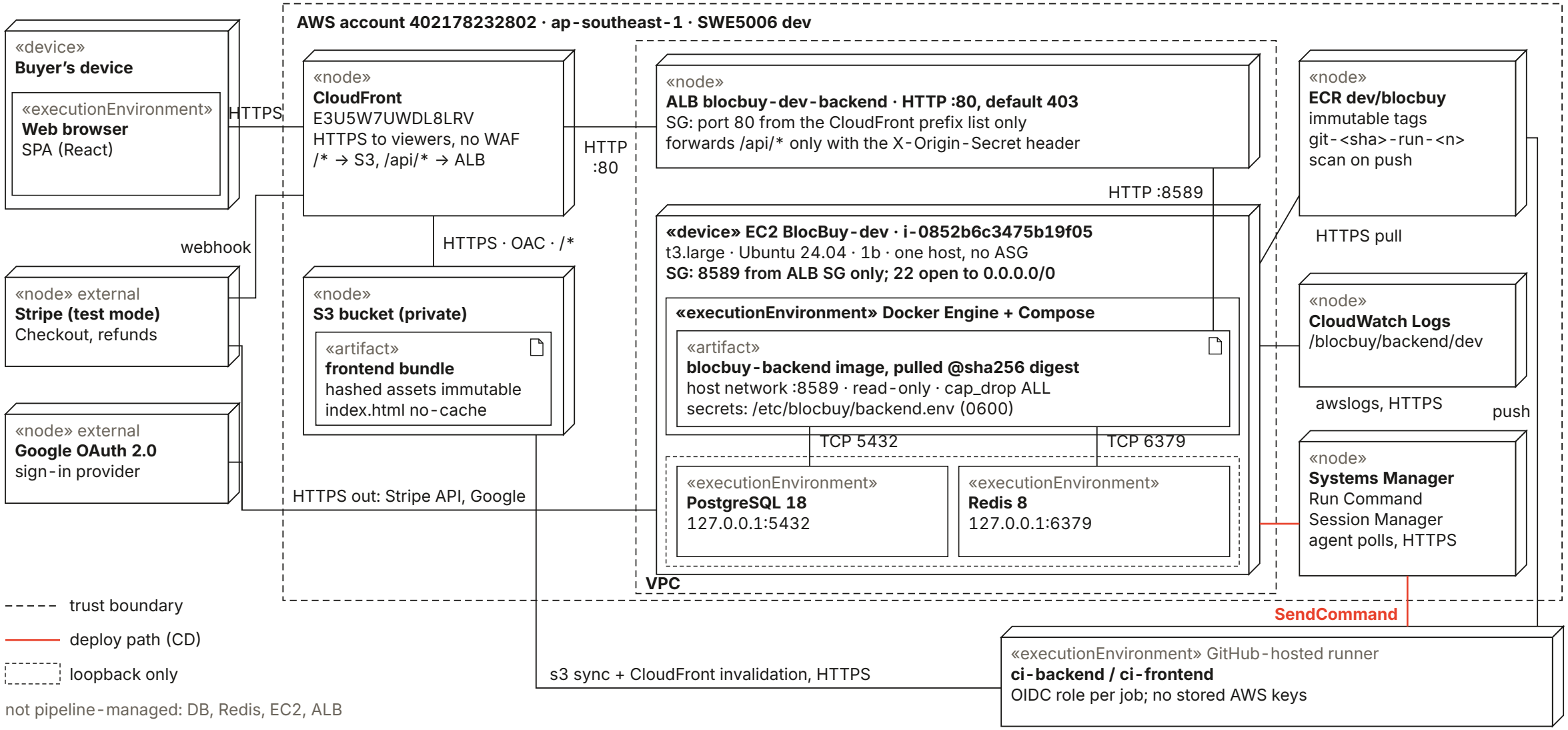
- **package-dev**
  mvnw package -DskipTests:
  merged commit
- **publish-ecr-dev**
  OIDC → ECR dev/blocbuy
  tag git-<sha>-run-<n>
  digest checked in ECR
- **deploy-existing-ec2 · environment backend-dev · OIDC**
  - **pre-check**
    current health
  - **SSM Run Command**
    deploy.sh pulls @sha256; on-host health fails → automatic rollback
  - **public health**
    via CloudFront

frontend/**, contracts/** changed and ENABLE_S3_DEPLOY = true

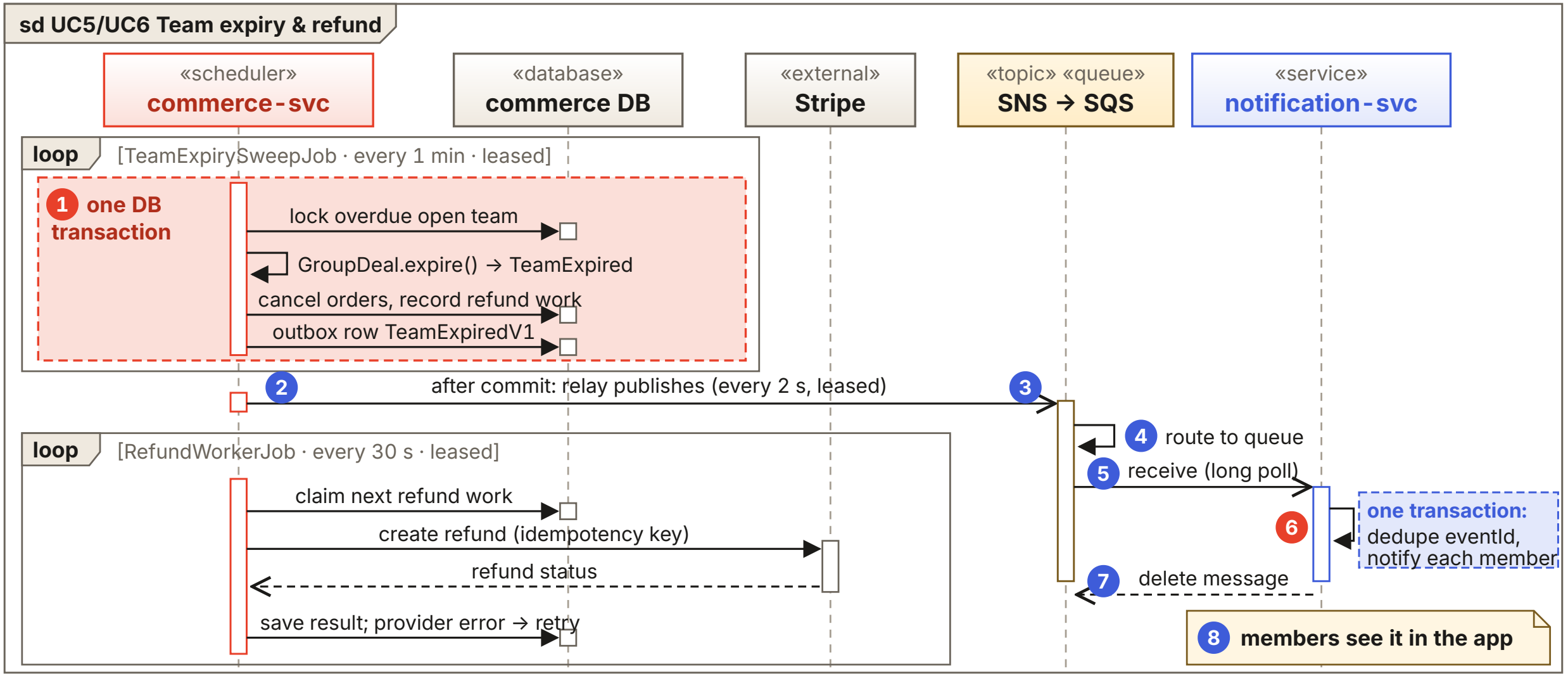
- **package-dev**
  pnpm build:
  dist/ from the merged commit
- **deploy-s3-dev**
  env frontend-dev, OIDC
  hashed assets immutable, index.html no-cache + CloudFront invalidation
- **e2e-dev**
  Playwright smoke against the dev CloudFront URL

**Also on push to dev**
SonarCloud dev baseline (backend, frontend)
gitleaks (security.yml)
weekly, Mon 02:17 UTC: full security.yml

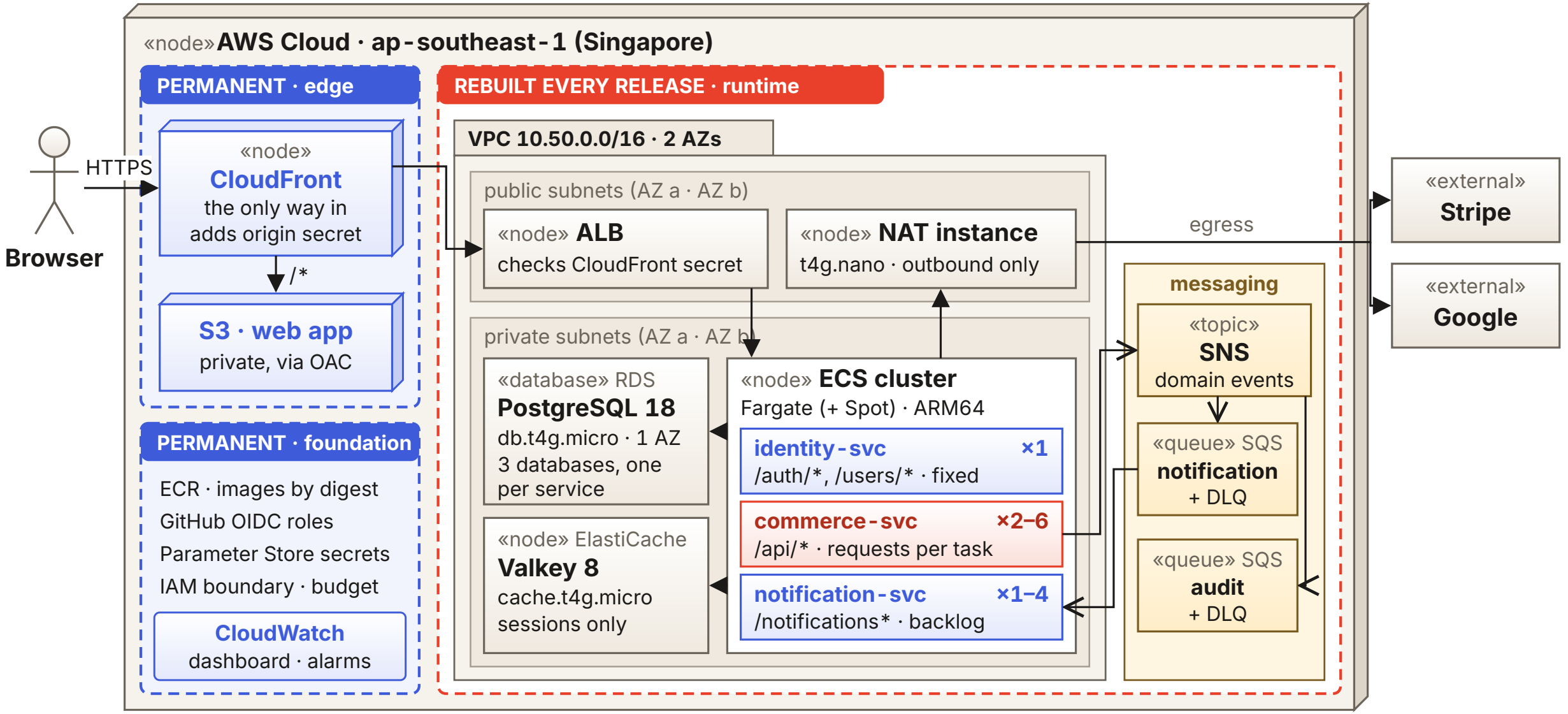
# Dev deployment



# Expiry and refund sequence (ECS)

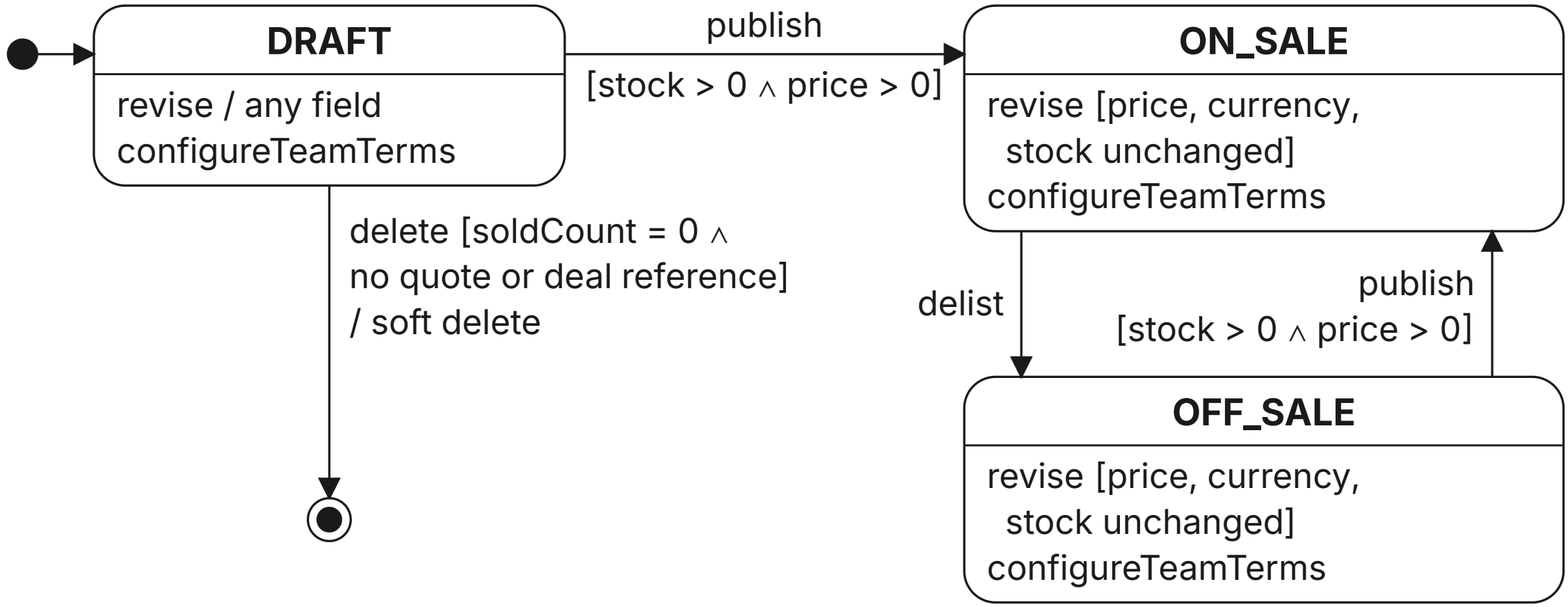


# SWE5001 production topology



# UC9 listing statechart

## stm Listing lifecycle (Product.status)




Nothing returns to DRAFT; any other event → 409, state unchanged.

# Entity-relationship diagram, all tables

